A STUDY IN DEEP LEARNING FOR ASTRONOMY



Clear or Cloudy?

Deep Learning for Sky Monitoring at the Higashi-Hiroshima Observatory

MUHAMMAD WAQAS

Clear or Cloudy?

Deep Learning for Sky Monitoring at the Higashi-Hiroshima Observatory

From all-sky camera images to a convolutional neural network, unsupervised sky types, network interpretation and the Sky Monitor AI application

MUHAMMAD WAQAS

2026

Clear or Cloudy? Deep Learning for Sky Monitoring at the Higashi-Hiroshima Observatory

© 2026 Muhammad Waqas. All rights reserved.

All-sky camera images: Higashi-Hiroshima Observatory, Hiroshima University. Data provided by Prof. Makoto Uemura.

Figures were produced by the author from the project's analysis. Screenshots show the Sky Monitor AI application developed in this work.

Sky Monitor AI is a research prototype. Its verdicts must not be used to operate observatory equipment unattended.

Typeset in TeX Gyre Pagella and Inter.

Preface

Every clear night at an observatory starts with the same question: *can we open the dome?* For most of the history of astronomy, a person answered it by stepping outside and looking up. Small and remotely operated observatories cannot always afford that person, and so the question passes to instruments. This book is the story of teaching one such instrument, the all-sky camera of the Higashi-Hiroshima Observatory, to answer it.

The project began as a course assignment: develop a convolutional neural network that classifies the condition of the sky, fine or cloudy, from the camera's images, and, as extensions, find out whether the classes can be discovered without labels and what the network actually looks at. Those three tasks are covered in Chapters 1 to 4. Along the way the project turned out to be richer than the assignment suggested. The images carried a printed date that would have let a network cheat. The Moon turned clear skies blue and cloudy skies white. The labels, assigned in blocks of an hour or two, were sometimes wrong for whole stretches of a night. And without being told anything, the data separated themselves not only into clear and cloudy skies but also revealed a fifth kind of night: water droplets on the camera's dome.

The final part of the work, described in Chapter 5, turned the trained models into Sky Monitor AI, an application that runs on an ordinary computer and analyses single images, whole nights, Google Drive folders or a live camera, and sets the camera's verdict beside a weather forecast and a satellite picture of the same place and time. Chapter 6 is its user guide, illustrated with screenshots of the real application.

I have tried to write for two readers at once. Students meeting deep learning for the first time will find every method explained from first principles, with the reasons for each decision and the mistakes that were avoided. Astronomers and observatory staff who simply want a working cloud monitor can read

Chapters 1, 5 and 6 and treat the rest as the evidence behind it. Throughout, I have reported results honestly, including the ones that did not work. Small datasets are unforgiving, and knowing why a method fails is often more useful than knowing that another one succeeded.

The data were provided by Prof. Makoto Uemura of Hiroshima University. All analysis was carried out in Google Colab with Python, PyTorch and scikit-learn.

— Muhammad Waqas

How to read this book

Table 1 A map of the book.

Chapter	Question it answers	Read it if you want to…
1. The Sky, the Camera and the Data	What does the camera see, and what can simple measurements tell us?	Understand the data and its traps
2. Teaching a Network to See Clouds	How do we build and train a CNN on 32 nights of images?	Learn the training methods
3. Testing in the Real World	How well do the models work on nights they have never seen?	Judge the system's reliability
4. Learning Without Labels and Looking Inside the Network	Do clear and cloudy skies form natural groups? What does the network look at?	See the extensions of the project
5. From Notebook to Observatory Tool	How do the models become a usable application?	Understand Sky Monitor AI
6. User Guide	How do I use the application?	Install and operate the tool

Each chapter opens with a short summary. Key terms appear in **bold** where they are introduced and are collected in the glossary at the end of the book. Boxes marked *Tip* and *Note* contain practical advice. Figures and tables are numbered by chapter.

Contents

Preface 5

1 The Sky, the Camera and the Data 9

1.1 Why automate sky monitoring? 9
1.2 The all-sky camera 10
1.3 The dataset 12
1.4 The date-stamp shortcut and the sky mask 15
1.5 What simple physics can tell us 16
1.6 How well do simple features classify? 20
1.7 Splitting the data honestly 22
1.8 Summary 24

2 Teaching a Network to See Clouds 25

2.1 From hand-made features to learned features 25
2.2 How a convolutional network works 26
2.3 Data augmentation: showing the network what can change 29
2.4 Model A: a small CNN trained from scratch 30
2.5 Model B: transfer learning with ResNet-18 34
2.6 Choosing the final models 39
2.7 Summary 40

3 Testing in the Real World 43

3.1 Measuring performance 43
3.2 Test results of the individual models 45
3.3 A fair comparison with the baseline 48
3.4 Night by night 49
3.5 The ensemble: strength in disagreement 50
3.6 The Moon: measuring a hidden variable 52
3.7 Temporal smoothing 55
3.8 The error gallery 56
3.9 Following the labels through the night 57
3.10 Summary 59

4 Learning Without Labels and Looking Inside the Network 61
4.1 Part I — Learning without labels ... 61
4.2 Part II — Looking inside the network ... 68
4.3 Summary ... 80

5 From Notebook to Observatory Tool 81
5.1 What an observatory tool must do ... 81
5.2 Architecture ... 82
5.3 Leaving PyTorch behind: ONNX export ... 84
5.4 Evidence maps without gradients ... 86
5.5 Sky type and the out-of-distribution warning ... 87
5.6 From numbers to a decision ... 88
5.7 Getting images in ... 89
5.8 Packaging: one zip, one double-click ... 92
5.9 Testing the tool ... 93
5.10 Limitations ... 97
5.11 Future work ... 97
5.12 Closing ... 98

6 User Guide: Sky Monitor AI Step by Step 99
6.1 Before you start ... 99
6.2 Finding your way around ... 101
6.3 Classify a sky ... 103
6.4 Night timeline & batch analysis ... 109
6.5 Live camera ... 116
6.6 Weather & satellite ... 121
6.7 Sky quiz ... 125
6.8 Explore sky types ... 126
6.9 Results ... 129
6.10 Troubleshooting ... 132
6.11 Quick reference ... 133

Glossary 135

References 139

Acknowledgements 141

CHAPTER 1

The Sky, the Camera and the Data

IN THIS CHAPTER

An observatory can only work when the sky is clear, and someone has to decide when that is. This chapter introduces the all-sky camera of the Higashi-Hiroshima Observatory, the 8,806 frames it recorded over 32 nights, and the first lessons the data taught: that consecutive frames are far from independent, that a printed date can become a dangerous shortcut, and that simple physical measurements of the sky already separate clear from cloudy skies surprisingly well, but not well enough.

1.1 Why automate sky monitoring?

An optical telescope is a light bucket. It collects photons from distant stars and galaxies, and every photon that is scattered or absorbed on its way through the Earth's atmosphere is lost. Clouds are the most obvious thief. A thin veil of cirrus can dim a star by a few tenths of a magnitude and ruin precise photometry; a thick layer of stratus blocks the sky entirely. On top of the scientific loss, clouds often arrive together with humidity, dew and rain, and water is the natural enemy of mirrors, electronics and mechanical drives.

For these reasons every observatory needs a rule for when to open the dome and when to close it. At a staffed observatory, an observer looks at the sky, checks weather sensors and makes the call. At a small university observatory, or one operated remotely, there may be nobody on site for most of the night. The decision then has to be made from instruments, and this is where an **all-**

sky camera becomes valuable. It is a small, inexpensive camera with a fisheye lens that photographs the entire visible sky at regular intervals. A human can glance at its latest frame and see at once whether the sky is clear. The goal of this project is to let a computer make that glance.

The task sounds easy because it is easy for us. A child can tell a starry sky from an overcast one. Yet several things make it genuinely difficult for a machine:

- **The night sky is dark, and its brightness changes enormously.** A moonless clear sky and a cloudy sky lit by the town below can differ by a factor of ten in brightness, but so can a moonless and a moonlit clear sky.
- **Clouds come in every form.** They can be thick or thin, uniform or broken, high or low, dark against a bright background or bright against a dark one.
- **The camera sees fixed objects too.** A mast, part of the dome and the horizon appear in every frame and carry no information about the weather.
- **The labels people give are coarse.** As we will see in Chapter 3, humans tend to label whole stretches of a night at once, while the sky changes from minute to minute.

The assignment that started this work asked for a convolutional neural network that classifies the frames as *fine* (clear) or *cloudy*. That binary decision maps directly onto the operational question: open or keep closed. The rest of this book shows how far that simple framing can be pushed, and where it breaks down.

> **KEY IDEA**
> The aim is not to beat a human at recognising clouds. It is to make a reliable, tireless and consistent decision every minute of every night, and to know when that decision should not be trusted.

1.2 The all-sky camera

The images used in this book come from a small auxiliary camera at the **Higashi-Hiroshima Observatory** of Hiroshima University in western Japan. Each frame is a colour photograph of 600 × 600 pixels. A caption printed along

the bottom edge of every frame records the settings: an aperture of f/3.2, an exposure time of 15 seconds, a sensitivity of ISO 3200 and automatic white balance. The same caption records the date and the local time at which the frame was taken.

1.2.1 The fisheye view

The lens is a fisheye: it maps the whole hemisphere of the sky, from the zenith straight overhead down to the horizon in every direction, onto a circle inside the square image. The zenith sits near the centre of the circle and the horizon lies along its rim. Letters printed just outside the circle mark north at the top and east on the left. East appears on the left, not the right as on a map, because we are looking *up* at the sky rather than down at the ground.

This projection has consequences for everything that follows. Objects near the zenith appear large and undistorted, while objects near the horizon are squeezed into a thin ring at the edge of the circle. A cloud bank on the horizon therefore occupies only a small fraction of the image even when it covers a large range of azimuth. The edge of the circle is also darker, because of vignetting in the lens and because light from near the horizon passes through more atmosphere. Outside the circle the image is black, apart from the printed captions.

Two fixed objects appear in every frame. On the left a mast carrying another instrument pokes into the field of view, and on the right the curved edge of a dome or building cuts into the circle. A well-trained classifier must learn to ignore both.

1.2.2 What a 15-second exposure sees at night

At night, a 15-second exposure at ISO 3200 is long enough to record a surprising amount of light. Three sources dominate:

1. **Stars.** On a clear moonless night hundreds of stars appear as tiny points of light. Their presence is the most direct proof that the sky is clear.
2. **The Moon.** Moonlight is reflected sunlight. When it is scattered by the air molecules of a clear sky it behaves exactly like daylight, preferring short wavelengths (Rayleigh scattering), so a moonlit clear sky photographed

with a long exposure looks *blue*, almost like a daytime sky. The Moon itself appears as a bright glare blob, often with faint rings from internal reflections in the lens.

3. **Artificial light.** Street lamps in the town below illuminate the underside of low clouds. Because many lamps emit warm orange light, clouds lit from below appear orange or brown.

The result is that the colour and brightness of a frame depend as much on the Moon and the town as on the clouds. A bright frame can be a moonlit clear sky or an overcast sky lit by the Moon; a dark frame can be a clear moonless sky or a thick cloud deck on a moonless night far from lights. This ambiguity is the central difficulty of the whole project, and it will return in every chapter.

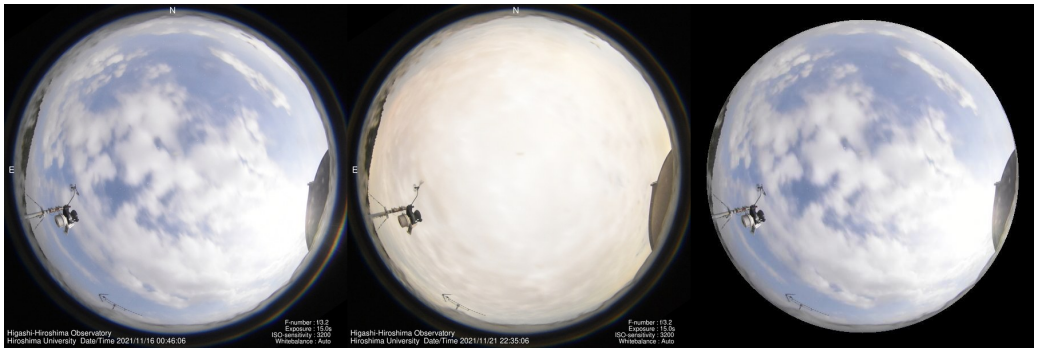


Figure 1.1 Three frames from the dataset. Left: a frame labelled *fine*, taken in moonlight at 00:46 on 16 November 2021 — note the broken cloud despite the label. Centre: a frame labelled *cloudy* from 21 November 2021, a bright overcast sky. Right: the first frame again after applying the sky mask described in Section 1.4, which blacks out everything outside the sky circle, including the date stamp.

1.3 The dataset

1.3.1 Where the images came from

The images were provided through a shared Google Drive folder named *WeatherClasssifier*. It contains two subfolders, which are the labels of the classification task: `Data_fine` for frames judged clear and `Data_cloudy` for frames judged cloudy. Someone had already sorted the frames, so the problem is a supervised one from the start.

Counting the files directly in the Drive listing gives 3,782 fine and 4,984 cloudy frames, 8,766 in total. When the folder was copied into the working environment, the fine folder contained 3,822 files instead of 3,782. The difference turned out to be duplicates: a handful of files exist twice under names such as `211116231058(1).jpg` and `211116231058(2).jpg`. The final working set therefore contains **8,806 frames: 4,984 cloudy and 3,822 fine**. Because duplicates share the same night, they cannot leak information between training and test data (Section 1.6), and they were left in place.

The classes are moderately imbalanced: about 57% of the frames are cloudy. A classifier that always answered "cloudy" would already be right 57% of the time. Any useful model must clearly beat this **majority baseline**.

1.3.2 File names are timestamps

Every file name is a timestamp in the form `YYMMDDhhmmss`. The file `211116011922.jpg`, for example, was taken on 16 November 2021 at 01:19:22. The times are in **local Japan time (UTC+9)**. Two observations confirm this. First, the frames of each night begin around 20:00 and continue past midnight, which only makes sense as local evening. Second, the time printed on an image agrees with its file name to within about a minute; the file `211124194628.jpg`, for instance, carries the caption *Date/Time 2021/11/24 19:45:05*. The small offset probably reflects the start of the exposure versus the moment the file was written.

Timestamps turn out to be one of the most useful properties of the dataset. They allow us to group frames by night, to compute the position of the Moon for every frame (Chapter 3), and to follow a night minute by minute.

1.3.3 Nights, not frames, are the real samples

Because one night spans many hours, it is convenient to define an **observing night** by shifting each timestamp back by 12 hours and taking the date. A frame taken at 01:19 on 16 November then belongs to the night of 15 November, together with the frames of the previous evening. With this convention the dataset spans 32 observing nights between 15 November and 19 December 2021.

Table 1.1 Number of frames per observing night and class. Fine frames come from 12 nights, cloudy frames from 20; four nights contain both.

Night	Cloudy	Fine	Night	Cloudy	Fine
2021-11-15	0	293	2021-11-30	396	0
2021-11-16	121	359	2021-12-01	208	0
2021-11-17	165	0	2021-12-02	206	0
2021-11-18	0	277	2021-12-03	279	0
2021-11-19	0	532	2021-12-04	385	0
2021-11-20	0	203	2021-12-05	56	0
2021-11-21	534	0	2021-12-07	229	0
2021-11-22	176	421	2021-12-09	187	0
2021-11-24	298	240	2021-12-11	451	0
2021-11-25	0	534	2021-12-15	33	0
2021-11-26	99	112	2021-12-16	358	0
2021-11-27	0	538	2021-12-17	477	0
2021-11-28	0	203	2021-12-18	145	0
2021-11-29	0	110	2021-12-19	181	0

Two features of this table shape the entire project.

First, frames within a night are highly correlated. The camera takes a frame roughly once a minute, and the sky rarely changes much in a minute. Two consecutive frames are almost identical. A dataset of 8,806 frames therefore does not contain 8,806 independent examples. A better estimate of the number of truly independent samples is the number of nights, about 32, or perhaps a few times that if we count distinct weather episodes within nights. This is a small number for training a neural network, and it explains much of the behaviour we will meet in Chapter 2.

Second, the classes are partly separated in time. Every fine frame was taken before 30 November, while cloudy frames continue until 19 December. Any property that changes with the date, such as the phase of the Moon, the time of sunset or the printed date itself, is therefore correlated with the label. A model

can exploit such correlations without learning anything about clouds. This danger is the subject of the next section.

1.4 The date-stamp shortcut and the sky mask

1.4.1 How a network could cheat

Look again at the raw frames. Along the bottom edge each one carries a printed caption with the observatory's name, the exposure settings and, crucially, the date and time. A convolutional network sees the whole image. If every fine frame is from November and most cloudy frames are from December, then the few pixels that spell out "2021/12" are an almost perfect predictor of the label. A network trained on raw frames could reach a very high accuracy simply by reading the date.

This kind of failure is known as **shortcut learning**: a model finds a feature that predicts the label in the training data but has nothing to do with the real task. Shortcut learning is not a rare curiosity. It has been documented in medical imaging, where networks learned to recognise the hospital that produced an X-ray instead of the disease, and in object recognition, where networks learned the background instead of the object. It is dangerous precisely because the model's accuracy looks excellent. Only when the model meets new data, here a frame from a new month, does it fail.

> **KEY IDEA**
> High accuracy proves nothing until you know *what* the model is using. Every pixel that carries information about the label without carrying information about the sky is a potential shortcut.

1.4.2 The circular sky mask

The cure is simple: remove everything that is not sky. Before any image reaches a model, every pixel outside a circle centred on the image is set to black. The circle's radius is a fraction of the image width,

$$r_{\text{mask}} = 0.455 \times W$$

which for a 600-pixel frame gives a radius of about 273 pixels. This radius was chosen by inspecting real frames: it keeps the whole sky while removing the printed captions, the north and east markers and the black margin. The right panel of Figure 1.1 shows the result. The date stamp is gone; the mast and the dome edge remain, because they lie inside the sky circle, and the network will have to learn to ignore them on its own. Chapter 4 checks, with a quantitative test, that it does.

The mask is applied to every image in every experiment of this book: during training, during evaluation, in the unsupervised analysis and in the final application. Because the mask is a circle centred on the image, it has a convenient property: rotating or flipping a masked image leaves the black region unchanged. This matters for data augmentation in Chapter 2, where images are rotated by arbitrary angles.

1.4.3 How much of the image is sky?

The fraction of the square image that lies inside the mask is the ratio of the circle's area to the square's area:

$$1 - f = \frac{\pi r_{\mathrm{mask}}^2}{W^2} = \pi \times 0.455^2 \approx 0.650$$

So about 65% of every image is sky, and a fraction f ≈ 0.35 of the pixels are forced to zero. This number looks like a technical detail, but it immediately explained a puzzling result in the next section.

1.5 What simple physics can tell us

Before training any neural network it is worth asking how far simple, physically motivated measurements can go. If two numbers computed from each image already separate clear from cloudy skies perfectly, a neural network is unnecessary. If they fail, the way they fail tells us what the network will have to learn. This section describes that exploration, including a mistake that turned out to be instructive.

1.5.1 A first plot, and a straight line that meant nothing

The first idea was to measure two numbers for each image: the **mean brightness**, which should be higher when clouds scatter moonlight and town light back towards the camera, and the **standard deviation of the pixel values**, a measure of contrast, which should be higher for broken cloud with bright and dark patches. Both were computed on grey-scale versions of 1,500 randomly chosen masked frames, and plotted against each other.

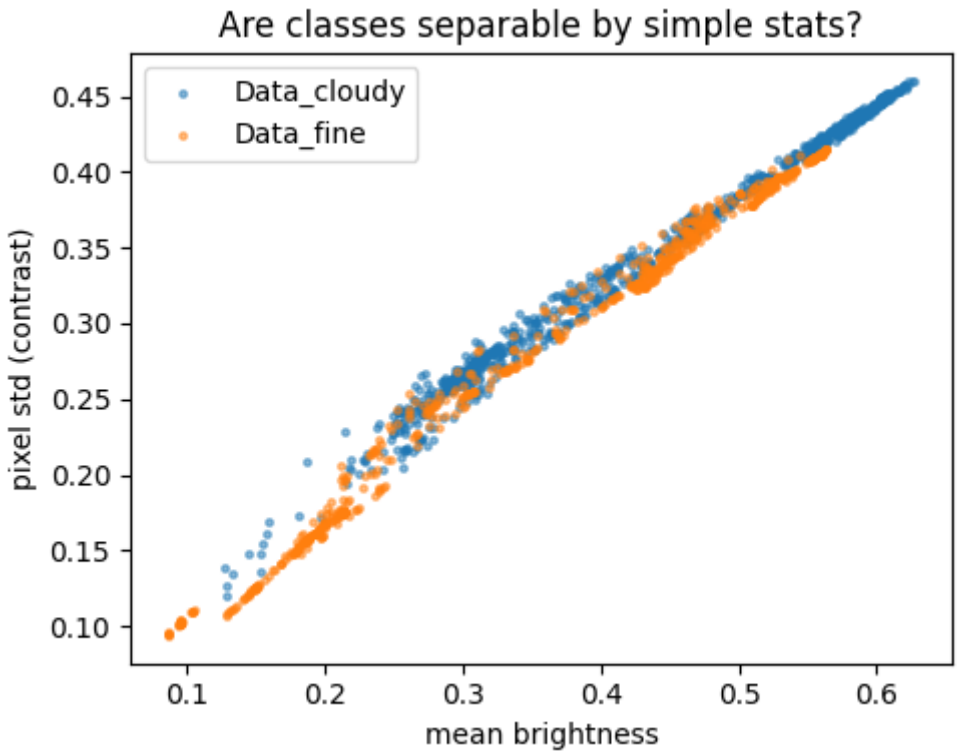


Figure 1.2 Mean brightness against pixel standard deviation for 1,500 masked frames, computed over the whole square image. Almost every point lies on a straight line through the origin with slope close to 0.73. The line is an artefact of the black mask, not a property of the sky.

The result, Figure 1.2, was striking: nearly every frame lies on a single straight line. At first sight this suggests that brightness and contrast are tightly linked properties of the sky. In fact the line is produced entirely by the mask.

To see why, suppose that a fraction f of the pixels are set to zero by the mask and that the sky pixels have mean b and standard deviation σ_{sky}. The mean m and the variance σ^2 of the whole image are then

$$m = (1 - f)\, b$$

$$\sigma^2 = (1 - f)\, \sigma_{\text{sky}}^2 + f(1 - f)\, b^2$$

The first term of the variance comes from the variation within the sky; the second comes purely from the contrast between the black corners and the sky. Since the first term can never be negative, the image standard deviation has a lower bound:

$$\sigma \;\geq\; \sqrt{f(1-f)}\; b \;=\; m\sqrt{\frac{f}{1-f}}$$

With f ≈ 0.35 from Section 1.4, the square root evaluates to $\sqrt{(0.35/0.65)} \approx 0.73$. Reading the plot confirms it: at a mean of 0.62 the standard deviation is about 0.46, a ratio of 0.74. Every frame sits on or just above the line $\sigma = 0.73$ m. The slope of the line is a property of the mask geometry, and only the small distance of each point *above* the line carries information about the sky itself.

This little derivation is worth remembering beyond this project. Whenever an image contains a constant region (a mask, a border, a frame), statistics computed over the whole image mix the content with the geometry of that region. The fix is to compute statistics over the region of interest only.

1.5.2 Statistics of the sky pixels only

The analysis was repeated using only the pixels inside the sky circle, and two further features were added, giving four numbers per image:

1. **Mean sky brightness**: the average grey value of the sky pixels.
2. **Sky standard deviation**: the contrast within the sky.
3. **Mean gradient**: the average magnitude of the brightness gradient after light smoothing. Cloud edges and cloud texture produce gradients; a uniform clear or uniform overcast sky does not.

4. **Star-like peaks**: the number of small bright points standing out from their surroundings. The image is high-pass filtered by subtracting a 9 × 9 local mean, and a pixel counts as a peak if it is the maximum of its 5 × 5 neighbourhood and exceeds its local background by more than 0.06 (on a 0–1 scale). Stars are exactly such points, so a high count means a clear sky.

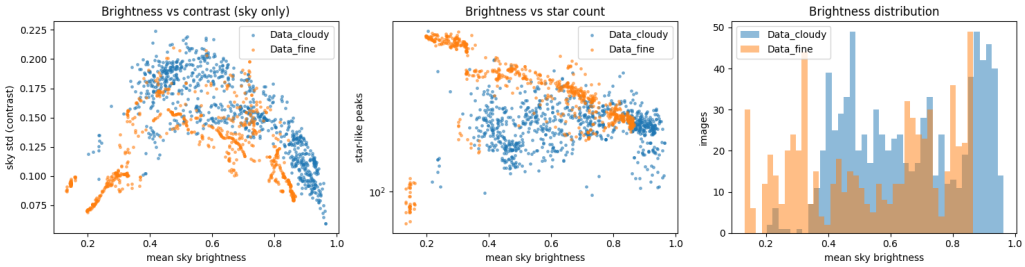


Figure 1.3 Sky-only statistics for 1,500 frames. Left: mean sky brightness against sky contrast. Centre: brightness against the number of star-like peaks (logarithmic axis). Right: distribution of brightness for the two classes. Fine frames form thin continuous tracks — individual clear nights seen minute by minute — while cloudy frames scatter.

Figure 1.3 shows the result, and it is rich with physics.

Left panel: brightness against contrast. Fine frames (orange) form thin, continuous tracks, while cloudy frames (blue) form a scattered cloud of points. Each orange track is one clear night seen minute by minute: as the Moon rises or twilight fades, brightness and contrast change smoothly, so consecutive frames trace a path. Cloud cover changes erratically from minute to minute, so cloudy frames scatter. At intermediate brightness (0.4 to 0.7) cloudy frames have the higher contrast: broken cloud, with bright clouds next to dark gaps. At the highest brightness (above 0.85) cloudy frames have the *lowest* contrast: a uniform overcast deck lit by the Moon. Almost no fine frames are that bright.

Centre panel: brightness against star count. This is the most informative view. Between brightness 0.3 and 0.75 the classes separate cleanly: fine frames form an upper band with many star-like peaks, cloudy frames a lower band with few. This is exactly the physics we would expect: if you can see stars, the sky is clear. Above a brightness of about 0.8, however, the two bands merge. A very bright sky washes out the stars, whether it is bright because of cloud or because of the Moon, so star counting loses its power. We can predict already

that any classifier will struggle most with bright, moonlit frames. Chapter 3 confirms this prediction.

One cluster of fine frames looks odd: around brightness 0.15 there is a tight group with very few peaks, about 100. A dark clear sky should show the *most* stars, not the fewest. Possible explanations are dew or frost on the lens, a different camera setting, or images placed in the wrong folder. The tightness of the group suggests a single night or sequence. It is a reminder that real data contain surprises worth checking by eye.

Right panel: the brightness histogram. The histograms are spiky, and each spike is mostly a single night. This is the correlation discussed in Section 1.3.3 seen from another angle: frames come in large, internally similar batches, so the effective sample size is much smaller than the number of frames.

1.5.3 Why the Moon matters so much

The same analysis also suggested a hidden variable. Full moon fell on 19 November 2021, in the middle of the run of fine nights (18 to 22 November), and new moon on 4 December, during the long cloudy period. Bright clear skies in this dataset are therefore mostly moonlit skies, and the class split partly follows the lunar cycle. A model that learns "bright means clear" or "bright means cloudy" can look successful on some nights and fail badly on others. Chapter 3 measures this effect precisely by computing the Moon's altitude and phase for every frame.

1.6 How well do simple features classify?

To turn these impressions into numbers we need a measure of how well a single quantity separates two classes, independent of any particular threshold.

1.6.1 The area under the ROC curve

Suppose we classify a frame as fine whenever some score s exceeds a threshold t. As t varies, we trade the **true positive rate** (the fraction of fine frames correctly called fine) against the **false positive rate** (the fraction of cloudy frames wrongly called fine). Plotting one against the other gives the receiver

operating characteristic (ROC) curve, and the area under it, the **AUC**, summarises separability in a single number between 0 and 1.

The AUC has an intuitive interpretation. It equals the probability that a randomly chosen fine frame receives a higher score than a randomly chosen cloudy frame:

$$\mathrm{AUC} = P\left(s_{\mathrm{fine}} > s_{\mathrm{cloudy}}\right)$$

An AUC of 0.5 means the score is useless, a coin flip; an AUC of 1.0 means perfect separation. Because a feature can separate the classes in either direction (higher brightness can mean cloudier), the value reported below is $\max(\mathrm{AUC}, 1 - \mathrm{AUC})$.

Table 1.2 How well each sky-only feature separates fine from cloudy frames (1,500 random frames).

Feature	AUC	Physical interpretation
Mean sky brightness	0.654	Weakest: the Moon brightens clear and cloudy skies alike
Sky standard deviation	0.741	Patchy cloud raises contrast; uniform overcast and dark clear skies both have low contrast
Mean gradient (texture)	0.775	Cloud edges create gradients; a more direct measure of cloud structure
Star-like peaks	**0.810**	Best: visible stars mean a clear sky, unless moonlight washes them out

The ranking is itself a result. Brightness, the most obvious measurement, is the weakest because of the Moon. Star counting, the measurement closest to the astronomer's own test ("can I see stars?"), is the strongest. No single feature exceeds an AUC of about 0.81.

1.6.2 Combining the features

Individual AUCs do not tell us how well the features work together. Two classifiers were therefore trained on the four features at once: **logistic regression**, which draws a straight boundary in feature space, and **gradient boosting**, an ensemble of small decision trees that can draw curved boundaries. Both were evaluated with **grouped five-fold cross-validation**: the

frames were divided into five groups by night, and each group was predicted by a model trained on the other four. Every prediction is therefore made for a night the model has never seen.

Table 1.3 Combining the four features, evaluated on unseen nights.

Method	Accuracy	AUC
Always answer "cloudy"	0.569	0.500
Best single feature (star-like peaks)	—	0.810
Logistic regression, 4 features	0.822	0.858
Gradient boosting, 4 features	**0.845**	**0.870**

Combining the features adds about 0.05 to the AUC, so they carry partly independent information: stars, texture and contrast each catch cases the others miss. Gradient boosting, however, is only slightly better than logistic regression. A flexible classifier gains little from these four numbers, which tells us that the limit lies in the *features*, not in the classifier. Global statistics throw away *where* things are in the image. A cloud bank low on the horizon and a cloud overhead can give the same mean and standard deviation; stars visible in only half of the sky are counted the same as stars spread evenly. That spatial information is exactly what a convolutional network can exploit.

This sets the bar for the rest of the book. A CNN that reaches only about 85% has learned nothing beyond simple brightness and texture statistics. A CNN that clearly exceeds it must be using spatial structure. Chapter 3 will also show that this cross-validated figure was flattering: on the specific test nights used later, the same hand-crafted model reaches only 77.1%.

1.7 Splitting the data honestly

The final preparatory step is to decide which frames are used for training, which for choosing models (validation) and which for the final, untouched evaluation (test). The usual recipe is a random split, for example 70% / 15% / 15%. For this dataset a random split would be seriously misleading.

Recall that consecutive frames are nearly identical. With a random split, the frame taken at 21:14 might land in the training set and the frame taken at 21:15 in the test set. The model would then be "tested" on near-copies of its own training data, and the test accuracy would overestimate how well the model handles a genuinely new night. This is a form of **data leakage**.

The remedy is to split by **night**: all frames of an observing night go into the same set. The test set then contains only nights the model has never seen, which is exactly the situation the model faces in operation. To make sure that every set contains both classes, the nights were assigned separately for each class, according to the class that dominates each night.

One further refinement made the test more demanding. The four nights that contain both fine and cloudy frames (16, 22, 24 and 26 November) are the most valuable for evaluation, because within one night the Moon, the season and the camera settings are the same and only the clouds differ. The night of 24 November, with 298 cloudy and 240 fine frames, was therefore placed in the test set.

Table 1.4 The final split by observing night (frames: cloudy / fine).

Set	Nights	Frames	Cloudy	Fine
Training	21 nights	5,705	3,150	2,555
Validation	11-20, 11-22, 12-03, 12-07, 12-09	1,495	871	624
Test	11-15, 11-24, 11-29, 12-11, 12-15, 12-19	1,606	963	643

The test set contains two purely fine nights (15 and 29 November), three purely cloudy nights (11, 15 and 19 December) and the mixed night of 24 November. It also contains, as Chapter 3 will reveal, the full-moon night of 19 December, which proved to be the hardest night in the dataset.

To compensate for the moderate class imbalance during training, each class was given a weight inversely proportional to its frequency:

$$w_c = \frac{N_{\text{train}}}{K\, N_c}$$

where $K = 2$ is the number of classes and N_c the number of training frames in class c. This gives $w_{\text{cloudy}} = 5705 / (2 \times 3150) = 0.91$ and $w_{\text{fine}} = 5705 / (2 \times 2555) = 1.12$, so each fine frame counts about 1.2 times as much as a cloudy one in the loss. The effect is small, but it prevents a model from scoring well simply by leaning towards the majority class.

1.8 Summary

This chapter built the foundation for everything that follows.

- The **all-sky camera** records 600 × 600 colour frames with a fisheye lens, once a minute, through the night. Brightness and colour depend on clouds, the Moon and the town's lights all at once.
- The **dataset** contains 8,806 frames from 32 nights, 57% of them cloudy. File names are local timestamps, which lets us group frames by night and later compute the Moon's position.
- Frames within a night are **strongly correlated**, so the effective number of independent samples is close to the number of nights, not the number of frames.
- A printed **date stamp** would let a network cheat, because the classes come from different weeks. A circular **sky mask** removes it, and the mask is used everywhere.
- A plot that seemed to show a law of the sky was an **artefact of the mask**, a lesson in computing statistics only over the region of interest.
- **Sky-only features** carry real physics: star counts are the best single cue (AUC 0.81), brightness the worst (0.65) because of the Moon. Combined, they reach 84.5% accuracy in cross-validation, but they cannot see *where* things are in the image.
- The data were **split by night**, with a mixed night in the test set, so that every evaluation in this book measures performance on nights the models have never seen.

With this groundwork in place we can turn to the central tool of the project: the convolutional neural network.

CHAPTER 2

Teaching a Network to See Clouds

IN THIS CHAPTER

Convolutional neural networks learn their own features from pixels instead of relying on hand-made measurements. This chapter explains how they work from first principles, then trains two kinds: a small network built from scratch and ResNet-18, a large network pre-trained on millions of everyday photographs. The central lesson is a surprising one. With only about two dozen training nights, letting more of a network learn makes it worse at judging new nights, and the best results come from keeping most of the pre-trained network frozen.

2.1 From hand-made features to learned features

Chapter 1 ended with four carefully designed numbers per image: brightness, contrast, texture and star count. Together they reached about 85% accuracy and then stopped improving, because they describe the image as a whole and ignore where things are. One could keep inventing features: star counts in the northern half, cloud texture near the zenith, colour ratios near the horizon. Each new feature would require a new idea and a new piece of code, and there is no guarantee that the important patterns are ones we would think of.

A **convolutional neural network** turns this around. Instead of being told which features to compute, it learns them from labelled examples. Its first layers learn to detect simple local patterns such as edges, spots and colour transitions. Later layers combine these into larger patterns such as cloud edges, star fields or the soft glow of moonlight. The final layer combines everything

into a single decision. All of these detectors are adjusted automatically during training to make the decisions as accurate as possible.

This section explains the building blocks from first principles. Readers who already know how CNNs work can skip to Section 2.3.

2.2 How a convolutional network works

2.2.1 Images as arrays of numbers

To a computer, a colour image of height H and width W is an array of H × W × 3 numbers: one red, one green and one blue intensity per pixel. In this project every masked frame is resized to 224 × 224 pixels, the standard input size of the networks used, so each input contains 224 × 224 × 3 = 150,528 numbers. Before entering the network the values are scaled to the range 0–1 and then normalised channel by channel, by subtracting a mean and dividing by a standard deviation. The values used, (0.485, 0.456, 0.406) for the means and (0.229, 0.224, 0.225) for the standard deviations, are those of the ImageNet dataset on which ResNet-18 was pre-trained; using the same normalisation for all models keeps the comparison fair.

2.2.2 Convolution: a sliding pattern detector

The core operation is the **convolution**. A small array of weights, called a **filter** or **kernel**, typically 3 × 3 pixels and spanning all input channels, slides across the image. At each position the filter's weights are multiplied by the pixel values beneath them and summed, and a bias is added. Written out for one output channel,

$$y_{i,j} = b + \sum_{c} \sum_{u=-1}^{1} \sum_{v=-1}^{1} w_{c,u,v}\, x_{c,\, i+u,\, j+v}$$

where x is the input, w the filter weights, b the bias and (i, j) the position. The result is a new image, called a **feature map**, whose value at each position says how strongly the local patch resembles the pattern encoded in the filter. A filter with positive weights on the left and negative weights on the right responds to

vertical edges; a filter with a positive centre and negative surround responds to small bright spots, exactly the shape of a star.

A convolutional layer contains many filters, each producing its own feature map. The first layer of the small CNN in this chapter has 32 filters, so it turns a 3-channel image into a 32-channel stack of feature maps. Two properties make convolution well suited to images. **Locality**: each output depends only on a small neighbourhood, matching the fact that pixels close together are related. **Weight sharing**: the same filter is used at every position, so a star detector learned in one corner works everywhere. A 3×3 filter on a 3-channel input has only $3 \times 3 \times 3 = 27$ weights plus a bias, however large the image.

2.2.3 Non-linearity, normalisation and pooling

A stack of convolutions alone would still be a linear operation, unable to represent complex decisions. After each convolution a simple non-linear function is therefore applied to every value. The most common is the **rectified linear unit**,

$$\mathrm{ReLU}(z) = \max(0, z)$$

which keeps positive responses and sets negative ones to zero. Combining many such layers lets the network represent highly non-linear functions.

Batch normalisation is usually inserted between the convolution and the ReLU. During training it rescales each channel so that, across the current mini-batch of images, its values have zero mean and unit variance, followed by a learned scale and shift. This keeps the numbers in a comfortable range and makes training faster and more stable. During evaluation, the network uses running averages of the means and variances collected during training. This detail turns out to matter in Section 2.6.

Max pooling shrinks each feature map by keeping only the largest value in every 2×2 block. It halves the height and width, reduces computation and makes the representation slightly tolerant to small shifts. After several pooling steps, each value in a deep feature map summarises a large region of the original image, its **receptive field**. Early layers see a few pixels; late layers see a large part of the sky.

2.2.4 From feature maps to a decision

At the end of the convolutional layers the network holds a stack of small feature maps. **Global average pooling** reduces each map to its average value, giving one number per channel. A final **fully connected** (linear) layer then computes one score, or **logit**, per class:

$$z_k = \sum_c W_{k,c}\, \bar{A}_c + b_k$$

where $\bar{A}_c$ is the average of channel c. This particular design, average pooling followed by a single linear layer, has a valuable consequence that Chapter 5 exploits: the decision can be decomposed exactly into contributions from each location in the image.

The logits are converted into probabilities with the **softmax** function,

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

so that the two probabilities, P(cloudy) and P(fine), are positive and sum to one. Throughout this book, P(clear) means the softmax probability of the *fine* class.

2.2.5 Learning: loss, gradients and optimisers

Training means adjusting all weights so that the predicted probabilities match the labels. The mismatch is measured by the **cross-entropy loss**. For an image with true class y,

$$L = -\, w_y \log p_y$$

where w_y is the class weight from Section 1.7. The loss is zero when the network assigns probability 1 to the correct class and grows without limit as that probability approaches zero. A confident mistake is punished much more heavily than a hesitant one, which will help explain the behaviour of the validation curves below.

The **gradient** of the loss with respect to every weight is computed by **backpropagation**, a systematic application of the chain rule from the output back through every layer. The weights are then nudged in the direction that

reduces the loss. Rather than computing the gradient over all training images at once, the network processes **mini-batches** of 32 images; one pass through all training images is an **epoch**.

The update rule used here is **AdamW**, a variant of the Adam optimiser. Adam keeps running averages of each weight's gradient and squared gradient, giving every weight its own effective step size; the "W" adds **weight decay**, a gentle pull of every weight towards zero (here with coefficient 10^{-4}) that discourages extreme values. The **learning rate**, the overall step size, follows a **cosine schedule**: it starts at its maximum and decreases smoothly towards zero along half a cosine period over the planned number of epochs, so that early epochs explore and late epochs refine.

2.2.6 Validation and early stopping

A network with hundreds of thousands of weights can memorise its training images. Its training loss then keeps falling while its performance on new images gets worse, a phenomenon called **overfitting**. To detect it, the validation set from Section 1.7 is evaluated after every epoch. Training stops when the validation loss has not improved for a fixed number of epochs (the **patience**, here 5), and the weights from the epoch with the lowest validation loss are restored. This procedure, **early stopping**, is the simplest and most important defence against overfitting.

> **KEY IDEA**
> The training set teaches the network; the validation set decides when to stop and which model to keep; the test set is touched only once, at the very end, to measure the final result honestly.

2.3 Data augmentation: showing the network what can change

With only about two dozen training nights, the network sees a limited variety of skies. **Data augmentation** creates additional variety by applying random, label-preserving transformations to the training images each time they are

used. The transformations must be *physically plausible*: they should mimic changes that could really occur without changing whether the sky is clear.

For an all-sky camera, the natural choices are:

- **Horizontal and vertical flips.** Cloud cover has no preferred handedness.
- **Rotations by up to 180°.** For an all-sky view, cloud cover has no preferred orientation, and because the mask is a centred circle, a rotated masked image is still a valid masked image.
- **Brightness and contrast changes.** Moon phase, exposure and atmospheric transparency all change the overall brightness. The first experiments used changes of $\pm$20%; later experiments increased this to $\pm$50% to make the models less sensitive to moonlight.

Augmentation is applied only during training. Validation and test images are always presented unchanged, apart from the mask, resizing and normalisation.

2.4 Model A: a small CNN trained from scratch

2.4.1 Architecture

The first model is deliberately small and simple, so that every part of it can be understood and later inspected. It consists of four convolutional blocks followed by a classification head:

- Each **block** is a 3 × 3 convolution, batch normalisation, ReLU and 2 × 2 max pooling.
- The number of filters doubles from block to block: 32, 64, 128 and 256.
- The **head** is global average pooling, dropout with probability 0.3 (randomly silencing 30% of the values during training, another defence against overfitting) and a linear layer with two outputs.

Starting from a 224 × 224 input, the four pooling steps reduce the feature maps to 112, 56, 28 and finally 14 pixels on a side. The final feature maps are thus 14 × 14 cells of 256 channels each, and each cell summarises a patch of roughly 16 × 16 input pixels plus its surroundings.

2.4.2 Counting the parameters

Counting parameters by hand is a good check that the architecture is what we think it is. A convolution with C_{in} input channels, C_{out} filters and 3×3 kernels has $C_{in} \times C_{out} \times 9$ weights plus C_{out} biases; batch normalisation adds $2 \times C_{out}$ learned scales and shifts.

Table 2.1 Parameters of the small CNN. The total agrees exactly with the 389,890 reported by PyTorch.

Layer	Calculation	Parameters
Conv 3→32 + BN	3·32·9 + 32 + 2·32	960
Conv 32→64 + BN	32·64·9 + 64 + 2·64	18,624
Conv 64→128 + BN	64·128·9 + 128 + 2·128	74,112
Conv 128→256 + BN	128·256·9 + 256 + 2·256	295,680
Linear 256→2	256·2 + 2	514
Total		**389,890**

Most of the parameters sit in the last convolutional block, which combines 128 channels into 256. Compared with ResNet-18's 11.2 million parameters, the small CNN is about 29 times smaller.

2.4.3 Training

The network was trained for at most 25 epochs with an initial learning rate of 10^{-3} and early stopping with patience 5. On a T4 GPU in Google Colab each epoch took about 90 seconds, most of it spent loading and resizing images rather than computing. Training stopped after 15 epochs.

Table 2.2 Training log of the small CNN (loss / accuracy). The model kept is from epoch 10, which has the lowest validation loss.

Epoch	Train loss	Train acc.	Val. loss	Val. acc.
1	0.227	0.916	0.705	0.780
2	0.179	0.938	0.334	0.872
3	0.180	0.939	0.361	0.902
4	0.165	0.945	0.450	0.898

Epoch	Train loss	Train acc.	Val. loss	Val. acc.
5	0.151	0.951	0.376	**0.910**
6	0.127	0.959	0.461	0.797
7	0.119	0.958	0.314	0.860
8	0.106	0.962	0.365	0.862
9	0.104	0.964	1.362	0.613
10	0.086	0.969	**0.292**	0.854
11	0.090	0.968	0.383	0.895
12	0.081	0.971	0.465	0.848
13	0.062	0.978	0.532	0.837
14	0.060	0.979	0.498	0.841
15	0.056	0.980	0.481	0.863

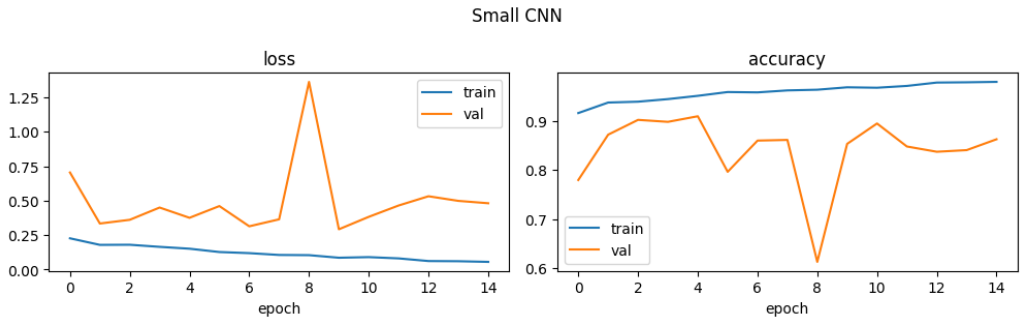


Figure 2.1 Learning curves of the small CNN. Training loss falls steadily and training accuracy climbs towards 0.98, while the validation curves jump between epochs — including a collapse to 0.61 accuracy at epoch 9 (index 8). The plot counts epochs from 0.

2.4.4 Reading the learning curves

The curves in Figure 2.1 tell a clear story, and it is not the textbook one of smoothly converging lines.

Validation accuracy was already 0.902 at epoch 3, above the 0.845 reached by the hand-crafted features. The network is clearly learning something beyond global brightness statistics, and it learns it fast.

Validation performance jumps around wildly. It swings between 0.61 and 0.91 from one epoch to the next. Three causes combine:

1. **The validation set is effectively five nights.** Frames within a night are so similar that if the model misjudges one night's conditions, hundreds of frames flip together. The collapse at epoch 9, with accuracy 0.613 and loss 1.362, is most likely one entire validation night going wrong.
2. **Overfitting to the training nights.** Training accuracy climbs steadily to 0.98 and training loss keeps falling, while validation loss *rises* after epoch 10 from 0.29 to 0.48. The network is increasingly memorising features specific to the training nights.
3. **Batch normalisation and night-to-night shifts.** Batch normalisation learns brightness statistics from the training nights. A validation night with a different moon phase looks unusual to these statistics, which destabilises the outputs.

Loss and accuracy disagree about the best epoch. Epoch 5 has the best validation accuracy (0.910) but a higher loss (0.376); epoch 10 has the lowest loss (0.292) but an accuracy of only 0.854. Accuracy counts only right versus wrong, while cross-entropy also punishes confident mistakes severely. Epoch 5 got more frames right but was very confident about the ones it got wrong; epoch 10 made more mistakes but hesitant ones. For an observatory, the epoch-10 behaviour is arguably safer: uncertain predictions can be flagged for a human, while confident mistakes are silent. Selecting by validation loss, as early stopping does, kept epoch 10.

The small CNN's verdict is therefore mixed. It learns quickly and beats the hand-crafted baseline on validation, but it is unstable across nights. The difficulty is not separating the classes within the training data, where the network reaches 98%. It is **generalising to new observing conditions**.

2.5 Model B: transfer learning with ResNet-18

2.5.1 Borrowing experience from a million photographs

A network trained from scratch must learn everything from our 5,705 training frames: what an edge looks like, what a texture is, how to tell a smooth gradient from a sharp boundary. Yet these low-level visual skills are the same for clouds as for cats or cars. **Transfer learning** exploits this. We start from a network already trained on a huge general dataset and adapt it to our task.

The network chosen is **ResNet-18**, trained on **ImageNet**, a collection of about 1.2 million photographs in 1,000 everyday categories. ResNet-18 has 18 weight layers and about 11.2 million parameters. Its early layers have learned general-purpose detectors of edges, colours and textures; its later layers detect increasingly complex shapes. None of it has ever seen an all-sky camera frame, but its visual vocabulary is rich enough to describe one.

2.5.2 Residual connections

ResNet's defining feature is the **residual connection**. In a plain network, each block computes a new representation from the previous one: $x \to F(x)$. In a residual network, each block computes a *correction* that is added to its input:

$$\mathbf{x}_{l+1} = \mathrm{ReLU}\left(\mathbf{x}_l + F(\mathbf{x}_l)\right)$$

where F consists of two convolutions with batch normalisation. If a block has nothing useful to add, it can learn $F \approx 0$ and simply pass its input through. This makes very deep networks much easier to train, because gradients can flow backwards through the identity shortcuts without fading away. ResNet-18 arranges its blocks into four stages, named layer1 to layer4, with 64, 128, 256 and 512 channels. Its final feature maps are 7×7 cells of 512 channels, followed by global average pooling and the final linear layer, exactly the head structure described in Section 2.2.4.

2.5.3 The two-stage recipe

The standard recipe for transfer learning has two stages:

1. **Stage 1, the head.** Replace the original 1,000-class output layer with a new two-class layer, **freeze** all other weights so they cannot change, and train only the new layer. The pre-trained features are used as they are, and only the final linear combination is learned: $512 \times 2 + 2 = 1{,}026$ parameters.
2. **Stage 2, fine-tuning.** Unfreeze some or all of the network and continue training with a much smaller learning rate, so that the pre-trained features can adapt gently to the new domain.

2.5.4 First attempt: unfreeze everything

The first experiment followed the textbook. Stage 1 trained the head for 5 epochs at a learning rate of 10^{-3}; stage 2 then unfroze all 11.2 million parameters and trained them at 10^{-4}.

Table 2.3 The first ResNet-18 run. Epochs 1–5 train the head only; epochs 6–9 fine-tune all layers. The run was stopped by hand after epoch 9.

Stage	Epoch	Train loss	Train acc.	Val. loss	Val. acc.
Head only	1	0.223	0.929	**0.293**	0.904
	2	0.118	0.965	0.375	0.895
	3	0.100	0.967	0.459	0.878
	4	0.096	0.971	0.405	0.893
	5	0.091	0.974	0.390	0.906
All layers	6	0.052	0.983	0.865	0.848
	7	0.025	0.991	0.956	0.851
	8	0.025	0.992	0.816	0.872
	9	0.020	0.993	1.545	0.793

The first epoch is remarkable. With only the 1,026 head parameters trained, on top of features learned from everyday photographs, the validation loss reached 0.293 and the accuracy 0.904, matching the *best* the small CNN achieved after ten epochs of full training. Generic ImageNet features describe sky images well.

Stage 2 was a different matter. The moment all layers were unfrozen, training accuracy shot up to 0.99 and the validation loss tripled, from about 0.3 to between 0.82 and 1.55. With only about 24 training nights, the full network did exactly what it is capable of: it memorised the training nights instead of learning general cloud features.

The experiment also exposed a flaw in the original training code. Each stage restored only its *own* best epoch. Stage 2 would have early-stopped and returned its best weights (validation loss about 0.82), and the much better stage-1 model would already have been lost. The lesson is general: when a training procedure has several stages, the best model must be kept **across all stages**, not within each.

2.5.5 Second attempt (v2): fine-tune only the last block

The second attempt changed three things. Only **layer4**, the last stage, was unfrozen in stage 2, so the general low-level features stay as they are. The learning rate was reduced to 3×10^{-5}, 30 times smaller than in stage 1. Brightness augmentation was increased to $\pm 50\%$, and contrast augmentation to $\pm 30\%$. And the best model was kept across both stages.

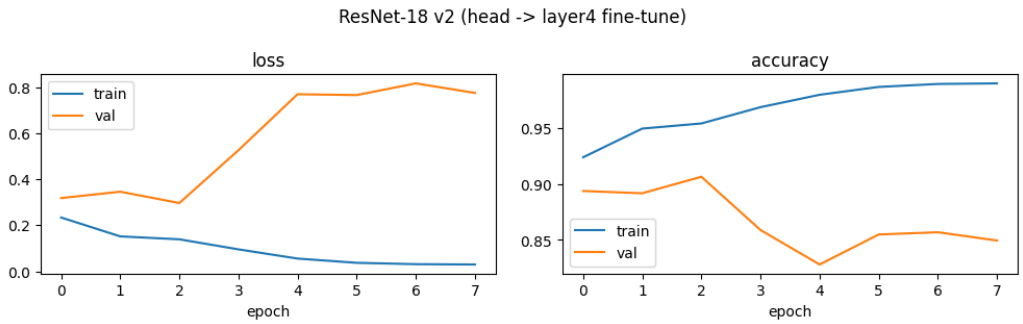


Figure 2.2 ResNet-18 v2. Epochs 0–2 (stage 1, head only) keep the validation loss near 0.30 and reach 0.906 accuracy at epoch 2. As soon as layer4 starts learning (epochs 3–7) the validation loss jumps to 0.77–0.82 while training accuracy climbs to 0.99.

Figure 2.2 shows the outcome. Stage 1 behaved even better than before: the validation loss improved to **0.297** by epoch 3 with an accuracy of **0.906**, and the gap between training and validation was smaller, suggesting that the stronger augmentation already reduced overfitting. But the moment layer4 began to

learn, the same pattern returned: validation loss jumped to 0.53 and then about 0.77, validation accuracy fell to 0.83–0.86, and training accuracy climbed towards 0.99. Layer4 alone holds about 8.4 of ResNet-18's 11.2 million parameters, still far more than enough to memorise 24 nights. Stage 2 never beat stage 1, so the safeguard restored the stage-1 model. This **v2 model**, a frozen ImageNet backbone with a trained linear head, became one of the final models.

2.5.6 Third attempt (v3): making fine-tuning robust

Could fine-tuning be made to help with better regularisation? The third attempt combined five techniques, each addressing a specific weakness observed so far.

1. Frozen batch-normalisation statistics. A subtle leak had been present all along. Even in the "frozen" stage 1, the batch-normalisation layers were in training mode, so their running means and variances kept updating to match the training nights. A frozen backbone was therefore not entirely frozen: its normalisation drifted towards the brightness statistics of the training nights, and a new night with a different moon phase was then mis-normalised. In v3 all batch-normalisation layers were kept in evaluation mode, retaining their ImageNet statistics.

2. L2-SP regularisation. Ordinary weight decay pulls weights towards zero. When fine-tuning a pre-trained network it makes more sense to pull them towards their *pre-trained values*, so that the network can adapt but not wander far. This is the idea of L2-SP (L2 penalty towards the Starting Point). An extra term is added to the loss:

$$L_{\text{total}} = L_{\text{CE}} + \alpha \sum_{i \in \text{layer4}} \left(w_i - w_i^{\text{pre}} \right)^2$$

with $\alpha = 0.01$, where w_i^{pre} are the ImageNet weights.

3. Label smoothing. Cross-entropy rewards the network for pushing probabilities all the way to 0 or 1, and over-confidence is exactly what made the validation loss explode. Label smoothing replaces the hard target (1 for the

true class, 0 for the other) with a softened one. With smoothing $\varepsilon = 0.1$ and $K = 2$ classes, the target probability for the true class becomes

$$q_y = 1 - \varepsilon + \frac{\varepsilon}{K} = 0.95$$

and 0.05 for the other class, so the network is never rewarded for being absolutely certain.

4. Physical augmentation. In addition to flips, rotations and the stronger brightness and contrast changes, three new random transformations imitate real effects. A **random gamma** between 0.6 and 1.6 mimics changes of exposure and moon brightness. A slight **Gaussian blur**, applied to 30% of the images, mimics dew or haze on the lens. **Gaussian noise** with a standard deviation of 0.02 mimics sensor noise at ISO 3200 and 15-second exposures. And **random erasing** of small rectangles in 30% of the images mimics aircraft, birds or other obstructions.

5. An even smaller learning rate of 10^{-5} for the layer4 fine-tuning, with patience 3.

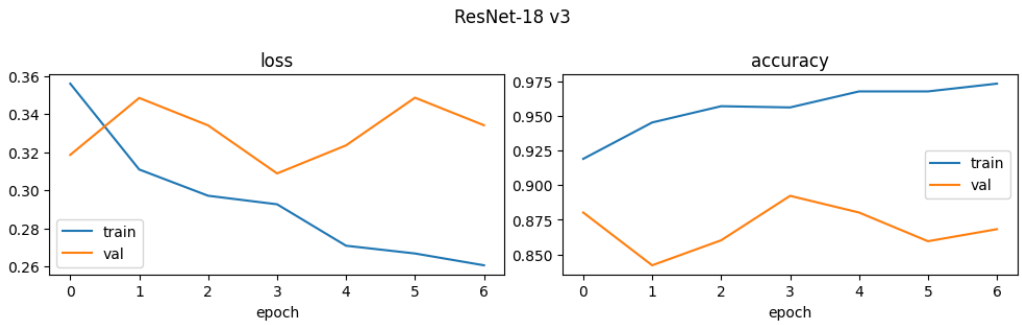


Figure 2.3 ResNet-18 v3. Epochs 0–2 are stage 1, epochs 3–6 fine-tune layer4 with frozen batch normalisation, L2-SP, label smoothing and physical augmentation. The validation loss now stays between 0.31 and 0.35 when layer4 is unfrozen, instead of exploding. The training loss includes the smoothing and penalty terms and is therefore not comparable to the earlier runs.

Figure 2.3 shows the effect. The safeguards worked: when layer4 was unfrozen, the validation loss stayed between 0.31 and 0.35 instead of exploding to 0.8 or beyond. Overfitting was under control. Yet v3's best validation loss, 0.309 with an accuracy of 0.892, did not beat v2's 0.297 and 0.906. With only five validation

nights, a difference of 0.012 in loss is far smaller than the night-to-night noise: statistically, v2 and v3 are tied on validation.

2.5.7 What the three ResNet runs teach

Table 2.4 The three ResNet-18 strategies compared on validation, in the phase where deeper layers are trained.

Strategy	Validation loss when deeper layers learn	Validation accuracy	Verdict
Head, then all layers (lr 10^{-4})	0.82 → 1.55	0.79–0.87	Severe overfitting
v2: head, then layer4 (lr 3×10^{-5})	0.53 → 0.82	0.83–0.86	Overfitting; stage-1 model kept (best 0.297)
v3: layer4 + frozen BN, L2-SP, smoothing, augmentation	0.31 → 0.35	0.86–0.89	Stable but no gain (best 0.309)

The pattern is consistent: **the more of the network we let change, the worse it does on unseen nights**, while training accuracy heads towards 99%. Regularisation can make fine-tuning *safe*, but in this dataset it does not make it *better* than simply using the frozen pre-trained features.

This is a genuine and useful finding, not a failure. It says that with about two dozen observing nights, the information available to adapt millions of parameters is too limited. The best-generalising model is a linear classifier on frozen ImageNet features, and the most promising way to improve further is not a bigger model but **more nights**, especially nights with varied moon phases.

2.6 Choosing the final models

After these experiments three trained models were available:

- the **small CNN** (epoch 10), trained from scratch;
- **ResNet-18 v2**, a frozen ImageNet backbone with a trained head;
- **ResNet-18 v3**, with regularised layer4 fine-tuning.

Rather than pick one, Chapter 3 tests the idea of combining them in an **ensemble**, averaging their probabilities. The decision to do so, and the choice of which combination to use, was made on the validation set, before the test set was examined in detail. On validation, both ensembles beat every single model:

Table 2.5 Validation results used for model selection.

Model	Validation AUC	Validation accuracy
Small CNN	0.953	0.854
ResNet-18 v2	0.919	0.906
ResNet-18 v3	0.927	0.892
Ensemble: small CNN + ResNet v2	**0.959**	0.901
Ensemble: all three	0.954	**0.908**

The three-model ensemble had the best validation accuracy, the metric that matters for an open-or-close decision, and was chosen as the final model.

2.7 Summary

- A **convolutional network** learns its own features: convolution filters detect local patterns, pooling widens their view, and a final linear layer turns averaged feature maps into class probabilities.
- Training minimises a **cross-entropy loss** with AdamW and a cosine learning-rate schedule; **early stopping** on the validation loss keeps the best epoch.
- **Augmentation** must be physically plausible: flips, full rotations and brightness changes suit an all-sky camera.
- A **small CNN** (389,890 parameters) beats the hand-crafted baseline on validation within three epochs but is unstable from night to night.
- **ResNet-18** with a frozen ImageNet backbone matches the small CNN's best validation loss after a single epoch of training only the head.

- **Fine-tuning deeper layers overfits** with about 24 training nights. Frozen batch normalisation, L2-SP, label smoothing and physical augmentation make fine-tuning stable but not better.
- When training has several stages, the best model must be kept **across stages**.
- On validation, an **ensemble of all three models** is best, and it becomes the final model evaluated in the next chapter.

CHAPTER 3

Testing in the Real World

IN THIS CHAPTER

A model is only as good as its performance on data it has never seen. This chapter evaluates the three networks on six held-out nights, compares them fairly with the hand-crafted baseline and combines them into an ensemble that reaches 86.7% accuracy with the smallest night-to-night variation. It then asks why the remaining errors occur. Computing the Moon's position for every frame shows that moonlight is the main difficulty, and following the labels through the night shows that many apparent errors are, in fact, errors in the labels.

3.1 Measuring performance

3.1.1 The confusion matrix

For a two-class problem every prediction falls into one of four cells. Calling *fine* the positive class:

- **True positive (TP):** a fine frame predicted fine.
- **True negative (TN):** a cloudy frame predicted cloudy.
- **False positive (FP):** a cloudy frame predicted fine.
- **False negative (FN):** a fine frame predicted cloudy.

Arranged as a 2 × 2 table, with the true class in the rows and the prediction in the columns, these counts form the **confusion matrix**. Every other measure in this chapter is computed from it.

3.1.2 Accuracy, precision, recall and F1

Accuracy is the fraction of correct predictions,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

It is easy to understand but can hide important differences between the two kinds of error. Two further measures, computed separately for each class, reveal them. **Precision** asks: of all frames predicted to be in a class, how many really are? **Recall** asks: of all frames really in a class, how many were found? For the fine class,

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}$$

The **F1 score** is their harmonic mean, F1 = 2PR / (P + R), which is high only when both are high.

3.1.3 Why the two errors are not equal

For an observatory the two kinds of error have different consequences:

- **Cloudy predicted fine (FP).** The system opens the dome or starts observing under cloud. Observing time is wasted, and if the clouds bring humidity or rain, the instruments are exposed.
- **Fine predicted cloudy (FN).** The system keeps the dome closed under a clear sky. Valuable observing time is lost, but nothing is endangered.

A model that rarely makes the first kind of error is *conservative*, and for equipment safety that can be preferable even at some cost in accuracy. Keeping both errors in view, rather than only accuracy, matters when choosing a model for operation.

3.1.4 The decision threshold

Every model outputs a probability P(clear). By default a frame is called fine when P(clear) $\geq$ 0.5. Moving this **threshold** trades one error for the other: a higher threshold makes the system more reluctant to declare the sky clear, reducing false positives at the cost of more false negatives. The AUC introduced in Chapter 1 measures performance across all thresholds at once,

which makes it the right measure for comparing models whose thresholds are not yet tuned.

3.2 Test results of the individual models

The three models were evaluated on the 1,606 frames of the six test nights. None of these nights had been used for training or for model selection.

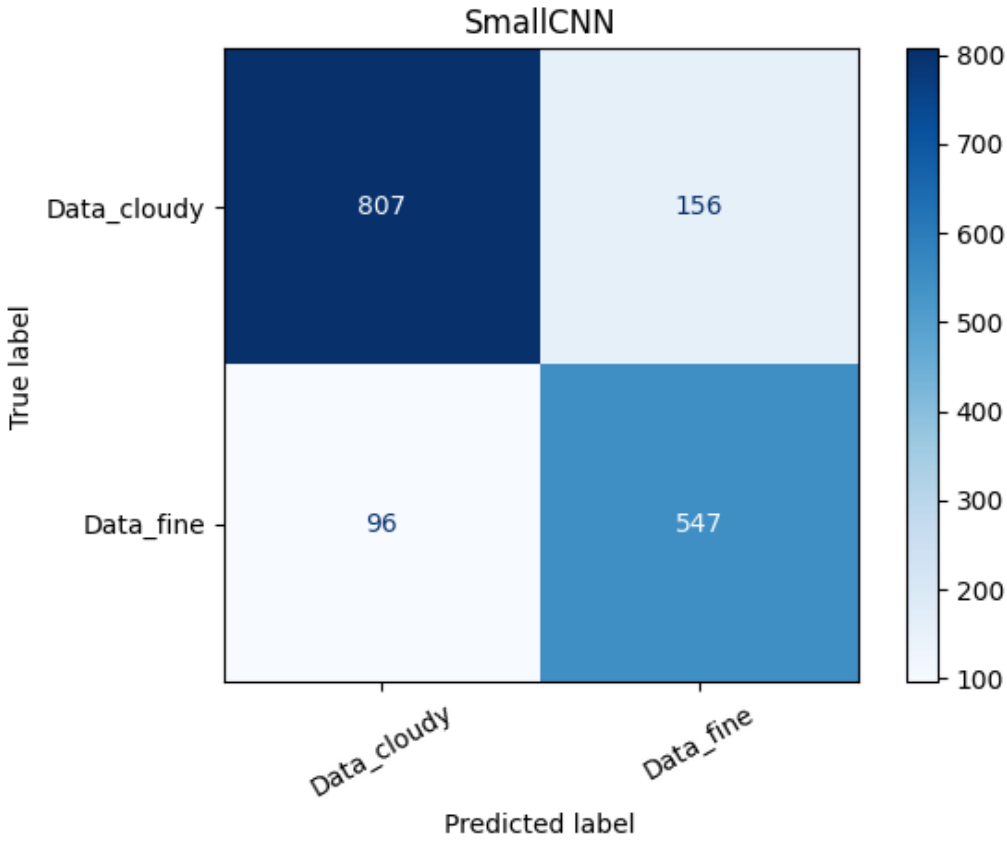


Figure 3.1 Confusion matrix of the small CNN on the test nights. Rows are the dataset labels, columns the predictions.

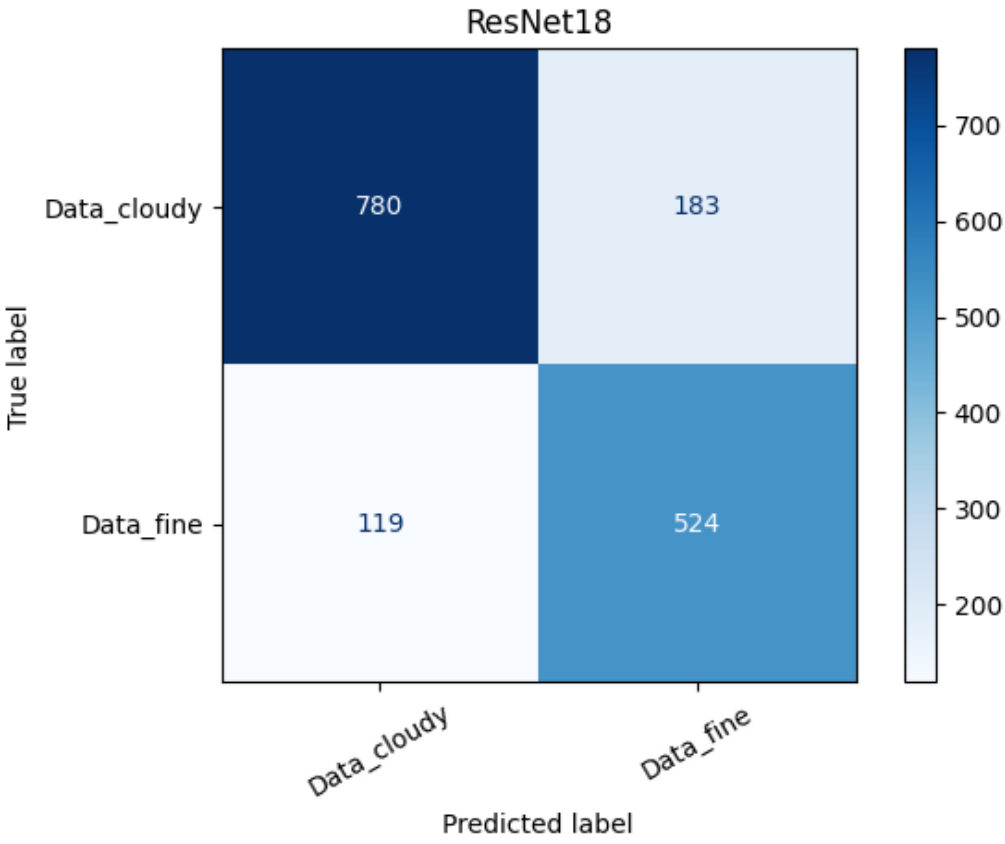


Figure 3.2 Confusion matrix of ResNet-18 v2 on the test nights.

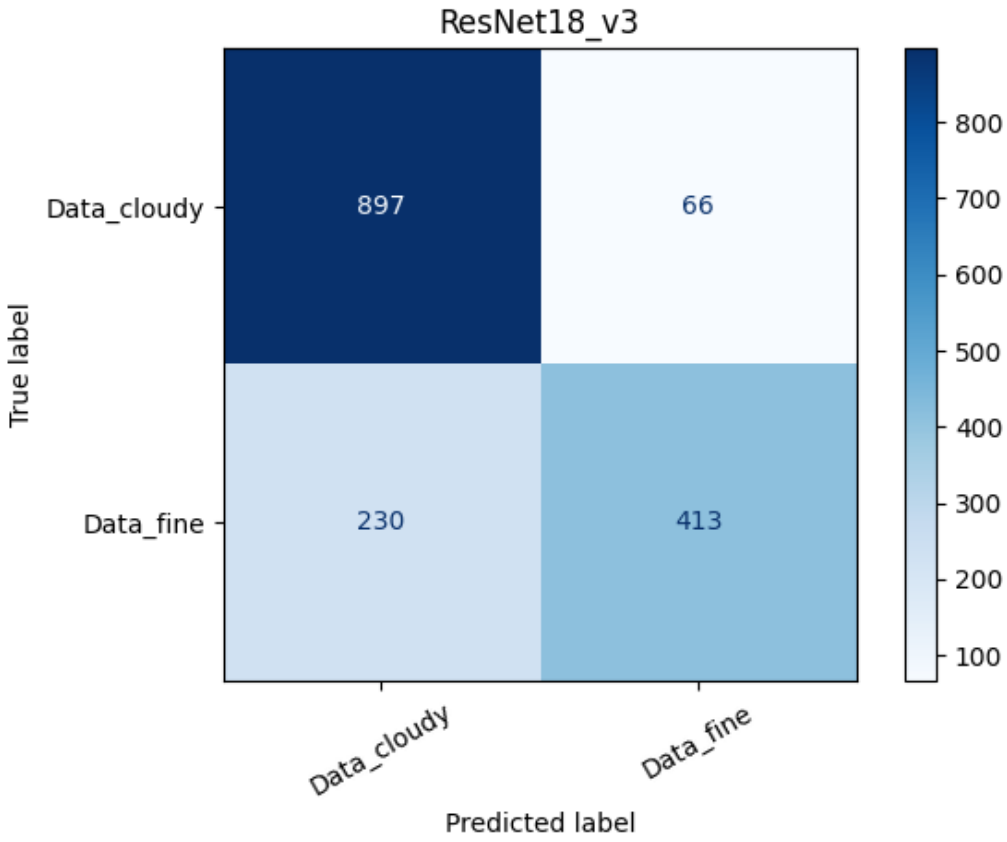


Figure 3.3 Confusion matrix of ResNet-18 v3 on the test nights. Note the very different balance of errors: only 66 cloudy frames called fine, but 230 fine frames called cloudy.

Table 3.1 Test results of the individual models (threshold 0.5). Recall of each class shows how the errors are distributed.

Model	Accuracy	Cloudy precision	Cloudy recall	Fine precision	Fine recall	Macro F1
Small CNN	**0.843**	0.894	0.838	0.778	0.851	**0.839**
ResNet-18 v2	0.812	0.868	0.810	0.741	0.815	0.807
ResNet-18 v3	0.816	0.796	**0.931**	**0.862**	0.642	0.797

Three observations stand out.

Validation was optimistic. ResNet-18 v2 reached 0.906 accuracy on the validation nights but only 0.812 on the test nights. The test nights are harder than the validation nights. With only five or six nights in each set, *which* nights land in which set matters more than which model is used, a direct consequence of the small number of independent samples discussed in Chapter 1.

The small CNN is the best single model on the test nights, with 84.3% accuracy and the most balanced errors: 156 cloudy frames called fine and 96 fine frames called cloudy.

v2 and v3 have similar accuracy but behave very differently. v3 is *conservative*. It almost never calls a cloudy sky clear (66 errors against v2's 183), but it misses many clear frames (230 against 119). Freezing the batch-normalisation statistics shifted its decision boundary. At the default threshold, accuracy hides this difference completely; the confusion matrices make it obvious. For equipment safety v3's behaviour may be the more desirable one.

3.3 A fair comparison with the baseline

Chapter 1 reported that four hand-crafted features reached 84.5% accuracy in grouped cross-validation. That number came from a different evaluation: cross-validation over all nights. To compare fairly, the gradient-boosting classifier was retrained on the features of 2,000 training frames and evaluated on exactly the same 1,606 test frames as the networks.

Table 3.2 All models on the same six test nights. AUC does not depend on the threshold.

Model	Test AUC	Test accuracy
Small CNN	**0.922**	**0.843**
ResNet-18 v2	0.892	0.812
ResNet-18 v3	0.892	0.816
Hand-crafted features (gradient boosting)	0.856	0.771

On the same nights the hand-crafted model drops to **77.1%**. Every network beats it, by 4 to 7 points in accuracy and by 0.04 to 0.07 in AUC. The networks are therefore genuinely using information that global brightness, contrast,

texture and star counts do not capture. The cross-validated 84.5% had flattered the baseline, because it averaged over easier nights.

3.4 Night by night

A single accuracy number averages over very different conditions. Breaking the results down by night reveals where each model succeeds and fails.

Table 3.3 Accuracy on each test night. The bottom two rows are ensembles, introduced in Section 3.5.

Model	11-15 (fine)	11-24 (mixed)	11-29 (fine)	12-11 (cloudy)	12-15 (cloudy)	12-19 (cloudy)	Mean ± sd
Hand-crafted	0.372	0.716	1.000	0.931	1.000	1.000	0.836 ± 0.231
Small CNN	0.901	0.799	1.000	0.843	0.697	0.812	0.842 ± 0.093
ResNet-18 v2	0.843	0.814	1.000	0.894	0.909	0.420	0.813 ± 0.185
ResNet-18 v3	0.724	0.799	0.509	0.980	0.970	0.762	0.791 ± 0.159
Ensemble CNN + v2	0.908	0.818	1.000	0.880	0.879	0.707	0.865 ± 0.089
Ensemble of all 3	0.891	0.816	1.000	0.931	0.909	0.729	**0.879 ± 0.086**

The table repays careful reading.

- **The hand-crafted model is essentially a brightness detector.** It is perfect on the three cloudy nights and on the dark fine night of 29 November, but on the fine night of 15 November it is right only 37% of the time. That night had a bright Moon, and a bright clear sky "looks cloudy" to brightness-based features.
- **ResNet-18 v2 fails in the opposite direction.** On the cloudy night of 19 December it reaches only 0.420. About 105 of its 183 cloudy-called-fine errors come from this single night, which, as Section 3.6 will show, was the night of the full moon. It calls moonlit cloud "fine".

- **ResNet-18 v3's conservative bias** costs it the dark fine night of 29 November, where it falls to 0.509 while every other model is perfect.
- **The small CNN is the most balanced** single model. It never drops below 0.70 on any night.
- **The mixed night of 24 November** is hard for every model, at 0.72 to 0.82. On this night the Moon, the season and the camera settings are identical for fine and cloudy frames; only the clouds differ. It is the most honest test of cloud detection in the dataset.

Most importantly, **the models fail on different nights**. That observation leads directly to the idea of combining them.

3.5 The ensemble: strength in disagreement

3.5.1 Why averaging helps

An **ensemble** combines several models by averaging their predicted probabilities:

$$P_{\text{ens}}(\text{clear}) = \frac{1}{M} \sum_{m=1}^{M} P_m(\text{clear})$$

with M = 3 here. Averaging helps when the models' errors are not perfectly correlated. A simple argument shows why. If each model's prediction is the true value plus an error of variance σ^2, and the errors of any two models have correlation ρ, then the variance of the averaged error is

$$\text{Var}\left(\frac{1}{M} \sum_m \epsilon_m \right) = \rho\, \sigma^2 + \frac{1-\rho}{M}\, \sigma^2$$

When the errors are identical ($\rho = 1$), averaging achieves nothing. When they are independent ($\rho = 0$), the error variance falls by a factor of M. Our three models are very different: one trained from scratch, one a frozen ImageNet backbone, one regularised fine-tuning with frozen normalisation. The per-night table shows that their failures are largely on different nights, so ρ is well below 1 and averaging should help.

3.5.2 Deciding honestly

There is a methodological trap here. The idea of an ensemble arose *after* looking at the test results. Choosing the final model by its test score would make the test set part of model selection, and the reported test accuracy would no longer be an honest estimate. The ensembles were therefore evaluated on the **validation** nights first (Table 2.5). On validation, both ensembles beat every single model. The decision to use an ensemble, and the choice of the three-model ensemble for its best validation accuracy (0.908), were made on validation. The test results are reported afterwards, for information.

3.5.3 Final results

Table 3.4 Final test results. The ensemble of all three models, chosen on validation, is the final model.

Model	Test AUC	Test accuracy	Per-night mean ± sd	Worst night
Hand-crafted features	0.856	0.771	0.836 ± 0.231	0.372 (11-15)
Small CNN	0.922	0.843	0.842 ± 0.093	0.697 (12-15)
ResNet-18 v2	0.892	0.812	0.813 ± 0.185	0.420 (12-19)
ResNet-18 v3	0.892	0.816	0.791 ± 0.159	0.509 (11-29)
Ensemble of all 3	0.924	**0.867**	**0.879 ± 0.086**	0.729 (12-19)

The ensemble improves accuracy and reliability together: **86.7%** accuracy, 2.4 points above the best single model and 9.6 points above the hand-crafted features on the same nights, with the **smallest spread from night to night** (standard deviation 0.086). Its worst night, the full-moon night, is still at 0.73, far better than v2's 0.42.

The per-night standard deviation deserves emphasis. Overall accuracy is dominated by the large nights (12-11 has 451 frames, 12-15 only 33). The mean and spread across nights measure how *reliable* a model is from one night to the next. For an observatory, a model that is 85% $\pm$ 5% every night is more useful than one that is 95% on most nights and 40% on full-moon nights.

The remaining weak spots are the full-moon night of 19 December (0.73) and the mixed night of 24 November (0.82). The next sections investigate them.

3.6 The Moon: measuring a hidden variable

Chapter 1 suggested that moonlight might be the main source of difficulty, and the per-night results point the same way: the worst nights for the networks, 19 December, and for the hand-crafted model, 15 November, both appear to be moonlit. A suspicion is not a measurement. Because every file name is a timestamp, the Moon's position at the moment each frame was taken can be computed precisely.

3.6.1 Computing the Moon's position

The calculation has four steps, all performed with the astronomy library *astropy*.

Step 1: the time. The file name `211124194628` gives 2021-11-24, 19:46:28 local time. Japan Standard Time is UTC+9, so this corresponds to 10:46:28 UTC. Astronomical calculations require universal time.

Step 2: the place. The image caption names the Higashi-Hiroshima Observatory. Its approximate coordinates are latitude $\varphi \approx 34.38°$ N, longitude $\approx 132.78°$ E and height ≈ 500 m. An error of a few kilometres in the site position changes the computed altitude by far less than 0.1°.

Step 3: the Moon's celestial coordinates. From the time, astropy's lunar theory gives the Moon's position on the celestial sphere as right ascension α and declination δ, the sky's equivalents of longitude and latitude.

Step 4: from celestial to horizontal coordinates. What matters for the camera is how high the Moon stands above the local horizon, its **altitude**. This follows from the standard relation of spherical astronomy,

$$\sin(\text{alt}) = \sin\varphi \sin\delta + \cos\varphi \cos\delta \cos H$$

where $H = \text{LST} - \alpha$ is the Moon's **hour angle** and LST the **local sidereal time**, which depends on the universal time and the observatory's longitude. An altitude above 0° means the Moon is up; below 0° it has set; 90° is directly overhead.

The **illuminated fraction** of the lunar disc follows from the angle ψ between the Sun and the Moon as seen from Earth, the *elongation*:

$$k \approx \frac{1 - \cos \psi}{2}$$

At new moon ($\psi = 0°$) the fraction is 0; at quarter moon ($\psi = 90°$) it is 0.5; at full moon ($\psi = 180°$) it is 1. The computation neglects atmospheric refraction, which raises apparent altitudes by about half a degree near the horizon, and approximates the phase angle by the elongation. Both simplifications are negligible at the level of detail needed here. As an independent check, the computed altitude for the frame taken at 21:57 on 19 December is 57°, and for the frame at 19:46 on 24 November it is −10°, values that can be confirmed in any planetarium program.

3.6.2 The Moon on each test night

Table 3.5 Moon conditions on each test night, computed from the frame timestamps, with the ensemble's accuracy.

Night	Class	Illumination	Max. altitude	Frames with Moon up	Ensemble accuracy
11-29	fine	0.30	−49° (never up)	0%	**1.000**
12-11	cloudy	0.55	44°	44%	0.931
11-24	mixed	0.77	73°	70%	0.816
11-15	fine	0.87	56°	100%	0.891
12-15	cloudy	0.89	70°	100%	0.909
12-19	cloudy	**1.00 (full)**	80°	100%	**0.729**

The ephemeris confirms the suspicion in striking form. The only night on which the Moon never rose, 29 November, is the one night that every model except the conservative v3 classified perfectly. The hardest night, 19 December, is the night of the full moon, with the Moon climbing to 80°, almost overhead.

3.6.3 Accuracy against Moon altitude

Grouping all test frames by the Moon's altitude at the moment each was taken gives a direct measure of the effect.

Table 3.6 Test accuracy by Moon altitude. The last column shows the share of fine frames in each group.

Moon altitude	Frames	Hand-crafted	Small CNN	ResNet-18 v2	Ensemble	Fine share
Below horizon	523	0.851	0.860	0.901	0.889	0.43
0–20°	201	0.736	0.866	0.771	0.861	0.29
20–40°	362	**0.649**	**0.751**	**0.715**	**0.837**	0.30
Above 40°	520	0.788	0.881	0.806	0.867	0.49
Range		**0.20**	0.13	0.19	**0.05**	

Three findings follow.

1. **Moonlight degrades every model.** All of them perform best when the Moon is below the horizon.
2. **The models differ greatly in how much they suffer.** Hand-crafted features and ResNet-18 v2 lose about 20 points across Moon conditions; the ensemble loses only 5. This is the quantitative evidence that the ensemble relies on cloud structure rather than on overall sky brightness, and it is perhaps the strongest argument for choosing it.
3. **The worst case is the Moon at 20–40°, not overhead.** A plausible physical explanation is that at intermediate altitude the Moon sits inside the field of view but away from the centre, creating strong glare and an uneven brightness gradient across the frame that resembles bright cloud on one side. Near the zenith the illumination is more symmetric; near the horizon the Moon is partly hidden behind the mask edge and horizon structures. This remains a hypothesis; Chapter 4 looks for evidence in the networks' attention maps.

An honest caveat belongs here. With only six test nights, each altitude group consists of frames from a few particular nights, so the effect of altitude is partly mixed with the effect of individual nights, and the class balance differs between groups (29% to 49% fine). The trend is strong evidence, not proof. More nights, especially moonlit clear nights and moonlit cloudy nights, would settle the question.

3.7 Temporal smoothing

Clouds drift and evolve over minutes, not seconds. If the model occasionally misjudges a single frame in the middle of a long clear spell, the neighbouring frames should outvote it. This suggests **temporal smoothing**: replace each frame's probability by the median of the probabilities in a window of neighbouring frames from the same night. The median is preferred to the mean because a single wild value cannot pull it far.

Table 3.7 Effect of a centred rolling median of P(clear) on the ensemble's test accuracy. Frames are about one minute apart.

Window (frames ≈ minutes)	1 (none)	3	5	9	15
Test accuracy	0.867	0.872	0.872	0.877	**0.878**

Smoothing helps, but only a little: at most 1.1 points, and the accuracy is still creeping upwards at a window of 15 frames. This modest gain is itself informative. If the errors were isolated single frames, a median over three to five frames would remove most of them and accuracy would jump. Instead, the gain is small and gradual, which means that **errors come in long runs**: stretches of many consecutive minutes misclassified together. The errors are systematic, not random. The model is consistently wrong under certain conditions that persist for a while, such as a moonlit hour, or, as Section 3.9 will show, a block of mislabelled frames. No amount of smoothing can fix an error that lasts an hour.

Two practical points matter for operation. First, the window should not be chosen by its test score; that would be selection on the test set again. A window of about five minutes is a reasonable operational choice, matching the reaction time of a dome. Second, a *centred* window uses future frames, which is fine for analysis but impossible in real time. A live system must use a *trailing* window, the last few frames only, which delays decisions by a few minutes. The trade-off between stability and reaction time is a design decision for the observatory.

3.8 The error gallery

Numbers tell us how often a model fails. Images tell us how. Figure 3.4 shows the twelve test frames on which the final ensemble was most confidently wrong, each with its timestamp, its label, the prediction with its confidence and the Moon's altitude.

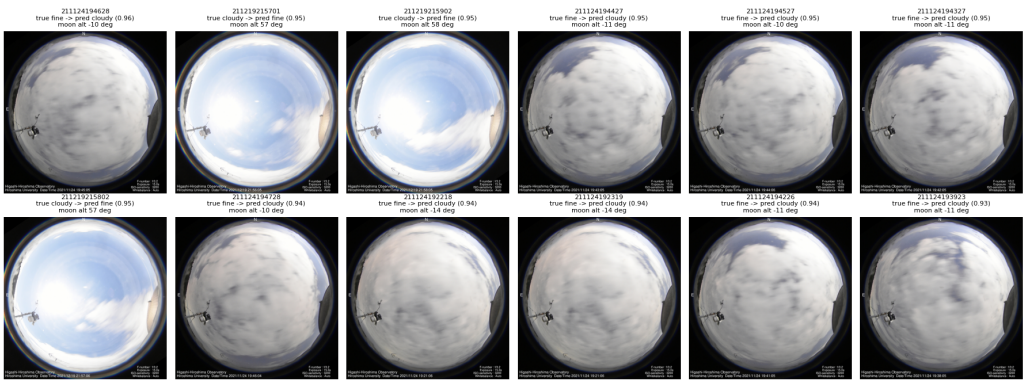


Figure 3.4 The twelve most confident errors of the final ensemble. Nine come from the night of 24 November between 19:21 and 19:46, labelled fine, predicted cloudy with 93–96% confidence; the Moon is below the horizon. Three come from 21:57 on 19 December, labelled cloudy, predicted fine with 95% confidence, under a full Moon at 57° altitude.

Of the 214 errors in total, the distribution over nights is uneven: 99 on 24 November, 49 on 19 December, 32 on 15 November, 31 on 11 December and 3 on 15 December. Nearly half of all errors come from the mixed night. The gallery shows why, and it contains two distinct groups.

Group A: 24 November, 19:21–19:46, labelled fine, predicted cloudy. Nine of the twelve images belong here. The Moon is below the horizon, so moonlight is not the cause. Looking at the frames, the sky is clearly **covered with cloud**: a textured white blanket over almost the whole circle, lit from below by the town. The model says cloudy with 93–96% confidence, and judging by eye, the model is right and the label is wrong.

Group B: 19 December, about 21:57, labelled cloudy, predicted fine. Under a full Moon at 57° altitude, moonlight scattered by the atmosphere makes the sky look almost like daytime, bright blue in a 15-second exposure. Most of the sky in these frames is **clear**, with cloud only near the eastern edge. This is

genuinely ambiguous: a *partly cloudy* sky. Whether it counts as cloudy depends on the labelling rule, for example "not usable for observing". The model's answer is defensible.

The gallery therefore suggests that the measured accuracy *underestimates* the model. A significant share of the "errors" seem to be problems with the labels, not with the network. That claim deserves a direct test.

3.9 Following the labels through the night

The test is simple: plot, for a whole night, the model's probability for every frame together with the given label and the Moon's altitude. If the label stays "fine" while the model confidently says "cloudy" for a continuous stretch, that stretch is suspect.

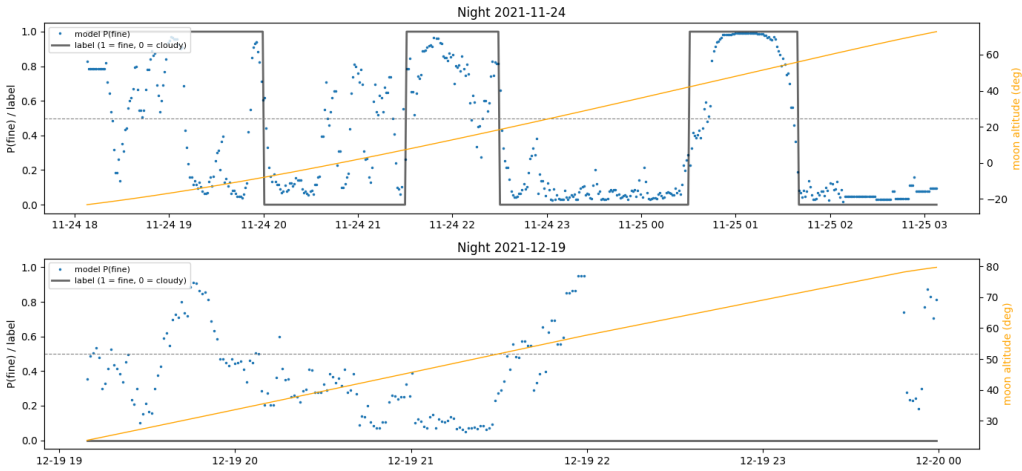


Figure 3.5 The ensemble's P(clear) (blue dots) for every frame of the nights of 24 November (top) and 19 December (bottom), with the dataset label (black line, 1 = fine) and the Moon's altitude (orange, right axis). The labels form rectangular blocks one to two hours long; the model follows changes on a scale of minutes.

3.9.1 The night of 24 November

The black line in the upper panel of Figure 3.5 reveals how the labels were made. It consists of **rectangular blocks**: fine, cloudy, fine, cloudy, fine, cloudy, switching at round-number times near 20:00, 21:30, 22:35, 00:30 and 01:40. Someone labelled time *ranges*, not individual frames.

Table 3.8 Model and labels during the night of 24 November.

Period	Label	Model's P(clear)	Interpretation
18:05–18:30	fine	about 0.8	Agreement
19:00–19:55	fine	about 0.05–0.2	Model confidently disagrees for an hour; the overcast frames of the error gallery; very likely mislabelled
20:00–21:30	cloudy	jumps between 0.1 and 0.8	Partly cloudy: clouds coming and going inside a block labelled cloudy
21:30–22:35	fine	0.6–0.95	Agreement
22:35–00:30	cloudy	about 0.05, very steady	Strong agreement: solid overcast
00:30–01:40	fine	rises to 1.0 within about 10 minutes	Agreement; the short lag is the sky actually clearing
after 01:40	cloudy	about 0.05	Agreement

Where the sky is *steady*, the model agrees with the labels almost perfectly. The disagreements occur inside or at the edges of blocks where the weather was changing, and there the model was more precise than the labels.

3.9.2 The night of 19 December

The lower panel shows a different situation. The **whole night is labelled cloudy**, a single block. The model mostly stays below 0.5, in agreement, but with three clear excursions to 0.8–0.95: around 19:40, between 21:40 and 21:58 (the bright, mostly clear moonlit frames of the error gallery), and around 23:50. These look like genuine breaks in the cloud, folded into an "all cloudy" label. The gap between about 22:00 and 23:45 simply means there are no frames in that period; the camera may have been off.

Notice also that even where the model agrees, it is **less confident** on this night ($P \approx 0.2$–0.5) than on the overcast blocks of 24 November ($P \approx 0.05$). Bright moonlight makes the model uncertain, which ties in neatly with the Moon-altitude result of Section 3.6.

3.9.3 What this means

Three conclusions follow, and they change how the results of this chapter should be read.

1. **The labels are coarse.** They were assigned to intervals of one to two hours, while cloud cover changes within minutes. Frames inside "fine" blocks can be cloudy and vice versa.
2. **The measured accuracy underestimates the model.** A large share of the 214 errors, especially the 99 on 24 November, are frames where the model probably describes the sky better than the label does.
3. **Two classes cannot describe a continuous sky.** Partly cloudy frames have no correct binary answer. Intermediate probabilities between 0.3 and 0.7 are the appropriate response, and the model gives them.

This is a common and important lesson in applied machine learning: the quality of the labels limits the quality of the evaluation. A model can only be measured against the labels it is given, and when the labels are coarse, part of the measured error belongs to the labels.

3.10 Summary

- On six unseen test nights the **small CNN** is the best single model (84.3%), and **ResNet-18 v3** is the most conservative, rarely calling a cloudy sky clear.
- On the **same nights**, hand-crafted features reach only **77.1%**; every network beats them.
- The models **fail on different nights**, so an **ensemble** of all three, chosen on validation, reaches **86.7%** accuracy and AUC 0.924 with the smallest night-to-night spread (0.879 ± 0.086).
- Computing the **Moon's altitude and phase** from the timestamps shows that moonlight is the main difficulty. The ensemble's accuracy varies by only 5 points across Moon altitudes; hand-crafted features vary by 20.
- **Temporal smoothing** adds about one point, revealing that errors come in long, systematic runs.

- The **error gallery** and the **label timelines** show that labels were assigned in one-to-two-hour blocks. Many apparent errors are mislabelled or partly cloudy frames, so the true performance is probably higher than measured.

The next chapter asks two deeper questions. Does the data organise itself into clear and cloudy skies without any labels at all? And when the network makes a decision, what in the image is it actually looking at?

CHAPTER 4

Learning Without Labels and Looking Inside the Network

IN THIS CHAPTER

Two questions remain after the evaluation. First, do the images organise themselves into clear and cloudy skies without anyone telling the computer which is which? Second, when a network makes a decision, what in the image is it actually looking at? This chapter answers both. Clustering pre-trained CNN features recovers the clear/cloudy split with 87% purity and discovers a category missing from the labels: water droplets on the camera dome. Gradient-based maps, occlusion tests and feature maps show that the networks base their decisions on genuine sky regions, not on shortcuts.

4.1 Part I — Learning without labels

4.1.1 Why unsupervised discovery matters

Everything so far relied on labels: someone sorted thousands of frames into fine and cloudy folders. Chapter 3 showed that those labels are coarse and sometimes wrong. Labelling is also expensive; doing it frame by frame for every new camera or season would take far longer than the original block labelling did. It is therefore worth asking whether the structure of the data alone, without any labels, already separates clear from cloudy skies. If it does, a human could label a handful of discovered groups instead of thousands of individual frames. If it does not, the unsupervised analysis may still reveal what the dominant natural variations in the data are.

The approach has three steps: describe each image by a vector of numbers without using labels; group similar vectors into clusters; and only then compare the clusters with the labels to see how well they match.

4.1.2 Two ways to describe an image

Two descriptions were compared, for a random sample of 4,000 frames.

Hand-crafted features. Each image is described by 21 numbers computed on the sky pixels only: a 16-bin histogram of grey values plus five physical quantities from Chapter 1 (mean brightness, contrast, mean gradient, the logarithm of the star-peak count, and the spread between the 99th and 50th brightness percentiles).

CNN embeddings. Each image is passed through ResNet-18 with its ImageNet weights, *without* any training on our data, and the 512 values from the global-average-pooling layer are taken as its description. This **embedding** is the network's internal summary of the image. Because these weights never saw our labels, the embedding is genuinely label-free.

Both descriptions were **standardised**: each of the features was shifted and scaled to zero mean and unit variance across the 4,000 images, so that no single feature dominates merely because of its units. The 512-dimensional embeddings were further compressed with **principal component analysis (PCA)** to the 50 directions of largest variance, which retain most of the information while removing noise and making distances more meaningful.

4.1.3 Clustering methods

K-means divides the data into k clusters by minimising the total squared distance of each point x_i to the centre μ of its cluster C:

$$J = \sum_{j=1}^{k} \sum_{\mathbf{x}_i \in C_j} \left\| \mathbf{x}_i - \boldsymbol{\mu}_j \right\|^2$$

The algorithm alternates between assigning each point to the nearest centre and moving each centre to the mean of its points. It tends to find compact, roughly spherical clusters of similar size.

A **Gaussian mixture model (GMM)** instead assumes the data come from a mixture of k Gaussian distributions, each with its own mean, covariance and weight, and assigns each point to the component most likely to have produced it. Because each component has its own covariance, a GMM can represent elongated, overlapping clusters of different sizes, a better match for a quantity like cloud cover that varies continuously.

4.1.4 Measuring clusters

Four scores were used. One of them uses no labels at all; the other three compare the clusters with the labels after the clustering is done.

- **Silhouette.** For each point, let a be its mean distance to the other points in its cluster and b its mean distance to the points of the nearest other cluster. Its silhouette is $s = (b - a) / \max(a, b)$. Averaged over all points, s ranges from −1 to 1; higher values mean cleaner, better separated clusters.
- **Purity.** Each cluster is assigned its majority class, and purity is the fraction of points that belong to the majority class of their cluster. In this sample the chance level, putting everything in one cluster, is 0.558.
- **Adjusted Rand index (ARI).** The fraction of pairs of points on which the clustering and the labels agree (both together or both apart), corrected so that random clusterings score 0 and identical partitions score 1.
- **Normalised mutual information (NMI).** How much knowing a point's cluster tells you about its label, scaled to the range 0–1.

4.1.5 Results

Table 4.1 Clustering 4,000 frames. Labels are used only to compute ARI, NMI and purity after clustering.

Features	Method	Silhouette	ARI	NMI	Purity
Hand-crafted	K-means, k = 2	0.304	0.005	0.003	0.558
Hand-crafted	K-means, k = 3	0.328	0.083	0.108	0.688
Hand-crafted	K-means, k = 5	0.319	0.083	0.118	0.701
Hand-crafted	GMM, k = 2	—	0.000	—	0.558
CNN embedding	K-means, k = 2	0.174	0.488	0.391	0.850

Features	Method	Silhouette	ARI	NMI	Purity
CNN embedding	K-means, k = 4	0.231	0.252	0.276	0.827
CNN embedding	K-means, k = 5	0.220	0.284	0.389	0.918
CNN embedding	**GMM, k = 2**	—	**0.548**	—	**0.870**

The contrast between the two descriptions is stark.

Hand-crafted features find structure, but not clouds. With two clusters the ARI is 0.005 and the purity exactly the chance level of 0.558: the two clusters have nothing to do with fine versus cloudy. Yet the silhouette, around 0.30–0.33, is the *higher* of the two descriptions. The clusters are well separated, just along a different axis. The mean standardised brightness of the two hand-crafted clusters is +0.83 and −0.88: they simply split **bright skies from dark skies**. The dominant natural variation in simple statistics is moonlight.

CNN embeddings find clouds. With a two-component GMM, the clusters match the labels with ARI 0.548 and purity 0.870. Purity at k = 2 equals the accuracy obtained by mapping each cluster to its majority class. **Without any labels, the clusters agree with fine/cloudy 87% of the time**, about the same as the supervised models achieved on the test nights. The comparison is not exact (different images, no held-out nights), but it is striking. The GMM beats K-means (ARI 0.488), as expected for a continuously varying quantity.

More clusters give purer sub-groups. The silhouette is highest at k = 4 and purity reaches 0.918 at k = 5: the data naturally fall into more than two groups, and each sub-group is mostly one class. The low silhouette values overall (0.17–0.23) show that these groups overlap rather than forming clean islands, consistent with a sky whose cloud cover changes continuously.

4.1.6 Seeing the structure: t-SNE

To see the structure, the 50-dimensional embeddings must be drawn in two dimensions. **t-SNE** (t-distributed stochastic neighbour embedding) does this by placing points so that images that are neighbours in the high-dimensional space stay neighbours on the page. It preserves local neighbourhoods well, but distances between far-apart groups and the sizes of groups on a t-SNE map should not be interpreted literally.

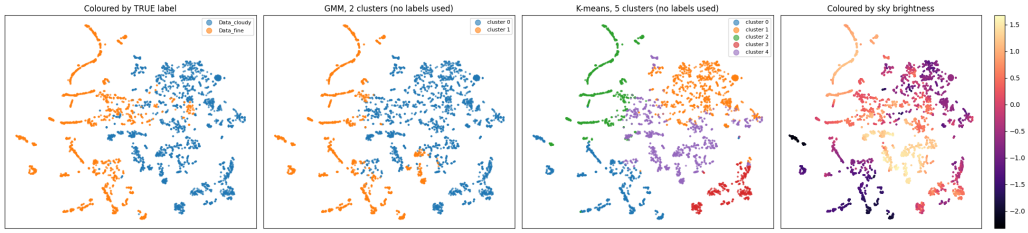


Figure 4.1 Four views of the same t-SNE map of 4,000 frames; each dot is one image. Colour shows, from left to right: the true label; the two GMM clusters (no labels used); the five K-means clusters; and the mean sky brightness.

Figure 4.1 shows four views of the same map.

Panel 1, true labels. Fine images (orange) form long, thin **strands**; cloudy images (blue) form scattered **clumps**. Each strand is one clear night seen frame by frame: a clear sky changes smoothly as the Moon rises and the stars drift, so consecutive frames form continuous paths. Cloudy skies change erratically, so their frames scatter. This is the same pattern seen in the very first brightness-contrast plot of Chapter 1, now appearing in the network's feature space.

Panel 2, GMM clusters. The two clusters match panel 1 closely. Of 2,234 cloudy frames, 2,081 fall in one cluster (93%); of 1,766 fine frames, 1,400 fall in the other (79%). Most of the mismatches sit in a central zone where orange and blue overlap: the partly cloudy and transition frames that also caused the supervised models' errors and the labelling problems of Chapter 3.

Panel 3, five K-means clusters. The data divide into sub-types, examined in detail below.

Panel 4, brightness. Brightness runs *across* both classes: there are dark and bright fine strands, and dark and bright cloudy clumps. **In CNN feature space, brightness and cloudiness are separate directions.** That is precisely why the CNN features succeed where brightness statistics fail.

4.1.7 Five sky types, and one nobody labelled

Table 4.2 The five K-means clusters and their composition.

Cluster	Cloudy	Fine	Purity
0	15	745	98% fine

Cluster	Cloudy	Fine	Purity
1	920	84	92% cloudy
2	26	733	97% fine
3	397	0	100% cloudy
4	876	204	81% cloudy

Statistics alone do not say what the clusters *are*. Figure 4.2 shows six random images from each, and looking at them turned out to be one of the most rewarding moments of the project.

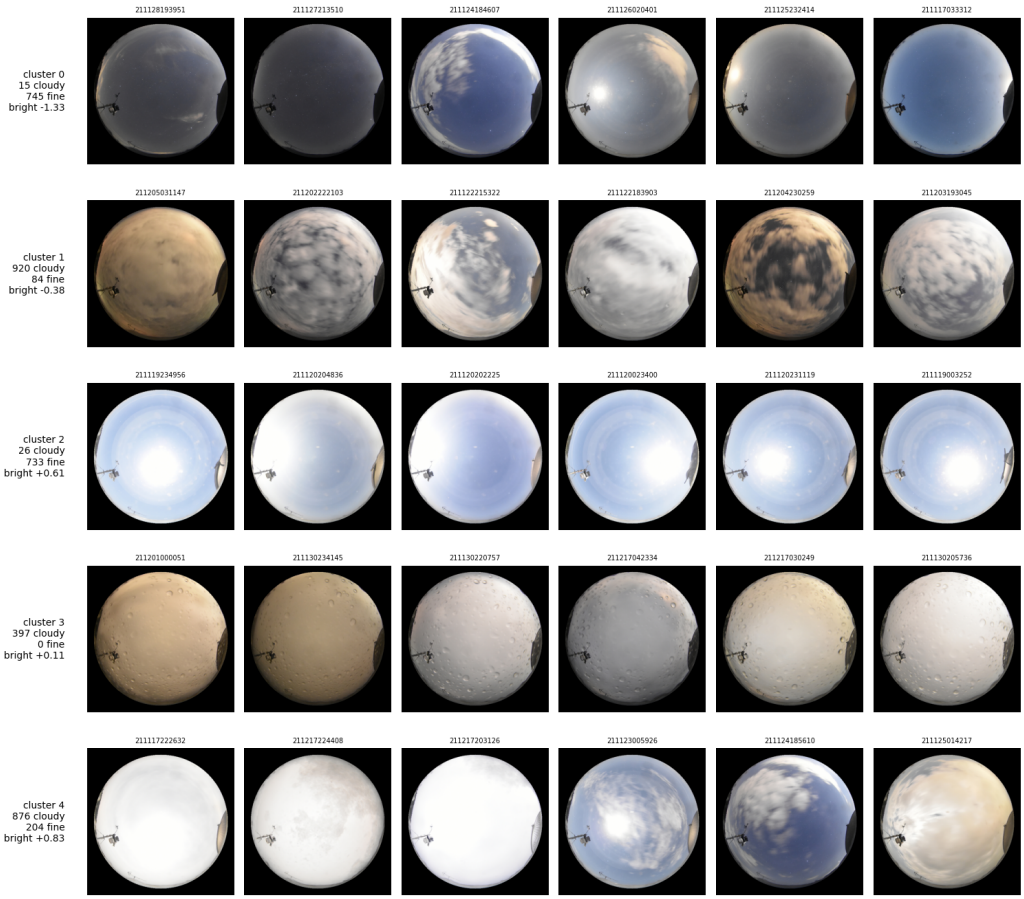


Figure 4.2 Six random frames from each of the five K-means clusters, with each cluster's composition and mean standardised brightness. Cluster 3 contains frames covered with small round spots: water droplets on the camera dome.

Cluster 0, dark clear sky (98% fine). Dark grey to black skies with faint stars, deep blue twilight-like skies and frames with the Moon low near the horizon. The common factor is a dark, mostly cloud-free sky. One frame in the sample, 211124184607, shows obvious cloud patches on a blue sky but is labelled fine; it belongs to the 24 November "fine" block of 18:00–20:00, another example of the coarse labelling found in Chapter 3.

Cluster 1, broken and textured cloud (92% cloudy). Skies with visible cloud structure: dark gaps between clouds, lumpy cloud bases, rippled patterns. Several frames are orange-brown: low cloud lit from below by street lights, a classic sign of a cloud deck over a town.

Cluster 2, moonlit clear sky (97% fine). Bright blue, almost daytime-looking skies with a large glare blob from a high Moon and faint concentric rings from reflections inside the lens. Every sample comes from 19–20 November, the nights around full moon. Moonlight is reflected sunlight, and the atmosphere scatters it blue exactly as it scatters daylight.

Cluster 3, wet camera dome (100% cloudy). Looking closely, these frames are covered with **small circular spots: water droplets on the camera's protective dome**. They come from the nights of 30 November, 1 December and 17 December, evidently rainy or very humid. The network, with no labels at all, separated *rain on the lens* from ordinary cloud.

Cluster 4, bright moonlit cloud (81% cloudy). Very bright, nearly uniform white overcast lit by a bright Moon, and moonlit broken cloud on a blue sky. This is the ambiguous group, where moonlight blurs the difference between thin cloud and clear sky, and where most of the supervised models' errors occur.

Cluster 3 deserves emphasis. All 397 of its frames are labelled cloudy, so the human labels merged two situations that are operationally very different. Under cloud, the right action is not to observe. Under rain, the right action is to **close the dome immediately**, because water endangers the telescope. The unsupervised analysis discovered the most operationally important category in the dataset, one that the original labelling scheme did not represent.

Table 4.3 The five sky types and what they mean for observatory operation.

Cluster	Content	Fine share	Operational meaning
0	Dark clear sky, low Moon	98%	Observe
2	Moonlit clear sky	97%	Observe (bright background)
1	Broken or textured cloud, town-lit cloud base	8%	Do not observe; may clear soon
4	Bright moonlit overcast and broken cloud	19%	Uncertain; needs review
3	Water droplets on the dome	0%	Close the dome; protect the telescope

The practical value is clear. Cluster 3 is 100% cloudy and clusters 0 and 2 are 97–98% fine. An observatory could label a new dataset by having a human name five clusters instead of 8,800 frames, and send only cluster 4 for careful frame-by-frame review. Chapter 5 uses the nearest of these five clusters as a "sky type" for every new image and turns cluster 3 into a rain warning.

4.2 Part II — Looking inside the network

4.2.1 Why interpretation matters

A network that reaches 87% accuracy might still be right for the wrong reasons. Chapter 1 showed how easily it could have read the date instead of the sky. The mask removed that particular shortcut, but others could remain: the mast on the left, the dome edge on the right, the boundary of the sky circle itself, or simply the overall brightness. Before trusting a model to run an observatory, we want to see *what* it looks at. Four complementary tools were used: Grad-CAM, a signed variant of it, occlusion sensitivity, and direct inspection of the first-layer filters and feature maps.

4.2.2 Grad-CAM

Gradient-weighted class activation mapping (**Grad-CAM**) highlights the regions of an image that pushed the network towards its decision. It works on the feature maps A^k of a chosen convolutional layer, usually the last one, where

each map has coarse spatial resolution but rich meaning. First, the gradient of the class score y^c with respect to each feature map is averaged over all positions, giving one importance weight per channel:

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}$$

where Z is the number of positions. The heat map is then the weighted sum of the feature maps, with negative values clipped:

$$L^c_{\text{Grad-CAM}} = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right)$$

The map is upsampled to the image size and overlaid on the image. Red and yellow regions contributed most to the predicted class; blue regions mattered little. For the small CNN the last convolutional layer gives 28 × 28 maps; for ResNet-18, layer4 gives only 7 × 7, so its maps are much blurrier.

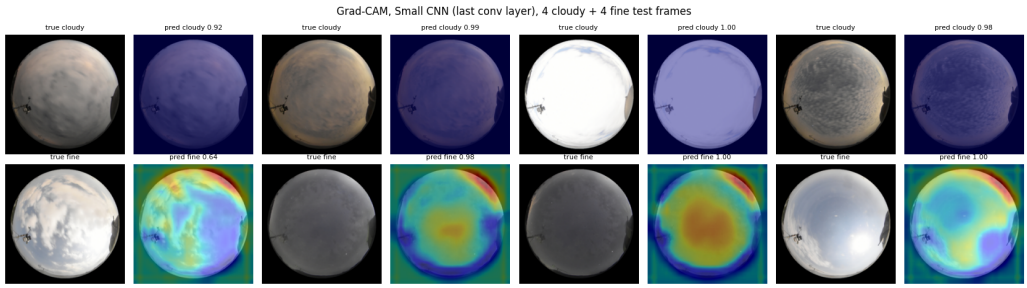


Figure 4.3 Grad-CAM of the small CNN for four cloudy and four fine test frames. The cloudy maps (top row) are completely blank; the fine maps (bottom row) highlight the sky — star fields in the dark frames, and the blue sky around the Moon's glare in the moonlit frame.

Figure 4.3 shows two striking things.

For cloudy frames the heat maps are completely blank. This does not mean the network ignores these images. Standard Grad-CAM keeps only *positive* evidence for the predicted class, through the ReLU in its definition. A blank map means that *no region provides positive evidence for "cloudy"*. The network seems to decide "cloudy" by default, when it does *not* find clear-sky features. Physically this is sensible: a clear sky has specific signatures, such as stars and a uniform dark or blue background, while "cloudy" is everything else. But

standard Grad-CAM cannot show this, which motivates the signed variant below.

For fine frames the attention falls on the sky. In the two dark, starry frames, the attention concentrates on the centre of the sky, where the star field is, exactly what we would hope. In the moonlit clear frame, the attention goes to the blue sky *around* the Moon, while the glare blob itself stays blue. The network has learned to look past the Moon, good evidence that it is not simply measuring brightness. The first fine frame, predicted fine with only 0.64 confidence, contains large clouds; it is another label-noise example, and the model's attention falls on the clear blue gaps between them.

One detail raised a concern: in all four fine frames there is a hot spot at the same position on the upper-right rim of the sky circle. That could be real physics, such as sky glow near the horizon in that direction from town lights, or a shortcut, such as something fixed at the edge of the lens. It needed to be measured, not guessed.

4.2.3 Signed Grad-CAM: evidence in both directions

To see evidence for *both* classes, the method was modified in two ways. Instead of the score of the predicted class, the gradient is taken of the **difference** between the two logits, $z_{\text{fine}} - z_{\text{cloudy}}$, and the final ReLU is removed:

$$L_{\text{signed}} = \sum_k \alpha_k A^k, \qquad \alpha_k = \frac{1}{Z} \sum_{i,j} \frac{\partial \left(z_{\text{fine}} - z_{\text{cloudy}} \right)}{\partial A^k_{ij}}$$

The map is scaled by its largest absolute value and shown with a diverging colour scale: **red regions push the decision towards "fine", blue regions towards "cloudy"**.

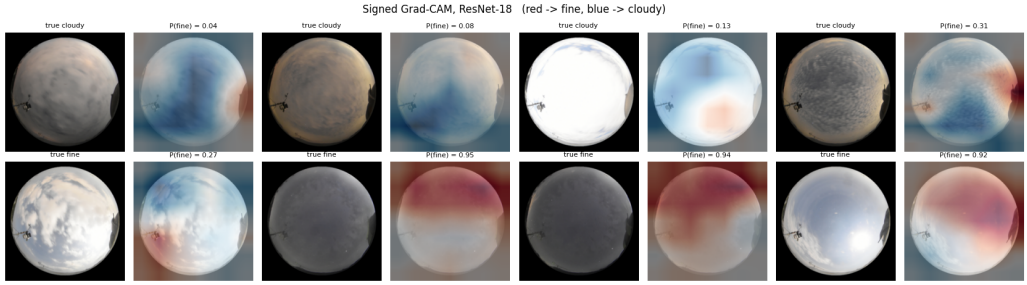


Figure 4.4 Signed Grad-CAM of ResNet-18 on the same eight frames (red: evidence for fine; blue: evidence for cloudy). Cloud-covered sky pushes towards cloudy; the star field in the dark frames and the blue sky around the Moon push towards fine.

Figure 4.4 makes the network's reasoning much clearer.

- **Overcast frames:** broad blue over the cloud-covered sky. The cloudy decision comes from the sky itself, not from fixed objects.
- **Bright white moonlit overcast** ($\mathrm{P}(\text{clear}) = 0.13$): mostly pale blue with a weak red patch. On a bright, nearly uniform sky the evidence is weak and mixed, matching the finding that the moonlit-cloud cluster is the hardest group.
- **Textured broken cloud** ($\mathrm{P}(\text{clear}) = 0.31$): blue over the cloud texture, with a little red at the dome edge and near the mast, a sign of minor influence from fixed structures in this particular frame.
- **Dark starry skies** ($\mathrm{P}(\text{clear}) \approx 0.95$): strong red over the upper, northern half of the sky. A possible physical reason for preferring the north is that this part of the sky may be darker, farther from town glow and the horizon, so stars stand out best there. This remains a hypothesis.
- **Moonlit clear sky** ($\mathrm{P}(\text{clear}) = 0.92$): red spread across the blue sky, avoiding the glare in the lower right. Like the small CNN, ResNet looks past the Moon.
- **The label-noise frame** (labelled fine, $\mathrm{P}(\text{clear}) = 0.27$): blue over the large clouds, red only on the small clear gap near the mast. The model correctly sees mostly cloud; the label is the problem.

4.2.4 A quantitative test for edge shortcuts

To settle the question of the rim hot spot, the share of the signed map's total absolute value lying in the **outer rim** of the sky circle (the outermost 15% of the radius) was computed for 200 random test frames and compared with the rim's share of the sky area.

Table 4.4 Attention on the outer rim of the sky circle, averaged over 200 test frames.

Network	Share of attention on the rim	Rim's share of the sky area	Verdict
Small CNN	0.30	0.28	In proportion to area: no rim bias
ResNet-18	0.24	0.28	Slightly less than area: no rim bias

If a network relied on the rim as a shortcut, its attention there would far exceed the rim's share of the area. Neither network does. The hot spot in Figure 4.3 was a feature of those particular frames, not a systematic shortcut. This quantitative check is more reliable than impressions from a handful of heat maps.

4.2.5 What the network sees when it is "wrong"

The most informative use of interpretation is on the cases where the final model disagreed with the labels. Signed maps were computed for eight of the ensemble's errors on each of the two hard nights.

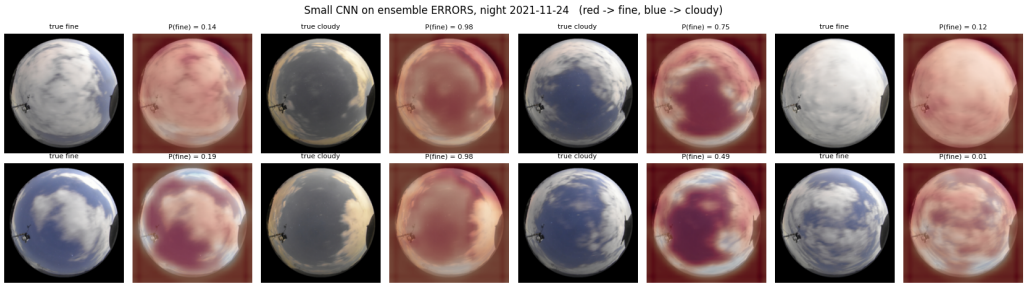


Figure 4.5 Small CNN, signed maps on ensemble errors from the mixed night of 24 November. Darker red marks stronger evidence for fine; read the colours relative to each other (see text).

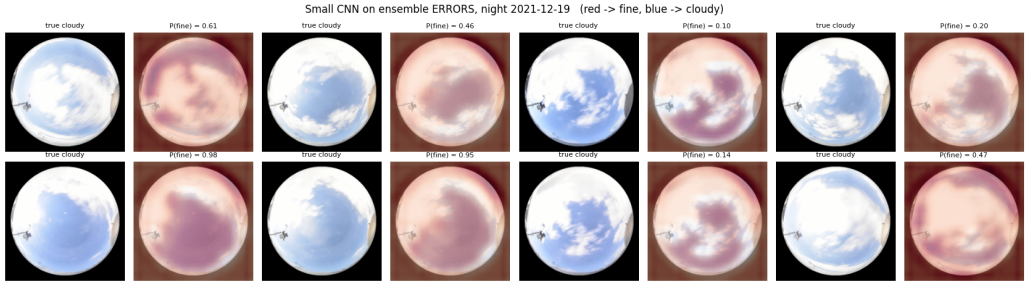


Figure 4.6 Small CNN, signed maps on ensemble errors from the full-moon night of 19 December.

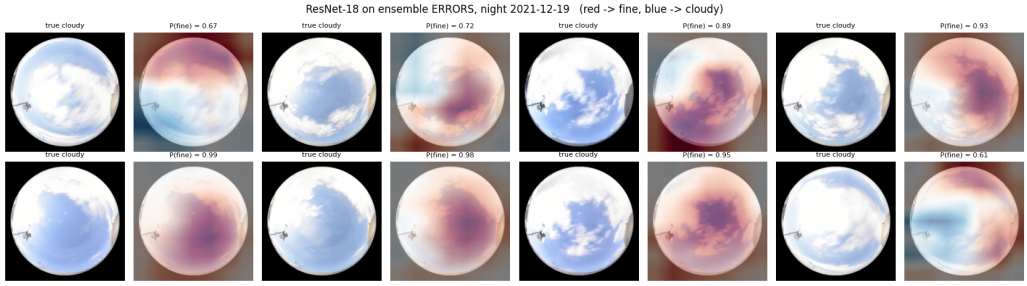


Figure 4.7 ResNet-18, signed maps on the same errors from 19 December. In every frame the red evidence for "fine" sits on the blue patches of clear sky, not on the Moon's glare.

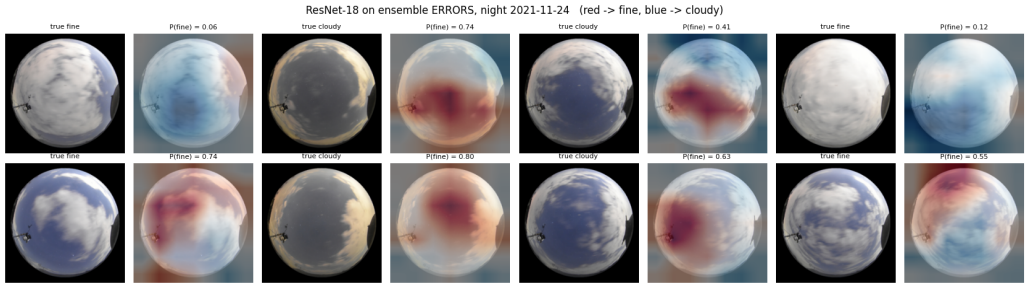


Figure 4.8 ResNet-18, signed maps on the errors from 24 November. "Fine" frames predicted cloudy are covered with cloud (blue evidence); "cloudy" frames predicted fine have a clear, dark centre with cloud only near the horizon (red evidence on the centre).

The full-moon night. All eight frames are labelled cloudy and predicted fine. In ResNet's maps (Figure 4.7), the red evidence sits squarely on the **blue patches of clear sky**, while the white cloud near the edges stays neutral or blue. These frames have large clear gaps, roughly 40–60% of the sky, with cloud mainly around the horizon. The Moon did not fool the model: the red lies on

the clear sky, not on bright glare. The label is the issue: the whole night was labelled cloudy as one block, but for these minutes the sky was partly open.

The mixed night. Here the errors run in both directions.

Table 4.5 What the networks see in the errors of 24 November.

Frame type	What the image shows	ResNet-18 evidence	Verdict
Labelled fine, predicted cloudy (P = 0.06, 0.12)	Sky covered with textured cloud	Blue across the cloud	Model right, label wrong
Labelled cloudy, predicted fine (P = 0.74–0.98)	Dark, clear centre; town-lit cloud only near the horizon	Red on the clear centre	Defensible: the zenith, where a telescope mostly points, is clear
Labelled cloudy, P = 0.41–0.63	Large blue clear patch surrounded by cloud	Red on the clear patch	Genuinely partly cloudy
Labelled fine, P = 0.55–0.74	Blue sky with large clouds	Mixed red and blue	Genuinely partly cloudy

The final model's errors therefore fall into two groups: labelling errors, where overcast frames sit inside "fine" time blocks and the model is right; and partly cloudy frames, which a two-class label cannot describe and where the model's intermediate probability correctly signals uncertainty. Very few errors look like genuine model failures.

A technical caveat applies to the small CNN's maps (Figures 4.5 and 4.6). They appear reddish everywhere, even outside the sky circle and even when P(clear) is 0.01, so they must be read **relatively**: darker red means more evidence for fine. Read that way, they agree with ResNet: darker red on the blue clear gaps, lighter over clouds. The overall tint arises because these maps were computed at the last convolution *before* batch normalisation, ReLU and pooling, so they do not add up linearly to the output. Chapter 5 shows how an exact, artefact-free version of these maps can be obtained at the network's final layer.

4.2.6 Occlusion sensitivity

Gradient-based maps depend on the network's internal mechanics. **Occlusion sensitivity** asks the same question without gradients: hide one patch of the

image at a time and see how much the prediction changes. If hiding a region lowers P(clear), that region supported "fine".

The usual choice is to cover each patch with a grey square, but on a night sky a grey square would *look like cloud* and bias the test. Instead, each 32 × 32 patch, moved in steps of 16 pixels, was replaced by a **heavily blurred copy of itself** (Gaussian blur with $\sigma = 12$ pixels). Blurring keeps the patch's brightness and colour but removes stars and cloud texture, so the test measures how much the network relies on *structure* rather than brightness. The colour scale shows the change in P(clear): red means blurring that patch lowered P(clear), so its texture supported "fine"; blue means its texture supported "cloudy".

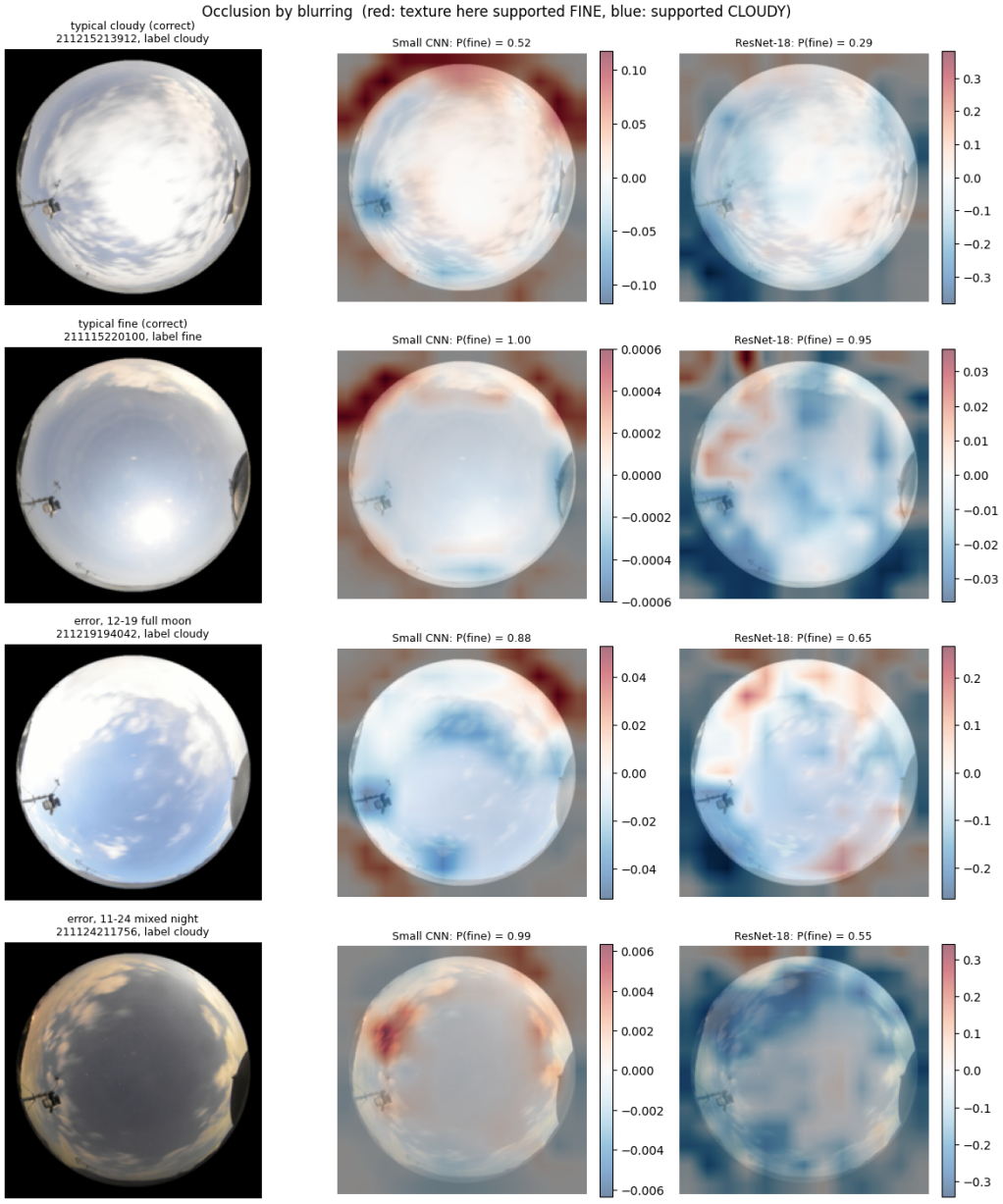


Figure 4.9 Occlusion by blurring for four frames: a typical cloudy frame, a typical fine frame, an error from 19 December and an error from 24 November. Note the very different colour-bar scales: confident predictions barely change at all.

Reading Figure 4.9 requires attention to the colour-bar scales, because the size of the effect matters more than its colour.

Table 4.6 Largest change in P(clear) when a single patch is blurred.

Frame	Small CNN P(clear)	Max. change	ResNet-18 P(clear)	Max. change
Typical cloudy (moonlit broken cloud)	0.52	±0.11	0.29	±0.35
Typical fine (moonlit clear)	1.00	±0.0006	0.95	±0.03
Error, 19 December	0.88	±0.05	0.65	±0.25
Error, 24 November	0.99	±0.006	0.55	±0.30

Three findings emerge.

1. **Confident decisions are holistic.** When a network is sure ($P \approx 0.99$–1.00), blurring any single patch changes almost nothing, as little as ± 0.0006. The decision is not based on one feature in one place but on the sky as a whole. This is a valuable robustness property: a bird, an aircraft or a dewdrop in one spot cannot flip a confident prediction.
2. **Sensitivity appears only for uncertain frames, mostly at cloud edges.** Where P lies between 0.3 and 0.9, blurring has real effects, up to ± 0.35. In the 19 December error, for example, the small CNN shows blue along the cloud edge bordering the clear gap: crisp cloud boundaries are evidence for "cloudy". This agrees with Grad-CAM.
3. **The strongest effects at the circle's edge are artefacts.** Much of the intense colour, especially in ResNet's maps, lies on the boundary of the sky circle and in the black corners. Blurring a patch that straddles the mask boundary smears bright sky into the black area and black into the sky, creating an artificial ring the network never saw in training. These effects describe the *method*, not the sky; a refined version that blurs only within the sky circle removes them.

Different interpretation methods therefore agree only partly. Grad-CAM, its signed variant and the quantitative rim check give the most reliable picture; occlusion adds the insight that confident decisions are holistic. Using several methods and understanding their limits is better practice than trusting any single heat map.

4.2.7 Inside the first layer

Finally, we can look directly at what the small CNN's first layer has learned. Figure 4.10 shows its 32 filters, each a 3 × 3 patch of colour weights.

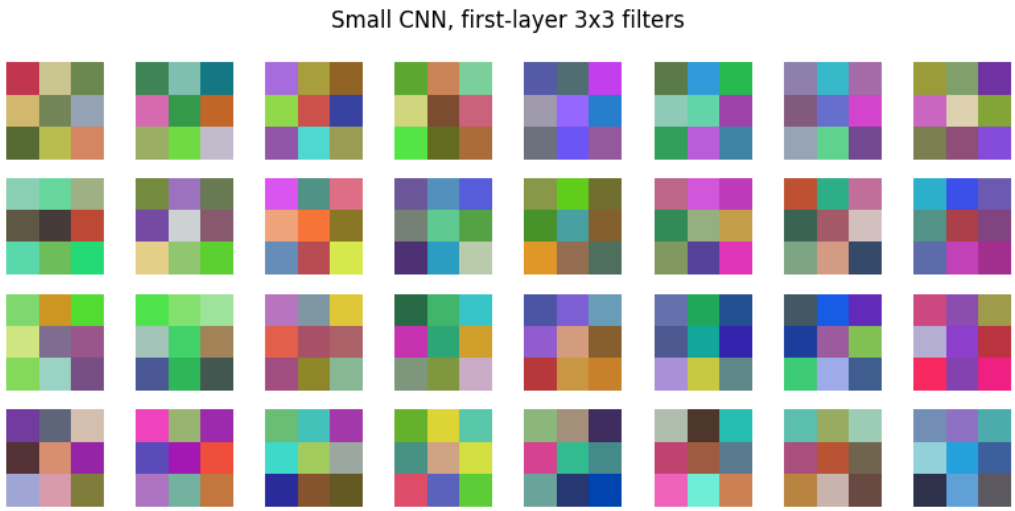


Figure 4.10 The 32 learned 3 × 3 colour filters of the small CNN's first convolutional layer, displayed after a common min–max scaling.

The filters look like random colour mosaics, without recognisable edge or spot patterns. This is normal and worth stating honestly. A 3 × 3 filter has only nine pixels, too few to show recognisable shapes, and clean, Gabor-like filters usually emerge only with much more training data than 15 epochs on about 24 nights. Some filters do show **colour opponency**, for example blue against orange or green, which suits the task of separating blue moonlit sky from white or town-lit orange cloud.

The **feature maps**, the outputs of these filters for an actual image, are far more informative.

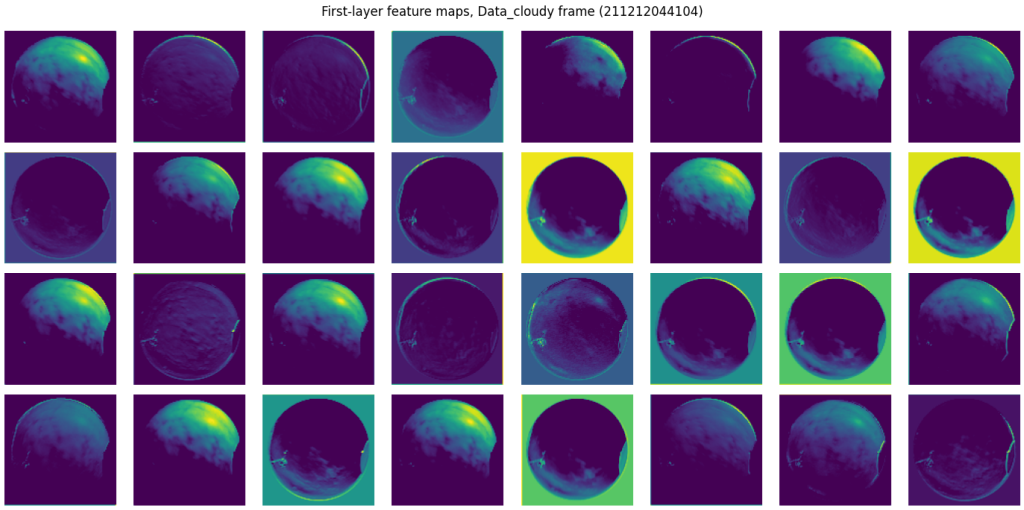


Figure 4.11 The 32 first-layer feature maps of the small CNN for a cloudy frame (211212044104). Brightness detectors, darkness detectors, oriented edge detectors and texture detectors can all be recognised.

Four kinds of detector can be recognised in Figure 4.11:

- **Brightness detectors** produce a smooth glow over the brighter upper-right part of the sky, with dark lumps where cloud blocks the light.
- **Darkness detectors** do the opposite: the black area outside the circle and the dark lower sky light up.
- **Oriented edge detectors** produce thin bright arcs on one side of the circle, responding to edges in a particular direction.
- **Texture detectors** produce an embossed relief of cloud lumps across the whole sky. They capture cloud structure, the key cue.

Two observations stand out. First, there is **heavy redundancy**: about a dozen maps are near-copies of the same brightness pattern. The first layer is wasting capacity, which suggests a smaller first layer would do as well and fits the overfitting seen in Chapter 2. Second, the **mask edge activates the edge detectors strongly**, yet the rim check showed that this does not carry through to the final decision. Later layers learn to ignore it.

4.3 Summary

- **Without labels**, clustering pre-trained CNN features recovers the clear/cloudy split with **ARI 0.55 and 87% purity**, while clustering hand-crafted statistics merely separates bright from dark skies (ARI ≈ 0).
- In CNN feature space, **brightness and cloudiness are separate directions**, which is why CNN features succeed where brightness statistics fail.
- Five clusters reveal **physically meaningful sky types**: dark clear, moonlit clear, broken cloud, bright moonlit cloud and, missing from the original labels, **water droplets on the camera dome**, the most important category for protecting the telescope.
- **Grad-CAM** shows that the networks attend to the sky: star fields and clear gaps support "fine", cloud structure supports "cloudy", and the Moon's glare is looked past rather than looked at. A **signed** variant shows evidence in both directions.
- A **quantitative rim test** rules out edge and mask shortcuts.
- On the hardest nights, the maps confirm that most apparent errors are **mislabelled or partly cloudy frames**.
- **Occlusion** shows that confident decisions are holistic, and that attribution methods have artefacts of their own; **feature maps** reveal brightness, darkness, edge and texture detectors.

The analysis has now answered the scientific questions of the project. The final chapter turns the models into a tool that an observatory could actually use.

CHAPTER 5

From Notebook to Observatory Tool

IN THIS CHAPTER

A model that lives only in a notebook helps nobody at the observatory. This chapter describes how the three trained networks became Sky Monitor AI, an eight-page application that runs on an ordinary computer without a GPU or PyTorch. It explains the design decisions: exporting the models to ONNX and verifying them, turning the networks' final layers into exact evidence maps and a cloud-cover estimate, converting probabilities into five practical sky conditions, accepting images from files, Google Drive folders and live cameras, and checking the camera against weather forecasts and satellite images. It closes with how the tool was tested, what it cannot do, and where the project should go next.

5.1 What an observatory tool must do

The experiments of Chapters 1 to 4 answered scientific questions. An observatory operator has practical ones. Is the sky clear right now? Has it been clear for the last half hour? Should the dome be closed because rain is starting? Can I trust this answer? The application had to translate the models into answers to these questions, and it had to do so under realistic constraints.

Five requirements guided the design.

1. **It must run anywhere.** An observatory computer is typically an ordinary desktop or laptop, often without a graphics card, sometimes without a

reliable internet connection. The application must run on a CPU, offline, on Windows, macOS or Linux.

2. **It must accept images the way cameras deliver them.** Some all-sky cameras write a JPEG file to a folder every minute; others publish their latest image at a web address; others provide a video stream. Archives of past frames may sit in a shared Google Drive folder. All of these should be usable without writing code.
3. **It must give an actionable answer, not just a probability.** "$P(\text{clear}) = 0.73$" is less useful at two in the morning than "Mostly clear — observe, but monitor passing clouds".
4. **It must show its reasoning.** Chapter 4 showed that interpretation exposes both the strengths and the failure modes of the models. The operator should be able to see, for every frame, which parts of the sky the decision rests on.
5. **It must say when it should not be trusted.** Images from a different camera, or conditions never seen in training, should trigger a warning rather than a confident answer.

The result is **Sky Monitor AI**, a web application built with Streamlit, a Python framework for data applications. It opens in an ordinary browser, either from a cloud server or locally at `http://localhost:8501`.

5.2 Architecture

5.2.1 Eight pages

The application is organised into eight pages, reachable from a menu on the left:

Table 5.1 The pages of Sky Monitor AI and what each provides.

Page	Purpose	Main inputs	Main outputs
Home	Overview and navigation	—	Key results, links to every page
Classify a sky	Judge one image	Sample, upload, Drive or web link, camera snapshot	Condition, P(clear), cloud cover, sky type, evidence map, per-model chart

Page	Purpose	Main inputs	Main outputs
Night timeline & batch	Judge many images	Test nights, uploads or zip, Drive folder, local folder	Condition counts, timeline with Moon altitude, frame viewer, CSV
Live camera	Continuous monitoring	Watched folder, latest-image URL, USB or RTSP stream	Live condition, evidence map, running chart, alerts, log
Weather & satellite	Independent check	Location (list, name or coordinates), time, optional camera frame	Forecast with cloud layers, satellite picture, camera-versus-forecast comparison
Sky quiz	Engagement and intuition	Your answer	Label, model verdict, running score
Explore sky types	Unsupervised structure	Colouring, selections on a map	Interactive t-SNE map, cluster gallery, clustering scores
Results	Evidence	Decision threshold	Model comparison, per-night accuracy, Moon effect, smoothing, confusion matrices, training curves

Chapter 6 walks through every page with screenshots. This chapter explains what happens behind them.

5.2.2 Code structure

The code is deliberately small and layered:

- `streamlit_app.py` defines the page menu and makes sure the model files are present.
- `skyapp/core.py` holds everything scientific: loading the models, preprocessing images exactly as in training, running the networks, computing evidence maps and cloud cover, finding the sky type, turning numbers into a verdict, and computing metrics such as AUC and smoothed probabilities.
- `skyapp/sources.py` handles the outside world: Google Drive links, web URLs, local folders and live cameras.
- `skyapp/weather.py` fetches forecasts and satellite pictures from two free web services.

- `skyapp/ui.py` contains the shared visual elements, such as the coloured verdict card and the evidence-map figure.
- `app_pages/` contains one short file per page, which only arranges inputs and outputs and calls the functions above.

This separation means the scientific logic is written once and used identically by every page. A frame classified on the *Classify* page, in a *timeline* batch or by the *live* monitor passes through exactly the same code.

5.3 Leaving PyTorch behind: ONNX export

5.3.1 Why export?

The models were trained in PyTorch on a GPU. Installing PyTorch on an observatory computer is possible but heavy, around two gigabytes, and unnecessary, because the application only needs to run the trained networks, not train them. The **Open Neural Network Exchange (ONNX)** format solves this. An ONNX file stores a network's structure and weights in a framework-independent form, and the lightweight **ONNX Runtime** library, about 15 megabytes, can execute it efficiently on an ordinary CPU.

5.3.2 Two outputs per network

Each classifier was exported with **two outputs** rather than one: the class logits, and the feature maps of the last convolutional stage ($14 \times 14 \times 256$ for the small CNN, $7 \times 7 \times 512$ for each ResNet-18). The second output is what makes exact evidence maps possible without gradients, as Section 5.4 explains. The ImageNet ResNet-18 used for the unsupervised analysis was exported too, as an **embedder** that returns the 512-number description of an image used to find its sky type.

All four networks were exported with a dynamic batch dimension, so the application can process one frame or a batch of 32 in a single call.

5.3.3 Verifying the export

An export is only useful if it reproduces the original exactly. Four test images were passed through both the PyTorch models and the exported ONNX files, and the largest absolute difference between their outputs was recorded.

Table 5.2 Agreement between the exported ONNX models and the original PyTorch models.

Network	Largest difference between outputs
Small CNN	7.2×10^{-7}
ResNet-18 v2	4.4×10^{-6}
ResNet-18 v3	7.7×10^{-7}
Embedder (ImageNet ResNet-18)	3.1×10^{-6}

The differences are at the level of floating-point rounding, a millionth or less. Two further checks confirmed the rest of the export. The test-set probabilities recomputed from the exported models matched the results of Chapter 3 with a maximum difference of exactly 0.0, and the re-fitted cluster model assigned all 4,000 clustering images to the same sky types as in Chapter 4 (agreement 1.0). The application therefore runs exactly the models whose performance this book reports.

5.3.4 Identical preprocessing

A frequent source of silent errors in deployed models is a mismatch between training-time and run-time preprocessing. The application repeats every step of Chapter 2 exactly: convert to RGB, apply the circular sky mask with radius 0.455 × width, resize to 224 × 224 with bilinear interpolation, scale to 0–1 and normalise with the ImageNet means and standard deviations. Because the verification above used images preprocessed by the training code and the application uses the same constants, any mismatch would have shown up as a difference in the test probabilities. None did.

5.4 Evidence maps without gradients

5.4.1 An exact decomposition

Chapter 4 used Grad-CAM, which requires gradients and therefore a training framework. The application obtains comparable maps without any gradients, by exploiting the structure of the networks' final layers.

Every one of the three classifiers ends with global average pooling followed by a single linear layer (Section 2.2.4). Let A(x, y) be the vector of feature values at position (x, y) of the last feature map, W and b the weights and biases of the linear layer, and N the number of positions. The logit of class k is

$$z_k = \sum_c W_{k,c} \frac{1}{N} \sum_{x,y} A_c(x,y) + b_k$$

The quantity that decides between the two classes is the difference of the logits. Because both the averaging and the linear layer are linear operations, they can be exchanged:

$$z_{\text{fine}} - z_{\text{cloudy}} = \frac{1}{N} \sum_{x,y} \Big[\underbrace{\Delta\mathbf{w} \cdot \mathbf{A}(x,y) + \Delta b}_{\ell(x,y)} \Big]$$

where $\Delta\mathbf{w} = \mathbf{W}_{\text{fine}} - \mathbf{W}_{\text{cloudy}}$ and $\Delta b = b_{\text{fine}} - b_{\text{cloudy}}$. The bracketed term $\ell(x, y)$ is a **local logit**: the decision the network would make if the whole image looked like position (x, y). The network's actual decision is exactly the average of these local decisions. Applying the sigmoid function to each local logit gives a **local probability of a clear sky**,

$$p_{\text{local}}(x,y) = \frac{1}{1 + e^{-\ell(x,y)}}$$

which is what the application draws on top of the image: red where a region, taken alone, would make the network say clear; blue where it would say cloudy.

This decomposition is exact, not an approximation. During testing, the mean of each model's local-logit map agreed with the model's actual logit difference to within 10^{-5} for every image. It also removes the tinting artefact noted for the

small CNN in Chapter 4, because it works at the true final layer, after batch normalisation, ReLU and pooling.

5.4.2 The ensemble map and the cloud-cover estimate

Each network produces its own map at its own resolution: 14 × 14 for the small CNN, 7 × 7 for the ResNets. The application upsamples all three maps to 224 × 224 with bilinear interpolation and averages them, producing the **ensemble evidence map**. It then defines the **estimated cloud cover** as the fraction of sky pixels (inside the mask) whose ensemble local probability is below 0.5:

$$\text{cover} = \frac{\#\{(x,y) \in \text{sky} : p_{\text{local}}(x,y) < 0.5\}}{\#\{(x,y) \in \text{sky}\}}$$

The cloud-cover estimate gives the binary classifiers a way to describe partly cloudy skies. It is not a calibrated physical measurement; a cell can vote "cloudy" because of glare or a fixed structure as well as because of cloud. But it captures the most useful spatial information the networks have, and on real frames it behaves sensibly. The full-moon frame used as a sample in the application, labelled *cloudy* in the dataset, receives $\text{P}(\text{clear}) = 0.95$ and a cloud cover of 9%, with blue evidence exactly over the cloud bank near the eastern horizon. That is precisely the partly cloudy situation diagnosed in Chapters 3 and 4.

5.5 Sky type and the out-of-distribution warning

For the sky type, the application passes each image through the embedder, applies the same standardisation and PCA as in Chapter 4, and assigns the image to the nearest of the five K-means centres: dark clear sky, broken or textured cloud, moonlit clear sky, wet dome, or bright moonlit overcast.

The distance to that nearest centre also provides a simple **out-of-distribution check**. For each cluster, the 99th percentile of the distances of its own clustering images to the centre was stored. If a new image lies more than 1.2 times this distance from its nearest centre, the application warns that the image looks unlike anything in the training data. A photograph from another camera, a

daytime frame or a severely corrupted image will usually trigger this warning, telling the operator not to trust the verdict.

5.6 From numbers to a decision

5.6.1 Five sky conditions

The models output a probability; the operator needs a condition. The application combines three quantities, P(clear) from the ensemble, the estimated cloud cover and the sky type, into one of five conditions, each with a recommended action.

Table 5.3 The five sky conditions shown by the application.

Condition	Rule (default thresholds)	Recommended action
☀️ Clear	P(clear) ≥ 0.70 and cloud cover ≤ 30%	Open the dome; good observing conditions
🌤️ Mostly clear	In between, with cloudiness index < 0.5	Observe, but monitor passing clouds
⛅ Mostly cloudy	In between, with cloudiness index ≥ 0.5	Stand by; wait for a clear gap
☁️ Overcast	P(clear) ≤ 0.30 and cloud cover ≥ 70%	Keep the dome closed
🌧️ Wet dome	Sky type = wet dome cluster	Close the dome; water on the camera dome, rain or dew likely

The **cloudiness index** used for the two intermediate levels is the average of the two indications of cloud,

$$c = \tfrac{1}{2}\,(1 - P(\text{clear})) + \tfrac{1}{2}\,\text{cover}$$

so that a frame is "mostly clear" when probability and cloud cover together lean towards clear.

The wet-dome rule takes precedence over everything else, because it is the condition with the most serious consequences. It comes directly from the discovery in Chapter 4 that one unsupervised cluster consists entirely of frames with water droplets on the camera dome. All thresholds can be adjusted in the

application, so that an observatory can make the system more cautious or more permissive.

It is important to be clear about what these levels are. The networks were trained on two classes only. The intermediate levels and the wet-dome warning are derived from the evidence map and the unsupervised clusters; they are informed interpretations, not separately trained and validated classes. The application says so on its home page.

5.6.2 Smoothing in operation

Chapter 3 showed that a median over neighbouring frames removes isolated errors but cannot fix systematic ones, and that a live system must use a *trailing* window. The timeline page offers both centred and trailing windows from 1 to 21 frames, with five frames as the default, so that the operator can see the trade-off between stability and reaction time directly.

5.7 Getting images in

5.7.1 Single images and batches

Uploading one image or a set of images is the simplest route, and Streamlit provides it directly. Two additions make it practical for observatory archives. A **zip file** of frames can be uploaded and is unpacked automatically, which is much faster than selecting hundreds of files. And frames are ordered by the **timestamp in their file names** (`YYMMDDhhmmss`), from which the application also computes the Moon's altitude for each frame with the method of Section 3.6. A batch of up to 600 frames is analysed in groups of 32 for efficiency.

5.7.2 Google Drive folders

Many research groups share data through Google Drive, as this project did. The application accepts the link of a shared folder ("anyone with the link can view") and reads its listing, including subfolders such as `Data_fine` and `Data_cloudy`. Common download tools stop at 50 files per folder, which would make them useless for archives of thousands of frames. The application instead reads Drive's embedded folder view, which lists every entry of a public folder on a single page. Each entry carries its file identifier, its name and whether it is

a subfolder. The user then chooses which folder, how many frames and which ones (the latest, the first, or evenly spread), and the selected files are downloaded one by one and analysed as a batch.

This path was tested on the computer where the application was installed, using the project's own shared folder. The listing found **8,806 images in 2 folders**, matching the dataset exactly. Twenty evenly spread frames from `Data_fine` were then downloaded, at roughly three seconds per frame, and classified as 18 clear, 1 mostly cloudy and 1 overcast. A single Drive file or a shared zip file can be given in the same way.

5.7.3 Live cameras

Continuous monitoring is the purpose the original assignment had in mind. All-sky cameras deliver their images in three common ways, and the *Live camera* page supports all three.

- **Watch a folder.** Most all-sky camera software saves a JPEG to a folder every minute. The application checks the folder at a chosen interval and analyses the newest file whenever its name or modification time changes.
- **Latest-image URL.** Some cameras publish their current picture at a fixed web address. The application downloads it at each interval, adding a time parameter to defeat caching, and analyses it whenever its content changes.
- **Camera or video stream.** A USB camera (given by its index, 0 for the first) or an IP camera with an RTSP or MJPEG stream is read with the OpenCV library; a few frames are read and discarded first, because the first frames of a stream are often dark or stale.

Each new frame is classified, displayed with its verdict and evidence map, and appended to a log of up to about two days at one frame per minute. A chart shows P(clear) and cloud cover since monitoring began, with the background coloured by condition. When the condition changes, for example from clear to mostly cloudy, a pop-up alert appears. Grabbed frames can optionally be saved to a folder, and the log can be downloaded as a CSV file.

The timing is handled by a Streamlit *fragment* that reruns at the chosen interval, so only the live panel refreshes, not the whole page. The folder and USB

options require the application to run on the computer connected to the camera, which is exactly what the standalone package of the next section is for.

5.7.4 An independent check: weather and satellite

A camera-based classifier has one blind spot that no amount of training removes: it cannot tell whether it is wrong. The last addition to the application gives the operator two independent opinions to set beside the camera's verdict. Both come from free services that need no account.

The first is a **weather forecast** from Open-Meteo, which combines the output of several national weather models. For any latitude and longitude it returns hourly values of temperature, humidity, wind, precipitation and, most usefully here, **cloud cover**, split into low, mid and high cloud. For dates older than about three months, the same service provides the **ERA5 reanalysis** of the European Centre for Medium-Range Weather Forecasts: a physically consistent reconstruction of past weather from all available observations. This makes it possible to ask what the weather was like on the 2021 nights of the dataset, at the hour a frame was taken.

The second is a **satellite picture** from NASA's Global Imagery Browse Services (GIBS). Over Japan and the western Pacific, Japan's geostationary Himawari-9 satellite images the whole disc every ten minutes. Its infrared band at 10.4 μm measures the temperature of whatever it sees, so cold cloud tops appear bright against the warm ground by day and by night. Over the Americas the GOES satellites play the same role. For other regions, and for dates before the geostationary archive, the application falls back to the daily daytime pass of the polar-orbiting VIIRS instrument. The application requests a picture of the area around the chosen location through the standard Web Map Service protocol, steps back in ten-minute intervals until an image is available, overlays coastlines and marks the location.

On the *Weather & satellite* page the forecast and the satellite picture appear side by side, for a location the user selects from a list of observatories and cities, finds by name, or enters as coordinates and saves. When the time is taken from a camera frame, the page also classifies the frame and compares the camera's estimated cloud cover with the forecast's. The two measure different things — a grid square of 10 to 30 kilometres and all cloud layers on one side, the sky

above one camera on the other — so the comparison is a plausibility check, not a validation. Its value lies in the disagreements: a confident "clear" from the camera while the forecast reports 90% low cloud and the satellite shows a cloud band overhead is a frame worth a second look. Used systematically over a season, the same comparison would be a cheap way to find mislabelled blocks of frames, the largest limitation identified in Chapter 3.

5.8 Packaging: one zip, one double-click

The final package is a single zip file of about 150 MB containing the application code, the four ONNX models with their parameters, the test-set predictions, the t-SNE data, thumbnails and sample frames. Three launchers make it usable without command-line knowledge:

- `run_windows.bat` for Windows,
- `run_mac.command` for macOS,
- `run_linux.sh` for Linux.

On first start, a launcher checks that Python is installed, creates a private environment in a `.venv` folder inside the application folder, installs the required libraries (Streamlit, ONNX Runtime, NumPy, pandas, Pillow, Plotly, Astropy, gdown and OpenCV) and starts the application in the browser. This takes a few minutes and needs an internet connection once. Later starts take seconds and work offline. Because the environment is private, the application cannot interfere with other Python software on the computer. Users of Anaconda can equally create a conda environment and start the application from a terminal, as Chapter 6 shows.

For sharing online, the same code can be published on GitHub and deployed to Streamlit Community Cloud, with the model files attached to a GitHub Release and downloaded automatically on first start. The online version supports uploads, links, Drive folders and the weather page, but not local folders or USB cameras, which a cloud server cannot reach.

5.9 Testing the tool

5.9.1 Tests without the real models

The application was developed in an environment that did not have the trained models, which live in the training notebook. To test it anyway, a **stand-in asset pack** was generated: small dummy ONNX networks with exactly the same inputs and outputs as the real ones, combined with real frames from the dataset. With these, every page was exercised automatically with Streamlit's testing framework, including every sample, every input source and every control. Each camera type was simulated: a folder of sky frames for the folder watcher, a small local web server publishing `latest.jpg` for the URL mode, and a video file standing in for a camera stream. The launcher was tested from a fresh copy: it created its environment, installed every library and started a responding application. The weather page was tested in the same way, with simulated replies in the exact format of the two web services, covering every way of choosing a place and a time, the switch to the reanalysis for old dates and the satellite fallback. The evidence-map identity of Section 5.4.1 and the Moon calculation (57.1° and −10.3° for the two reference frames) were checked numerically.

5.9.2 Tests with the real models

Once the real models were exported, the application was installed on an Ubuntu workstation in a conda environment with Python 3.11 and checked page by page. The home page shows the test accuracy of 86.7% computed from the exported predictions, confirming that the real models are in place. The *Results* page reproduces every number of Chapter 3. The *Classify* page gives the expected verdicts on the sample frames, including the 🌧️ wet-dome warning for a frame from 17 December. The Google Drive path worked end to end, as described in Section 5.7.2.

The real-world check also found three display problems that the automated tests had missed, which is a good argument for always testing on the target machine. The interactive t-SNE map stayed blank, because the plotting library switches to WebGL for more than 1,000 points and WebGL was disabled in that browser; the map is now drawn as SVG, which works everywhere. The per-

night heat map interpreted night labels as calendar dates, giving columns of uneven width; the axis is now categorical. And the Moon-altitude chart listed "below horizon" last instead of first; the categories are now in physical order.

5.9.3 Images the model was not built for

What happens when someone gives the application an image it was never designed for? To find out, an ordinary daytime phone photograph of a blue sky with cirrus clouds was converted to the camera's format: cropped to a square, warped to imitate a fisheye lens, masked to a circle and given a timestamped file name.

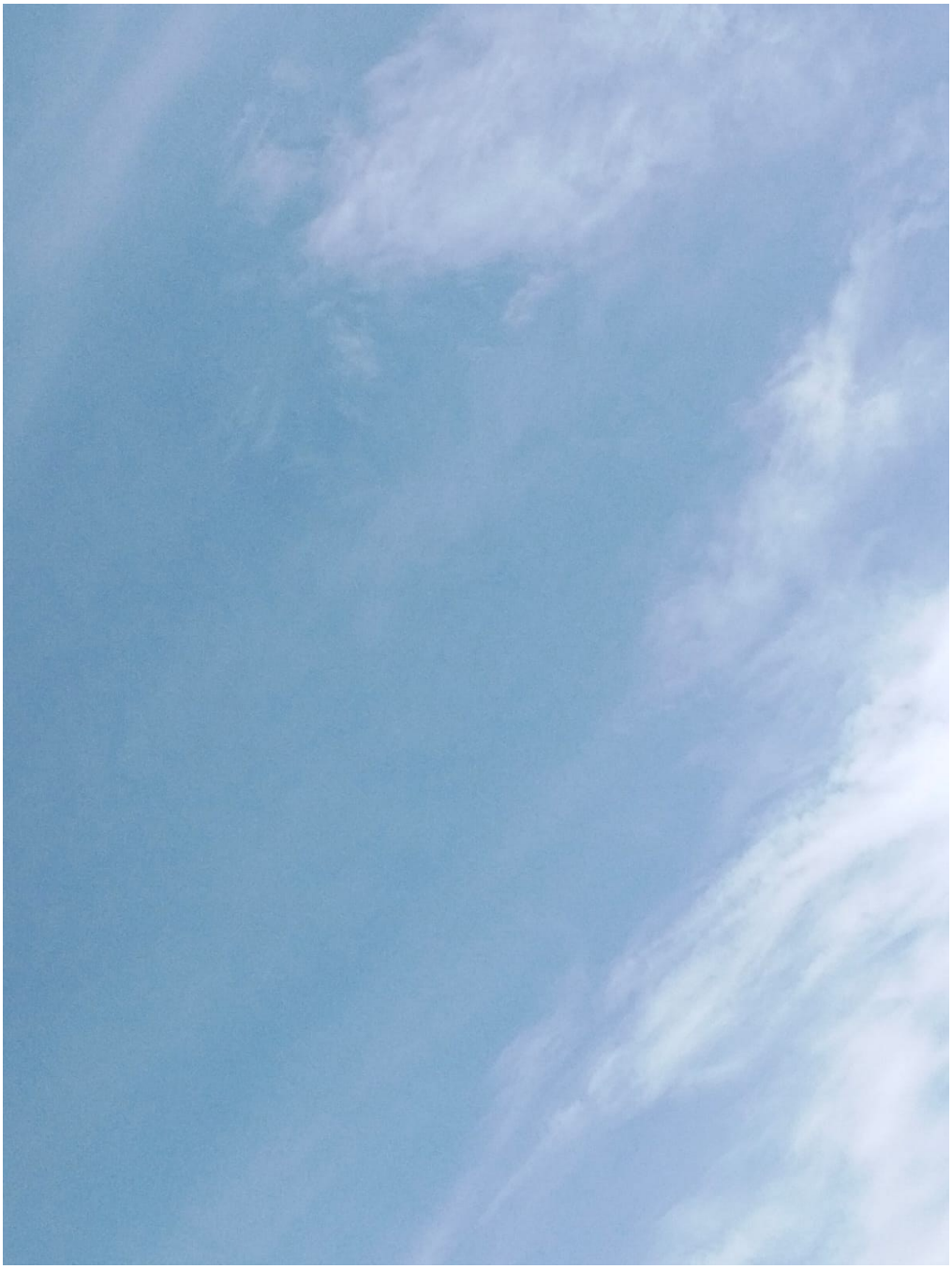


Figure 5.1 An ordinary phone photograph of a daytime sky with cirrus clouds, used to test how the application handles images from a different camera.

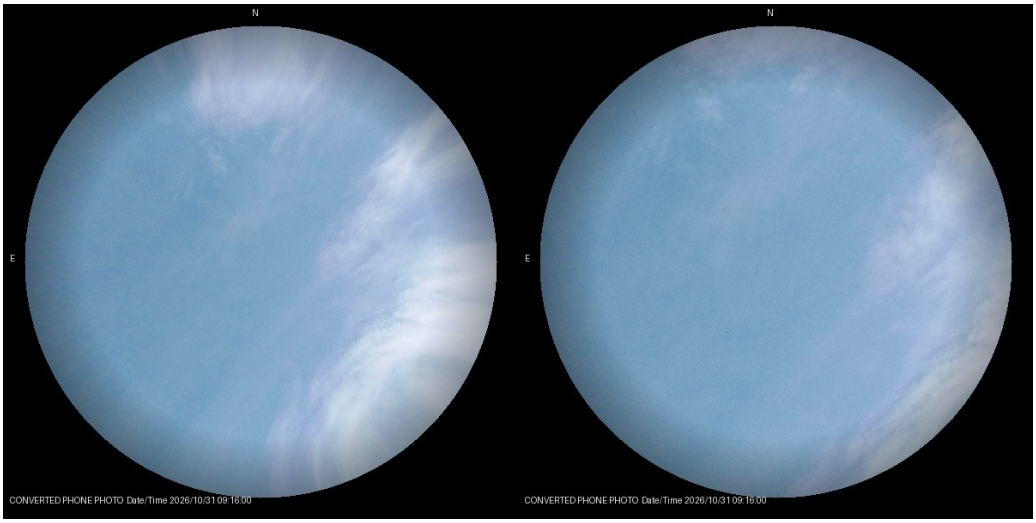


Figure 5.2 The phone photograph converted to the all-sky format. Left: with a fisheye-style warp that enlarges the centre and compresses the edges. Right: a plain circular crop.

Such an image differs from the training data in three ways at once: it is a daytime photograph while the models only saw night frames; the phone sees about 70° of sky while the fisheye sees 180°; and the warp cannot add the sky that the phone never recorded, which leaves streaks at the edge. The application may still produce a plausible verdict, but it is likely to show the *unlike the training data* warning. That is the intended behaviour: the model is specialised to one camera at night, and the warning tells the user when an image falls outside that domain.



Figure 5.3 The test pictures supplied with the application: four real frames from the dataset (left) and five synthetic all-sky frames drawn for testing — moonlit clear, partly cloudy, wet dome, overcast and dark starry sky.

A set of test pictures was also prepared for anyone trying the application: four real frames from the dataset, which test accuracy, and five synthetic all-sky frames drawn programmatically, which test that every feature works, plus a thirty-frame synthetic "night" that gradually clouds over for testing the timeline and the live monitor. The synthetic frames look simpler than real

photographs, so their verdicts are not a measure of accuracy; they are a functional test.

5.10 Limitations

An honest account of the tool must state what it cannot do.

- **It is a prototype.** It has not been operated at the observatory, and its verdicts have not been checked against an operator's decisions over many nights. It must not control equipment unattended.
- **It knows one camera at night.** Its training data come from a single camera over 32 nights in November and December 2021. Other cameras, other seasons and daytime images are outside its experience.
- **Its labels were coarse.** The models learned from labels assigned in one-to-two-hour blocks. They inherit some of that coarseness, and the measured accuracy is itself limited by it.
- **The intermediate conditions are derived.** "Mostly clear", "mostly cloudy" and "wet dome" come from the evidence map and the clusters, not from separately labelled training data.
- **Moonlight remains the hardest condition.** The ensemble is robust compared with the alternatives, but its accuracy still drops by several points under a bright Moon.

5.11 Future work

The analysis points clearly to what would improve the system most.

1. **Better labels.** Relabel frames individually, or at least around weather transitions. This is the largest single limitation.
2. **Richer targets.** Replace the binary label with three classes (clear, cloudy, rain) or, better, with a continuous cloud fraction from 0 to 100%. An observatory needs to know how much of the sky is usable, not only whether it is perfectly clear.

3. **More nights.** Every additional night adds new combinations of Moon phase and weather. Moonlit clear and moonlit cloudy nights are the most valuable.
4. **Temporal models.** Use sequences of frames, not single frames, so that the model can see clouds moving.
5. **Operational validation.** Run the tool alongside human operators for a season and compare decisions, especially the costly "cloudy called clear" errors.
6. **Comparison with published methods.** Cloud detection from all-sky cameras has been studied before; comparing with established approaches on the same data would place the results in context.

5.12 Closing

This book began with a modest assignment: train a network to tell clear skies from cloudy ones. Along the way the project showed that the obvious version of the task contains a shortcut that would have produced impressive and meaningless results; that a few physical measurements already reach 77% to 85% accuracy and that a small CNN learns more within minutes; that with two dozen nights a frozen pre-trained network generalises better than a fine-tuned one; that an ensemble of three different networks is the most accurate and the most reliable; that the Moon is the main difficulty; that many apparent errors are errors in the labels; that the data, without any labels, organise themselves into clear and cloudy skies and reveal rain on the dome; and that the networks base their decisions on genuine sky regions.

The final step turned these results into a tool that an observatory can install with a double-click and use with images from a file, a Google Drive folder or a live camera. It is not the end of the problem. But it is a working, well-evaluated prototype, honest about its limits, and a solid foundation for the next step: a sky monitor trained on better labels and more nights, running every minute of every night.

The next chapter is a practical guide to using it.

CHAPTER 6

User Guide: Sky Monitor AI Step by Step

IN THIS CHAPTER

This chapter is a practical manual. It shows how to install Sky Monitor AI, how to move around it, and how to use each of its eight pages, with screenshots of the real application running the trained models described in this book. Each section follows the same pattern: what the page is for, what you give it, what you get back, and step-by-step instructions. A troubleshooting table and a quick-reference card close the chapter.

6.1 Before you start

6.1.1 What you need

- A computer running **Windows 10/11, macOS or Linux**. No graphics card is needed.
- **Python 3.10 to 3.13**, installed once. On Windows, tick *Add python.exe to PATH* during installation.
- The file `SkyMonitorAI_standalone.zip` (about 150 MB), which contains the application, the trained models and the data.
- An internet connection for the **first start only**, to install the required libraries. Google Drive input also needs internet.

6.1.2 Installing with the launcher (simplest)

1. Unzip `SkyMonitorAI_standalone.zip`. A folder named `SkyMonitorAI` appears.
2. Open the folder and start the launcher for your system:
   - **Windows:** double-click `run_windows.bat`. If Windows shows *Windows protected your PC*, click **More info → Run anyway**.
   - **macOS:** double-click `run_mac.command`. If it is blocked, right-click it and choose **Open → Open**.
   - **Linux:** open a terminal in the folder and type `bash run_linux.sh`.
3. The first start installs the libraries into a private folder `.venv` inside `SkyMonitorAI`. This takes 3–10 minutes; let it run.
4. Your browser opens at **http://localhost:8501** with the Sky Monitor AI home page.

Keep the black terminal window open while you use the application. Closing it stops the application. Next time, simply start the launcher again; it opens within seconds.

6.1.3 Installing from the terminal with conda (Ubuntu example)

If you use Anaconda or Miniforge, a conda environment works equally well. These are the commands used to install the application on an Ubuntu workstation for this book:

```
cd ~/Downloads
unzip -q SkyMonitorAI_standalone.zip
cd SkyMonitorAI
conda create -n skymonitor python=3.11 -y
conda activate skymonitor
pip install -r requirements.txt
streamlit run streamlit_app.py
```

If Streamlit asks for an e-mail address on its first run, just press Enter. To start the application on later days, three commands suffice:

```
conda activate skymonitor
cd ~/Downloads/SkyMonitorAI
streamlit run streamlit_app.py
```

To stop the application, press **Ctrl + C** in the terminal.

> **NOTE**
> Only type the lines inside the grey command boxes. Output printed by the terminal or by other programs is not a command; pasting it into the terminal produces "command not found" errors but does no harm.

6.2 Finding your way around

Every page shares the same layout: a **navigation menu** on the left and the page content on the right. Click a page name in the menu to open it. The current page is highlighted.



Figure 6.1 The home page. The navigation menu on the left lists the pages (this screenshot was taken before the Weather & satellite page was added; see Section 6.6). The four numbers summarise the project's main results: 86.7% test accuracy on unseen nights, 87.8% with smoothing, 77.1% for the hand-crafted baseline and 87% purity for the unsupervised clusters.

The **Home** page introduces the project and shows its key results. Scrolling down reveals six cards, one per page, each with a short description and an **Open** link, followed by a summary of how the system works and its key findings.

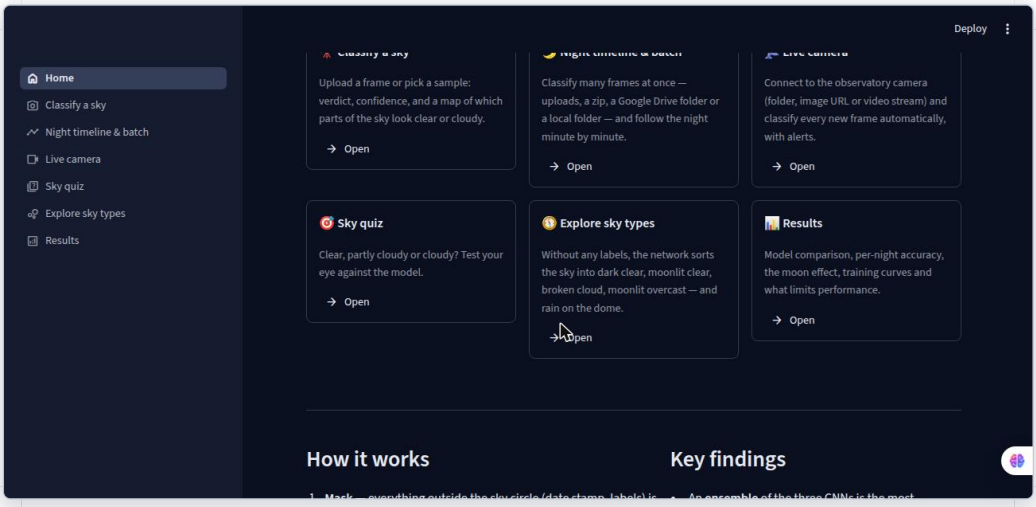


Figure 6.2 Further down the home page: one card per page, each with an Open link.

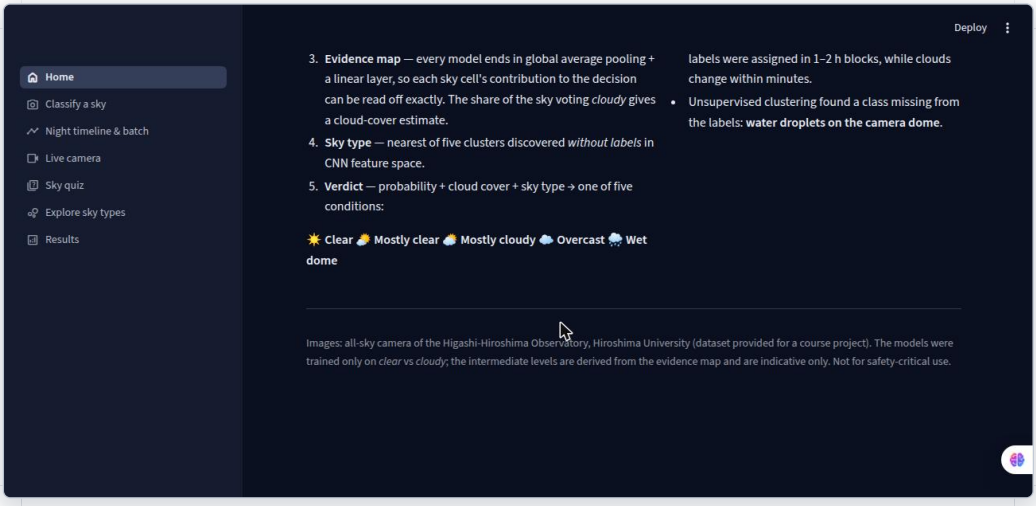


Figure 6.3 The bottom of the home page explains the five-step pipeline (mask, three CNNs, evidence map, sky type, verdict) and lists the five sky conditions the application can report.

Table 6.1 The menu at a glance.

Menu entry	Use it to…
Home	Read the overview and jump to a page
Classify a sky	Analyse one image in detail
Night timeline & batch	Analyse many images: a night, a zip, a Drive folder or a local folder
Live camera	Monitor a camera continuously

Menu entry	Use it to…
Weather & satellite	See the forecast and satellite picture for any place and time
Sky quiz	Test your own judgement against the model
Explore sky types	Explore the unsupervised map of 4,000 images
Results	See all evaluation results interactively

6.3 Classify a sky

Purpose: analyse a single all-sky image in detail. **Input:** a sample frame, an uploaded image, a Google Drive or web link, or a camera snapshot. **Output:** a sky condition with a recommended action, P(clear), estimated cloud cover, sky type, an evidence map and each model's opinion.

6.3.1 Trying a sample frame

1. Click **Classify a sky** in the menu.
2. Keep **Sample frames** selected. The page immediately analyses the first sample.

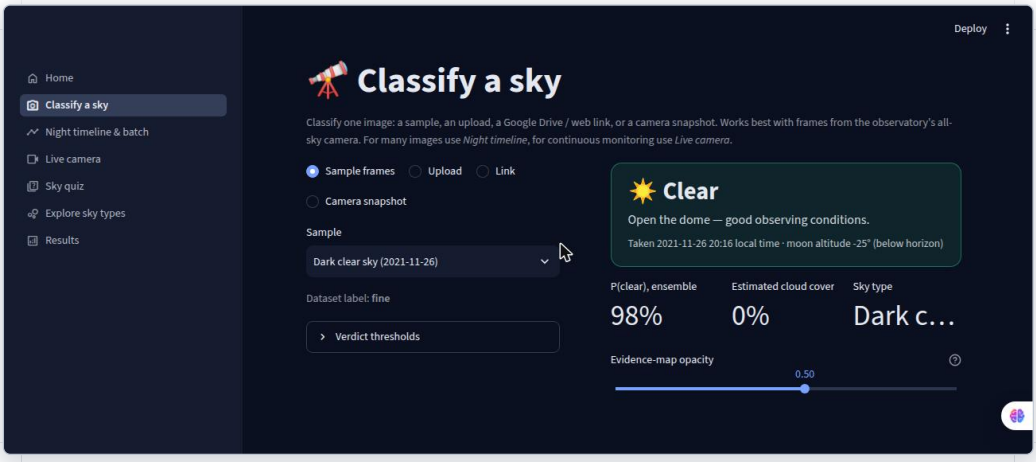


Figure 6.4 The Classify page with the first sample, a dark clear sky from 26 November 2021. The verdict card shows ☀ Clear with the advice to open the dome, the time the frame was taken and the Moon's altitude (−25°, below the horizon). Below: P(clear) 98%, cloud cover 0%, sky type Dark clear sky.

1. To choose another sample, click the **Sample** box. A list of samples opens, two for each sky type plus two hard cases. You can scroll the list or type a few letters to filter it.

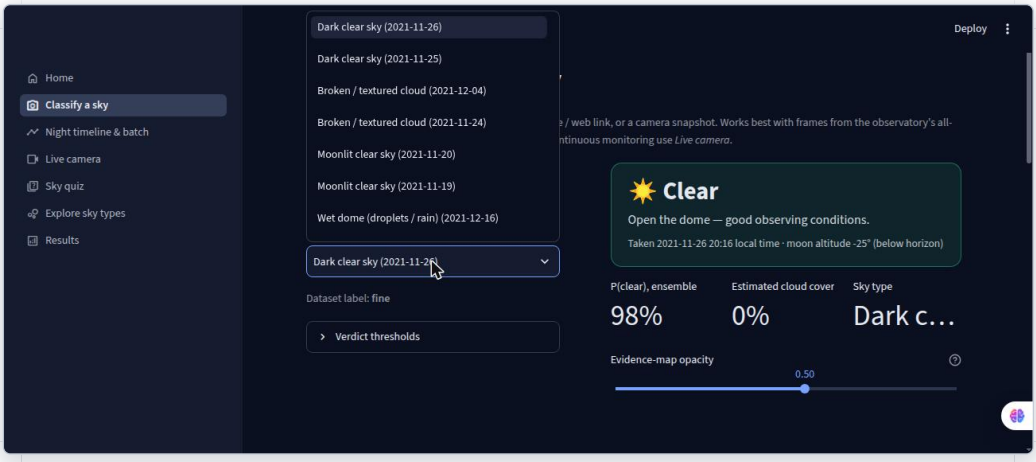


Figure 6.5 The sample list. Typing in the box filters the entries, for example "full moon" or "wet".

1. Select **Hard case: full moon, partly cloudy (labelled cloudy)**.

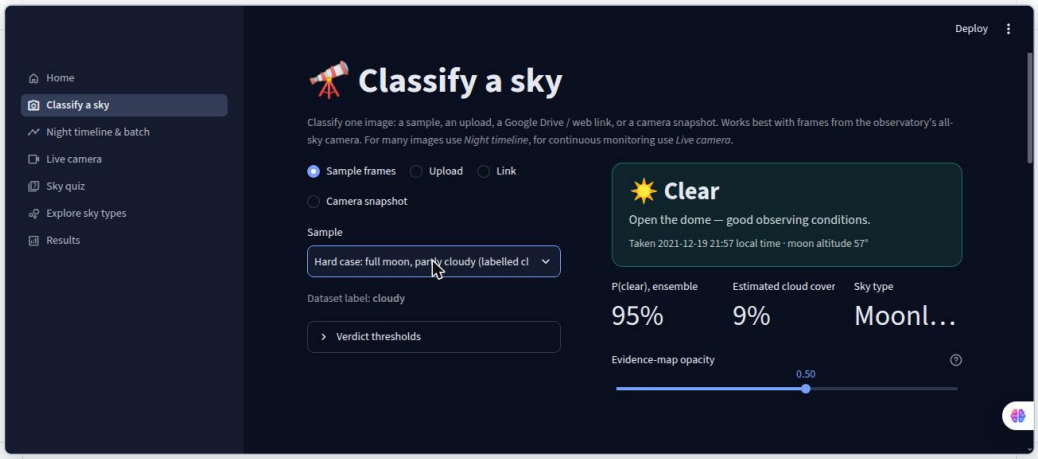


Figure 6.6 The full-moon sample from 21:57 on 19 December 2021, labelled cloudy in the dataset. The ensemble judges it ☀️ Clear with P(clear) 95% and only 9% estimated cloud cover; the Moon stands at 57°. Chapters 3 and 4 showed that this frame is mostly clear with cloud only near the horizon.

6.3.2 Reading the evidence map

Scroll down to see the **evidence map**: the sky image with a colour layer on top.

- **Red** areas look clear: on their own, they would make the models say *clear*.
- **Blue** areas look cloudy.
- White areas are undecided.

Move the mouse over the map to see the local probability at any point. Use the **Evidence-map opacity** slider above it to fade the colour layer in and out and compare it with the photograph underneath.

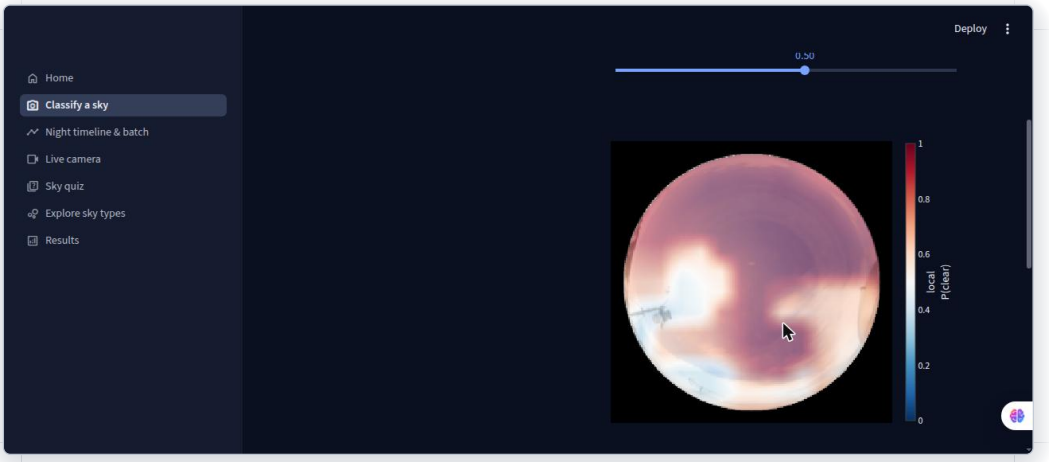


Figure 6.7 The evidence map of the full-moon frame. The open sky is red; the cloud bank near the lower-left horizon and around the mast is blue. The colour bar shows the local probability of a clear sky.

6.3.3 What each model says

Further down, a bar chart shows the probability of a clear sky from each of the three networks and from their average, the ensemble. The dashed line marks 0.5. If the three models disagree strongly, the page adds a warning: strong disagreement is typical of ambiguous, partly cloudy or moonlit skies. The panel on the right explains how to read every number on the page.

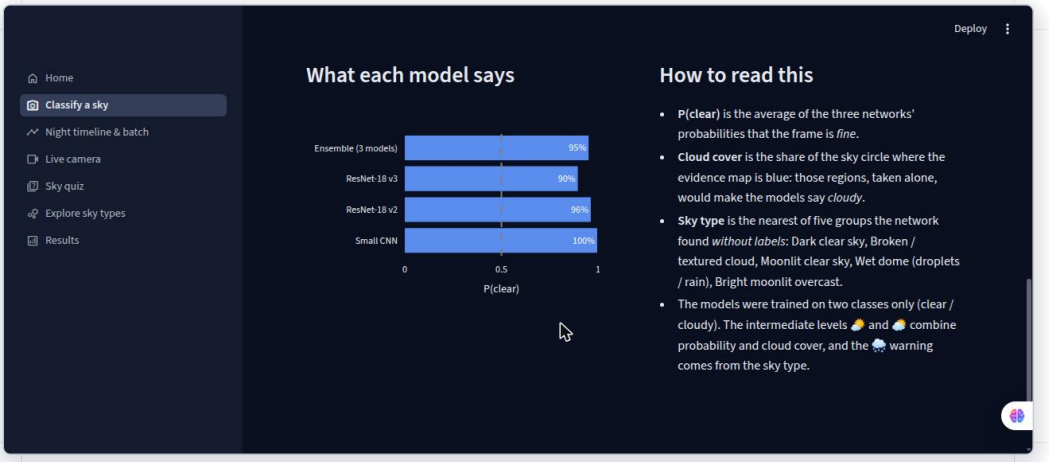


Figure 6.8 Each model's opinion on the full-moon frame: small CNN 100%, ResNet-18 v2 96%, ResNet-18 v3 90%, ensemble 95%.

6.3.4 The wet-dome warning

Choose the sample **Wet dome (droplets / rain)** to see the most important warning the application can give. Its sky type is the wet-dome cluster discovered in Chapter 4, so the verdict is 🌧️ **Wet dome**, with the advice to close the dome.

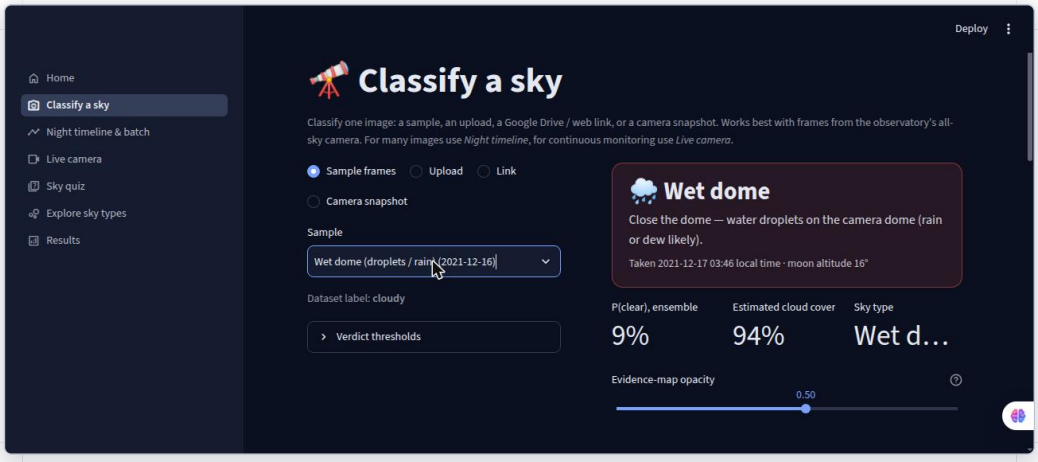


Figure 6.9 The wet-dome sample from 03:46 on 17 December 2021: 🌧️ Wet dome — close the dome; P(clear) 9%, cloud cover 94%.

6.3.5 Adjusting the thresholds

Click **Verdict thresholds** to open four sliders that control when a frame counts as *Clear* or *Overcast*. Raising "Clear if P(clear) ≥" makes the system more cautious about declaring a clear sky. Everything between the two thresholds is reported as *Mostly clear* or *Mostly cloudy*.

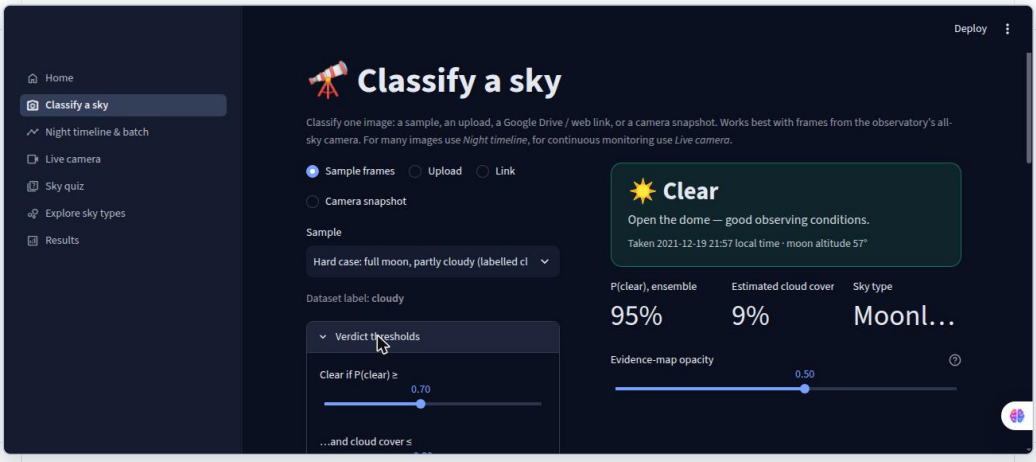


Figure 6.10 The Verdict thresholds panel, opened. The defaults are Clear if P(clear) ≥ 0.70 and cloud cover ≤ 0.30; Overcast if P(clear) ≤ 0.30 and cloud cover ≥ 0.70.

6.3.6 Your own images

Upload a file:

1. Select **Upload** at the top of the page.
2. Click **Upload** (or drag a file onto the box) and choose a JPG or PNG image.
3. The analysis appears on the right within a few seconds.

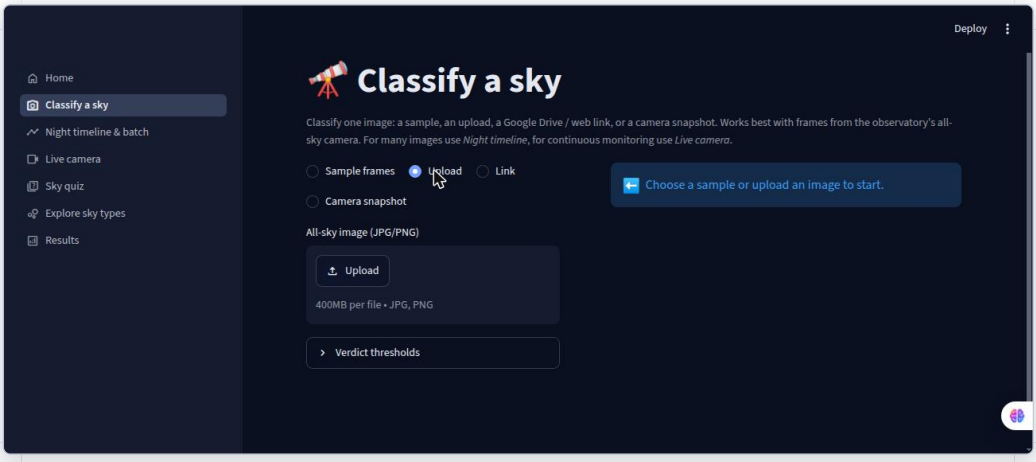


Figure 6.11 The Upload option. Click the Upload button or drag an image onto the box.

Use a link:

1. Select **Link**.

2. Paste a Google Drive file link (the file must be shared as *Anyone with the link*) or the address of an image on the web, then press Enter.

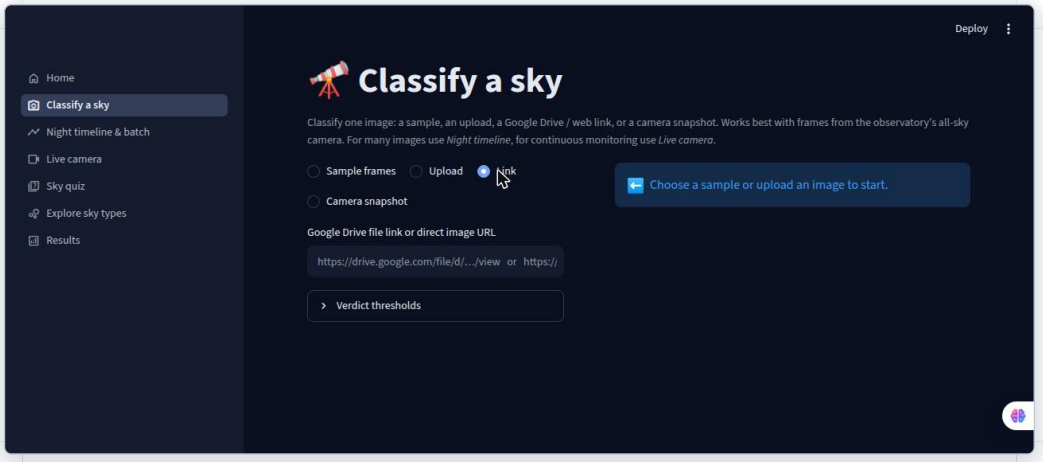


Figure 6.12 The Link option accepts a Google Drive file link or a direct image URL.

Take a camera snapshot: select **Camera snapshot** and allow the browser to use the camera. This uses your computer's webcam or phone camera and is meant for quick tests; for an observatory camera, use the *Live camera* page.

> **TIP**
> Name image files with their timestamp, such as `211124194628.jpg` for 24 November 2021 at 19:46:28. The application then shows the date, time and Moon altitude for the frame.

> **NOTE**
> The models were trained on night-time frames from one all-sky camera. Images from other cameras or taken in daylight may receive a warning that they look unlike the training data. The result is then unreliable.

6.4 Night timeline & batch analysis

Purpose: analyse many frames at once and follow the sky through a night. **Input:** one of the six test nights, uploaded files or a zip, a shared Google Drive

folder or file, or a folder on your computer. **Output:** counts per sky condition, a timeline with smoothing and Moon altitude, a frame viewer and a downloadable CSV file.

6.4.1 Exploring a test night

1. Click **Night timeline & batch** in the menu.
2. Keep **Test nights** selected and choose a night from the **Night** box. The mixed night **2021-11-24 (298 cloudy / 240 fine)** is the most instructive.

The page then shows, from top to bottom:

- **Condition counts:** how many frames of the night fall into each of the five conditions.
- **Controls:** the model (the ensemble by default), the smoothing window, centred or trailing smoothing, and the decision threshold.
- **Agreement with the labels**, before and after smoothing, and the share of frames with the Moon up.
- **The timeline:** blue dots are each frame's P(clear), the blue line is the smoothed value, the dotted white line is the dataset label (1 = fine), and the orange line is the Moon's altitude on the right-hand axis. The background colour shows the condition after smoothing.
- **The frame viewer:** drag the **Time** slider to any minute of the night to see that frame and its values.

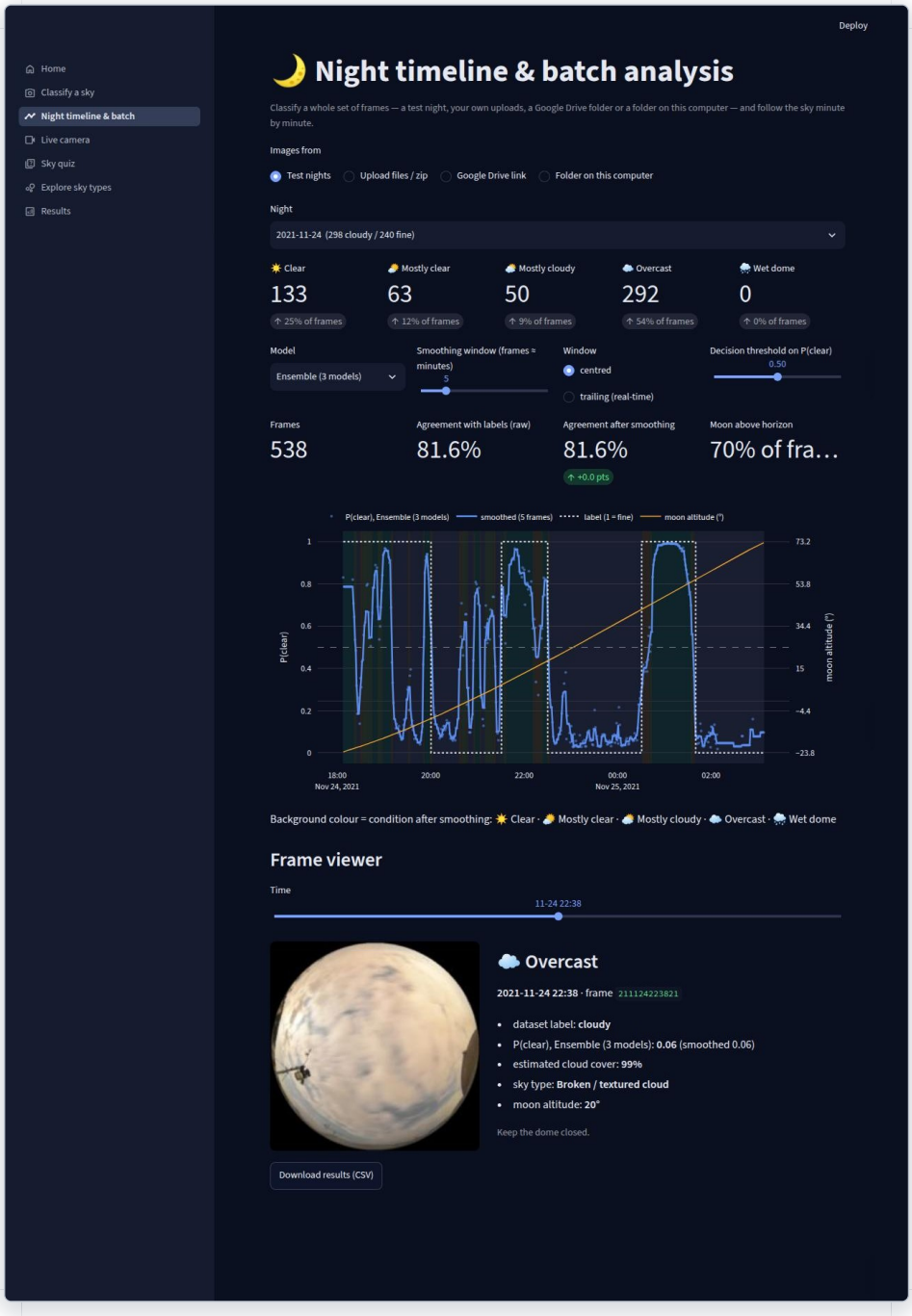


Figure 6.13 The complete Night timeline page for 24 November 2021. Of 538 frames, 133 are judged clear, 63 mostly clear, 50 mostly cloudy and 292 overcast; agreement with the labels is 81.6%. The timeline shows the rectangular blocks of the labels and the model following changes within minutes. The frame viewer at 22:38 shows an overcast frame with P(clear) 0.06 and 99% cloud cover.

> **TIP**
> Move the **Smoothing window** slider and watch the agreement change. Switch the window to **trailing (real-time)** to see how a live system, which cannot use future frames, would behave.

6.4.2 Analysing a Google Drive folder

1. Make sure the folder is shared: in Google Drive, right-click the folder → **Share** → under *General access* choose **Anyone with the link** (Viewer) → **Copy link**.
2. In the application, select **Google Drive link** at the top of the page.
3. Paste the link and press Enter, then click **List images**. Large folders take some seconds to list.
4. Choose the **Folder** (or (*all*)), **How many frames** (up to 600) and **Which**: the *latest* or *first* by name, or *evenly spread* through the folder.
5. Click **Download & analyse**. A progress bar shows the downloads (roughly three seconds per frame), then the analysis.

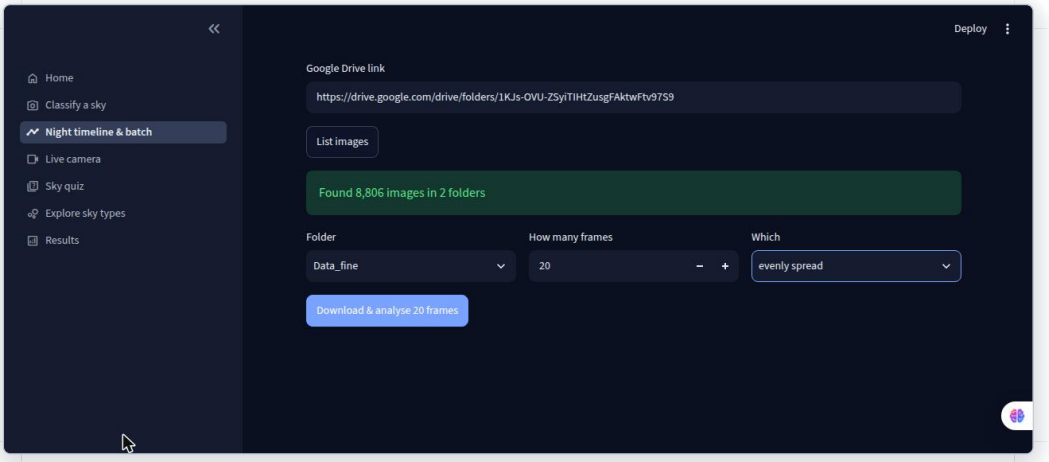


Figure 6.14 The project's shared folder listed in the application: 8,806 images in 2 folders. Here 20 frames, evenly spread, are selected from Data_fine.

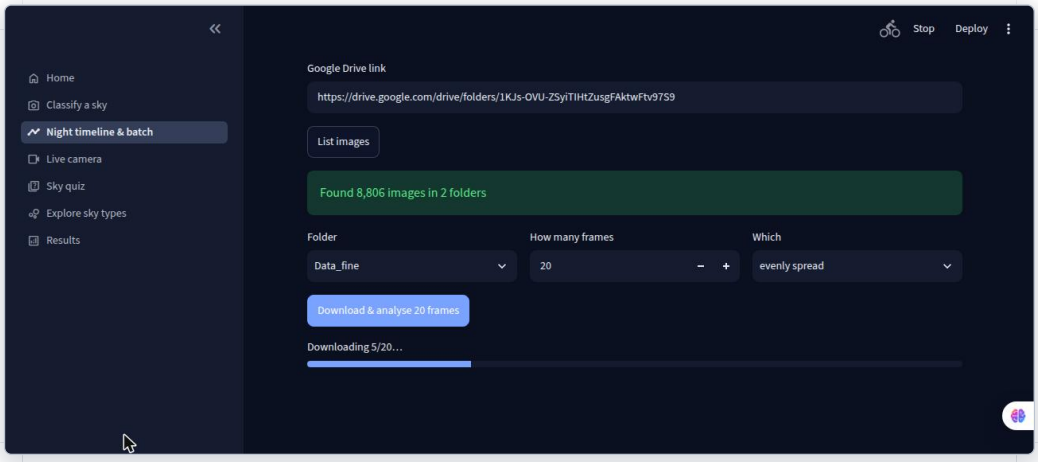


Figure 6.15 Downloading the selected frames from Google Drive: 5 of 20 done.

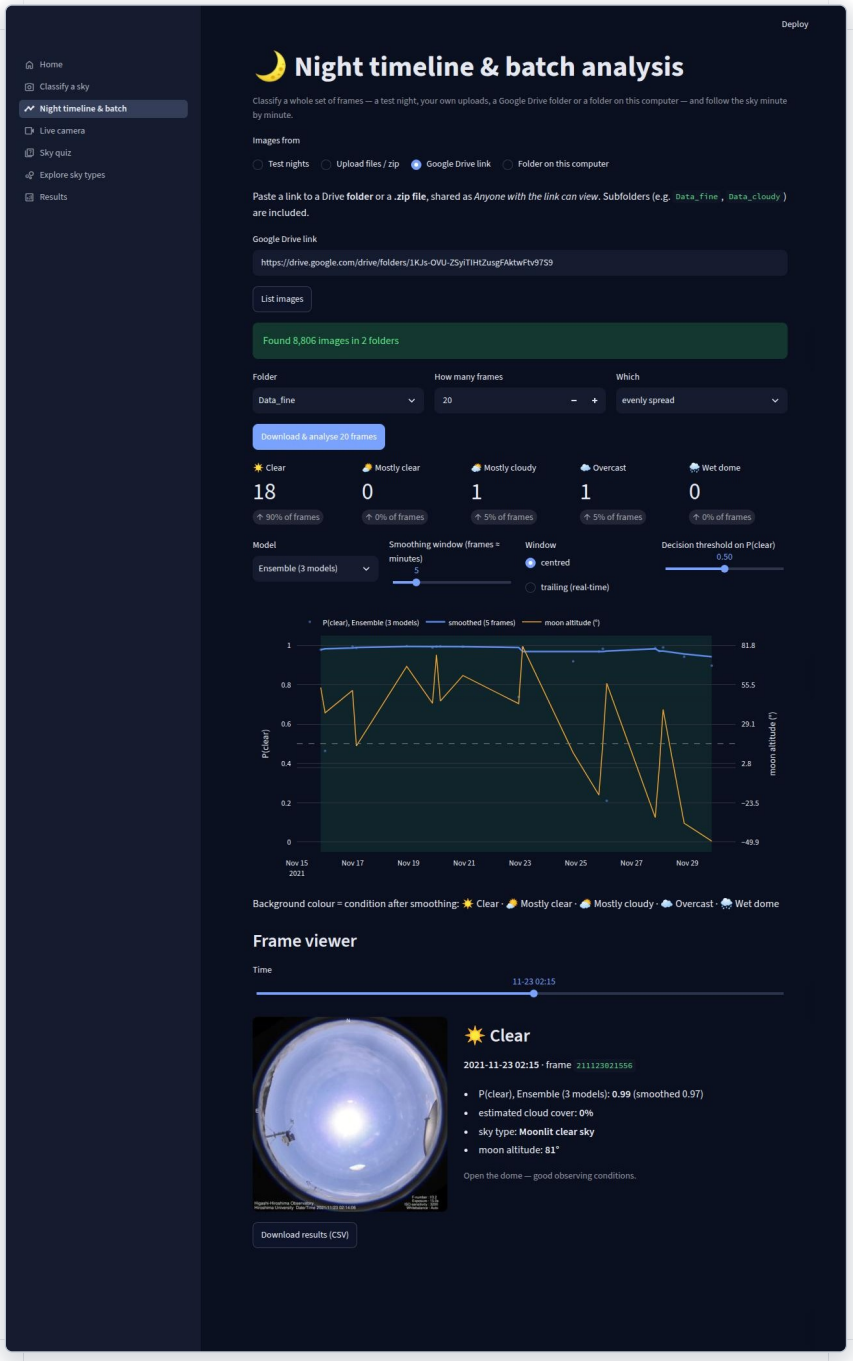


Figure 6.16 The result for the 20 Drive frames from Data_fine: 18 clear, 1 mostly cloudy, 1 overcast. Because the frames are spread over two weeks, the timeline spans 15 to 29 November. The frame viewer shows a moonlit clear frame from 23 November with P(clear) 0.99, 0% cloud cover and the Moon at 81°.

You can also paste the link of a single shared file or of a shared **zip file** of frames; the button then reads **Download & analyse** directly.

6.4.3 Analysing a folder on your computer

This option works when the application runs on your own computer, for example on the computer that receives the camera's frames.

1. Select **Folder on this computer**.
2. Type the full path of the folder, such as `/home/user/allsky/images` or `C:\AllSky\images`, and press Enter.
3. The page reports how many images it found. Choose how many of the latest to analyse and click **Analyse**.

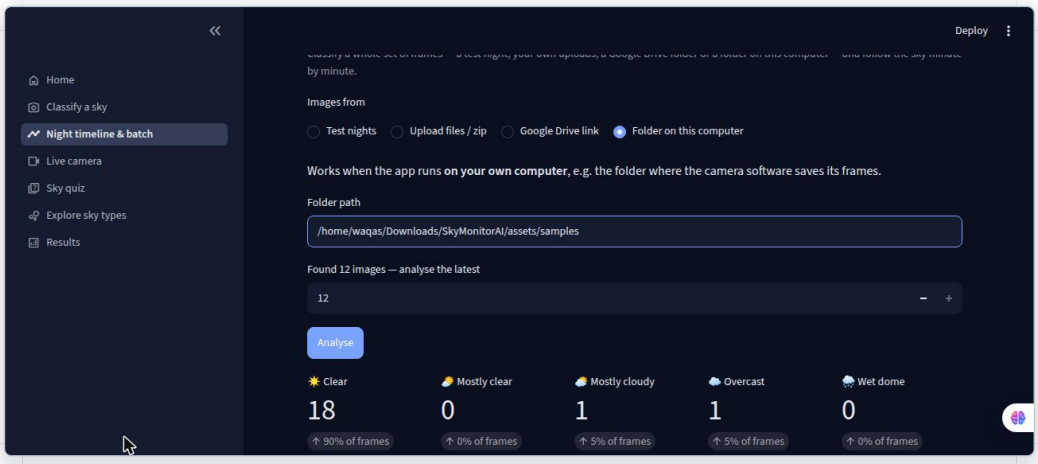


Figure 6.17 The local-folder option with the application's own sample folder: 12 images found. (The condition counts below belong to the previous batch until Analyse is pressed.)

6.4.4 Uploading many files or a zip

Select **Upload files / zip**, then choose many JPG or PNG files at once, or a single zip file containing frames, and click **Analyse**. A zip is the fastest way to upload a whole night.

6.4.5 Saving the results

At the bottom of every batch result, **Download results (CSV)** saves one row per frame, which can be opened in Excel or LibreOffice.

Table 6.2 Columns of the results file.

Column	Meaning
stem	File name without extension
time	Timestamp from the file name (local time)
p_ensemble	Ensemble probability of a clear sky
p_smallcnn, p_resnet_v2, p_resnet_v3	Each network's probability
cloud_cover	Estimated fraction of the sky that looks cloudy
sky_type, sky_type_name	Nearest unsupervised sky type (number and name)
moon_alt	Moon altitude in degrees (when the file name has a timestamp)
category	The sky condition

6.5 Live camera

Purpose: monitor a camera continuously and classify every new frame automatically. **Input:** a folder the camera writes to, a URL that serves the latest image, or a USB or RTSP/MJPEG camera. **Output:** the current condition and evidence map, a chart since monitoring began, pop-up alerts on changes, and a downloadable log.

6.5.1 Choosing the right option

Click **Which option do I need?** at the top of the page for a short guide.

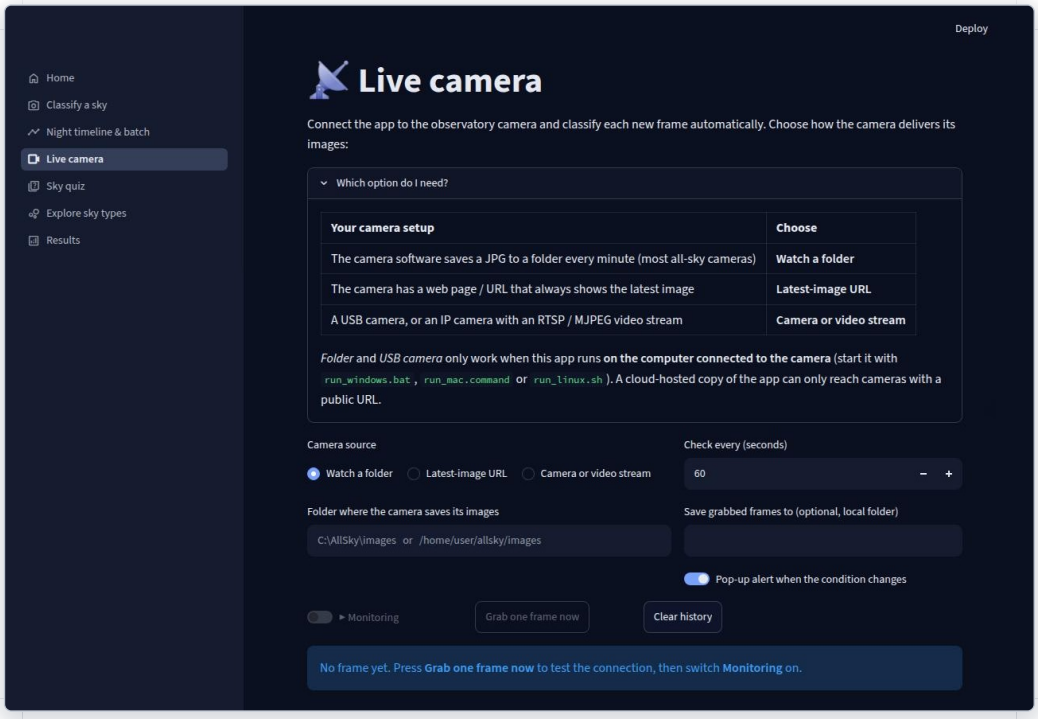


Figure 6.18 The Live camera page with the help panel opened. Most all-sky cameras save a JPEG to a folder every minute; choose Watch a folder for them.

Table 6.3 Which live option to choose.

Your camera setup	Choose	Enter
The camera software saves a JPG to a folder	Watch a folder	The folder path
The camera has a web address showing its latest image	Latest-image URL	The image address, e.g. `http://192.168.1.50/latest.jpg`
A USB webcam or an IP camera with a video stream	Camera or video stream	`0` for the first USB camera, or `rtsp://user:password@address:554/stream`

6.5.2 Starting the monitor

1. Choose the **Camera source** and enter the folder, URL or camera.
2. Set **Check every (seconds)**: 60 suits a camera that takes one frame per minute.
3. Optionally enter a folder in **Save grabbed frames to**, and keep **Pop-up alert when the condition changes** switched on.

4. Click **Grab one frame now** to test the connection. The newest frame is classified and shown.
5. Switch ▶ **Monitoring** on. From now on the page checks the camera at the chosen interval and analyses every new frame.

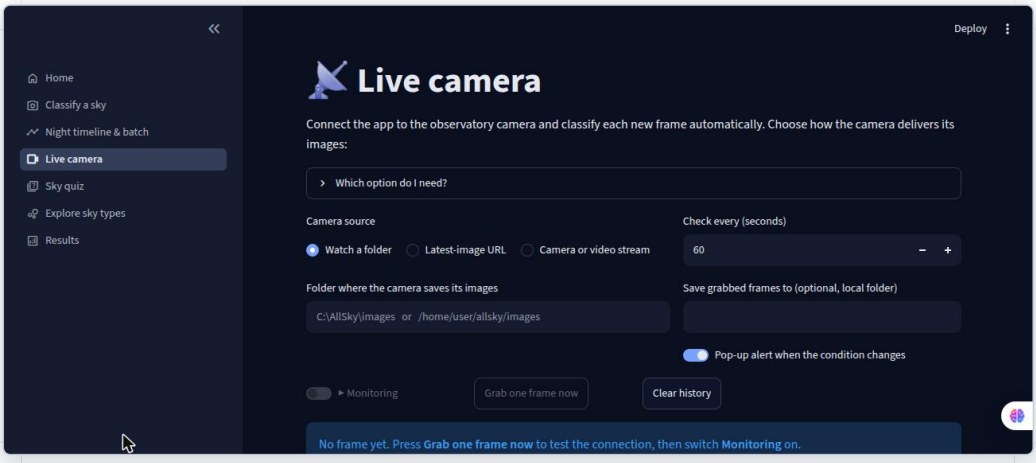


Figure 6.19 The Live camera page before the first frame: settings on top, then the Monitoring switch, Grab one frame now and Clear history.

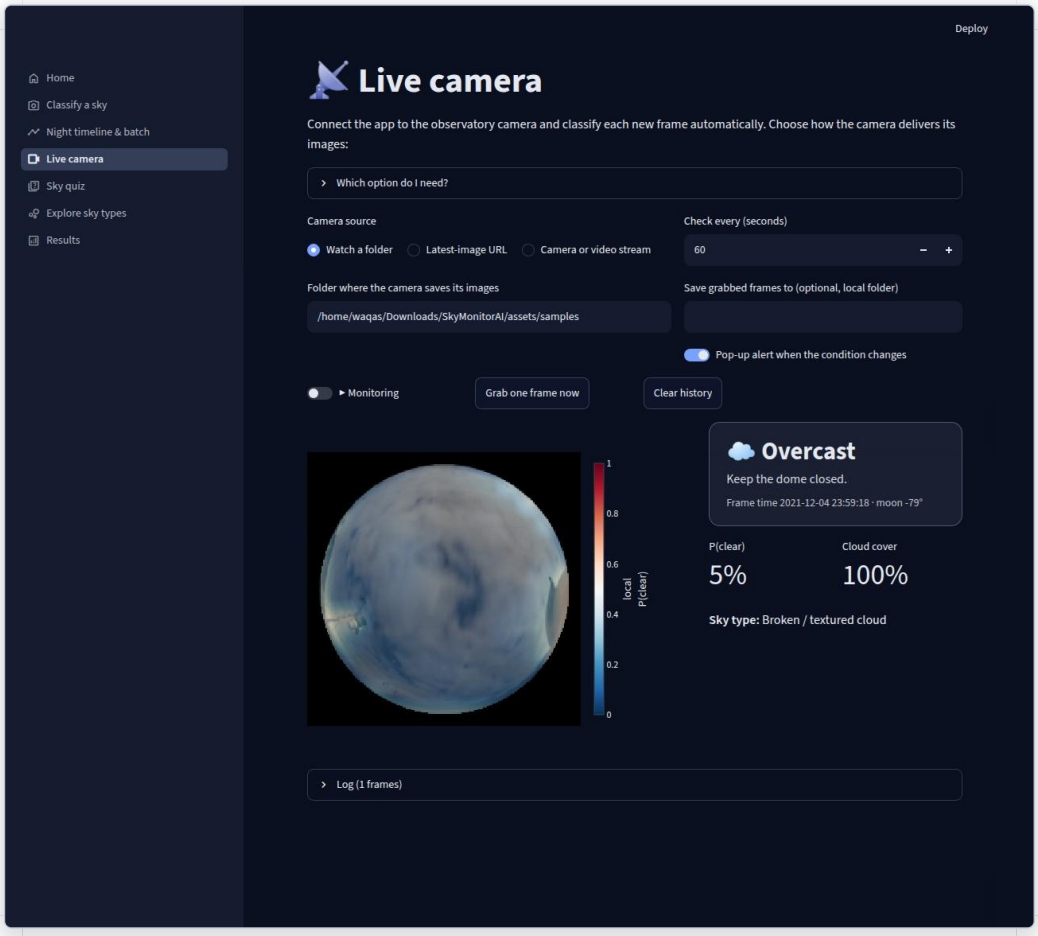


Figure 6.20 After Grab one frame now with the sample folder as a stand-in camera: the newest frame, from 23:59 on 4 December 2021, is judged ☁️ Overcast — keep the dome closed — with P(clear) 5%, 100% cloud cover and sky type Broken / textured cloud. The evidence map is blue over the whole sky. The Log expander lists every frame analysed so far.

Once two or more frames have been analysed, a chart of P(clear) and cloud cover since monitoring began appears below the image, with the background coloured by condition. Open **Log** to see every frame in a table and to download it as a CSV file. **Clear history** starts a new log.

6.5.3 Practising without a camera

You can simulate a camera with any folder of sky images:

1. Create an empty folder, for example `~/Downloads/fake_camera`.

2. On the Live camera page choose **Watch a folder**, enter that folder, set **Check every** to 5 seconds and switch **Monitoring** on.
3. Copy frames into the folder one by one, as a camera would. Each new file is analysed within a few seconds; when the condition changes, a 🔔 pop-up appears.

> **NOTE**
> *Watch a folder* and USB cameras only work when the application runs on the computer connected to the camera. A cloud-hosted copy can only reach cameras that have a public web address.

6.5.4 Using a USB webcam on Linux

An ordinary USB webcam is a convenient way to try the live monitor. These are the steps that worked on the Ubuntu laptop used for this book, with a Sonix MWB-15 webcam.

1. **Find the camera.** Plug it in and type `ls /dev/video*` in a terminal. One camera usually appears twice, for example `/dev/video0` and `/dev/video1`; the first is the picture. `cat /sys/class/video4linux/video*/name` shows which camera each entry belongs to.
2. **Use its permanent name.** The number can change when the camera reconnects. The command `ls /dev/v4l/by-id/` lists a name that never changes, such as `usb-Sonix_Technology_Co._Ltd._MWB-15_SN0001-video-index0`. Enter the full path `/dev/v4l/by-id/…-video-index0` as the camera on the Live camera page. A short link makes it easier to type: `ln -sf /dev/v4l/by-id/…-video-index0 ~/skycam`, after which `/home/<user>/skycam` works as the camera.
3. **See where it points.** `ffplay -f v4l2 -input_format mjpeg -video_size 640x480 /dev/v4l/by-id/…-video-index0` opens a live view (press Q to close it before starting the application, since only one program can use the camera at a time). Without `-input_format mjpeg` some cheap webcams look purple in this viewer; the application itself is not affected.

4. **If the camera keeps disconnecting**, `sudo dmesg | tail` shows "USB disconnect" messages. Plug the camera directly into the laptop, try another port and turn off USB power saving with `echo -1 | sudo tee /sys/module/usbcore/parameters/autosuspend`.

> **NOTE**
> A webcam proves that the live connection works, but its verdicts should be read as a demonstration only. The observatory frames were taken through a fisheye lens with 15-second exposures at ISO 3200 (the values printed in each frame's caption); a webcam sees a small patch of sky with exposures about 450 times shorter, and at night it records almost nothing. Expect the "unlike the training data" warning.

6.6 Weather & satellite

Purpose: an independent check on the sky — the weather forecast and the satellite picture for any place and time, side by side, and a comparison with what the camera saw. **Input:** a location (from a list, by name or by coordinates) and a time (now, any date, or the time of a camera frame). **Output:** current conditions and hourly forecast with cloud cover in three layers, an 8-day outlook, a satellite picture with the location marked, and a camera-versus-forecast comparison.

This page needs an internet connection. Its data come from two free services that require no account: **Open-Meteo** for forecasts and past weather, and NASA's **Global Imagery Browse Services (GIBS)** for satellite pictures.

6.6.1 Choosing the location

No location is preselected; the page stays empty until you choose one, in one of three ways.

- **From the list.** A drop-down menu of observatories (🔭: Higashi-Hiroshima, Bosscha, Mitaka, Okayama, Mauna Kea), cities (🏙️) and the places you saved yourself (📌). A saved place can be removed again with **Remove from my list**.

- **Search by name.** Type a place name and choose the correct match. Observatories are often missing from place databases; search the nearest town instead, or use coordinates.
- **Enter coordinates.** Give a name, the latitude (north positive) and the longitude (east positive), then click **Use this location**, or **Save to my list** to add it to the drop-down menu for next time. In Google Maps, right-clicking a spot shows its coordinates.

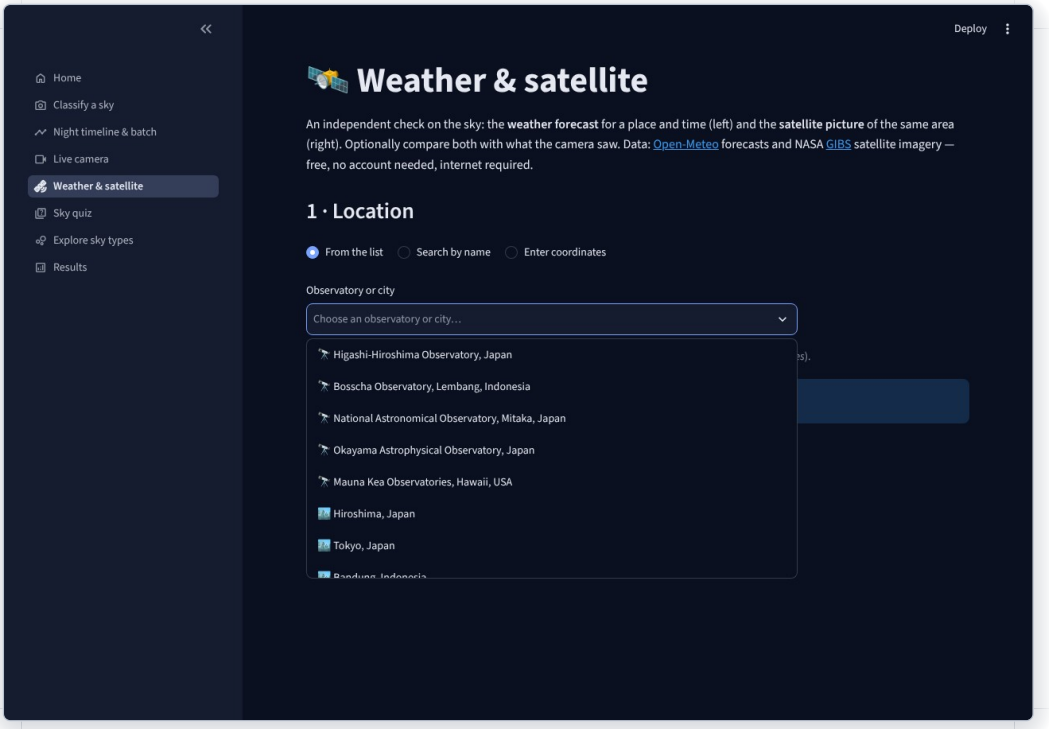


Figure 6.21 The Weather & satellite page before a location is chosen, with the drop-down list of observatories and cities opened. The page shows no data until a location is selected.

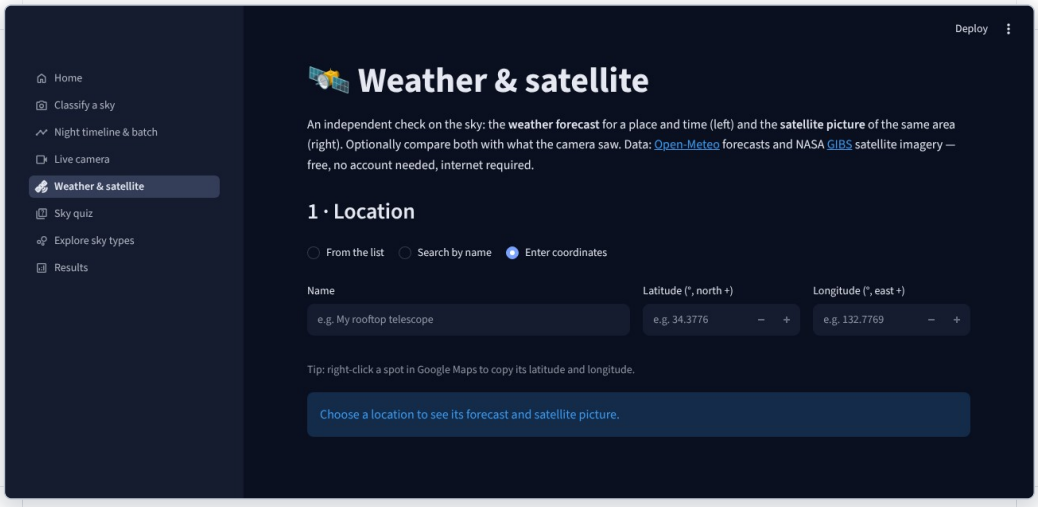


Figure 6.22 The same page with Enter coordinates selected: a name, latitude and longitude define any place, which can then be saved to the list.

6.6.2 Choosing the time

- **Now** — current conditions and the forecast for the coming hours and days.
- **Choose date & time** — any date from 1950 to two weeks ahead, in the local time of the location.
- **Time of a camera frame** — upload a frame; its time is read from a file name such as `211124194628.jpg`. The frame is also classified by the three networks for the comparison below.
- **Latest live-camera frame** — appears when the Live camera page has logged at least one frame.

6.6.3 Reading the forecast and the satellite picture

The **left half** shows a forecast card like those of common weather websites — an icon, the temperature, the conditions, cloud cover, chance of rain, humidity and wind — and below it an hourly chart for twelve hours either side of the chosen time, with tabs for cloud cover, temperature, precipitation and wind. The dotted vertical line marks the chosen time. The cloud-cover chart separates **low, mid and high cloud**, which matters for astronomy: thin high cirrus may still allow some observing, while a low layer does not. For times within the last three months and the coming two weeks the values are forecasts on a grid of about 10 km; for older dates, such as the 2021 nights of this book, the page

shows the **ERA5 reanalysis**, a reconstruction of the weather that actually occurred, on a grid of about 30 km.

The **right half** shows the satellite picture of the area, with a red circle on the location. Over East and Southeast Asia and Australia it comes from Japan's geostationary **Himawari-9** satellite, which photographs the region every 10 minutes; over the Americas from the **GOES** satellites. **Infrared** pictures work day and night and show cold, high clouds as bright; **visible** pictures look like a photograph but are dark at night. Live pictures appear with a delay, so the page shows the newest one available. Where no geostationary images are available — over Europe, Africa and western and southern Asia, or for dates before the archive begins — it falls back to the daily daytime picture of the polar-orbiting VIIRS satellite. **Open the interactive map** opens the same view in NASA Worldview, where it can be zoomed and animated.

6.6.4 Camera versus forecast

When the time comes from a camera frame, a third section compares the camera's verdict and estimated cloud cover with the forecast's cloud cover at that hour and states whether they agree. They need not match exactly: the forecast describes a grid square of 10–30 km and all cloud layers, while the camera sees the sky above one spot, and a forecast is a prediction. A disagreement is therefore not an error of either; it is a reason to look at the satellite picture and the camera image. Make sure the location chosen is where the camera stands, and that the time of the frame is the local time of that location.

> **TIP**
> For the observatory frames of this book, choose *Higashi-Hiroshima Observatory* and *Time of a camera frame*, and upload a frame. The page then shows the reconstructed weather of that night next to the network's verdict — a quick way to examine the doubtful labels discussed in Chapter 3.

6.7 Sky quiz

Purpose: practise judging the sky and compare your judgement with the model's. **Input:** your answer. **Output:** the dataset label, the model's verdict and a running score.

1. Click **Sky quiz**. A random test frame appears with its date, time and whether the Moon was up.
2. Click ☀️ **Clear**, ⛅ **Partly cloudy** or ☁️ **Cloudy**.
3. The page reveals what you said, the dataset label, the model's condition with its probability and cloud cover, and the sky type.
4. Click **Next sky ➜** for another frame.



Figure 6.23 A quiz question: a bright, moonlit frame from 23:46 on 15 November 2021. Choose one of the three answers.

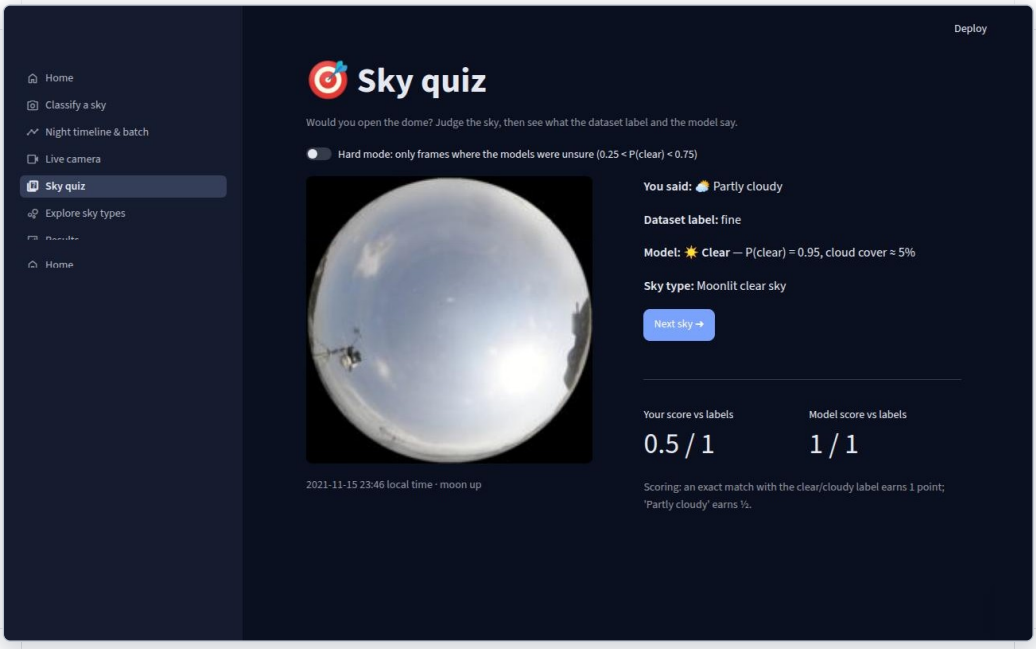


Figure 6.24 After answering ⛅ Partly cloudy: the dataset label is fine, and the model says ☀️ Clear with P(clear) 0.95 and 5% cloud cover, sky type Moonlit clear sky. An exact match with the label earns one point; "Partly cloudy" earns half a point.

Switch on **Hard mode** to see only frames on which the models were unsure (P(clear) between 0.25 and 0.75). When the model disagrees with the label, the page points out that many such frames are partly cloudy or sit inside a time block that was labelled as a whole.

6.8 Explore sky types

Purpose: explore how the network organises the sky without any labels. **Input:** a colouring choice and selections on the map. **Output:** an interactive map of 4,000 frames, images of selected points, the five sky types and the clustering scores.

1. Click **Explore sky types**. Each dot on the map is one image; images the network finds similar sit close together.
2. Choose **Colour by**: *sky type* (the five clusters), *true label*, *GMM cluster*, *brightness* or *night*. Compare *true label* with *sky type* to see how well the unsupervised groups match the labels.

3. **Select dots** by clicking, or drag a box or lasso around a group (choose the tool in the small toolbar above the map). The selected images appear below the map with their labels.

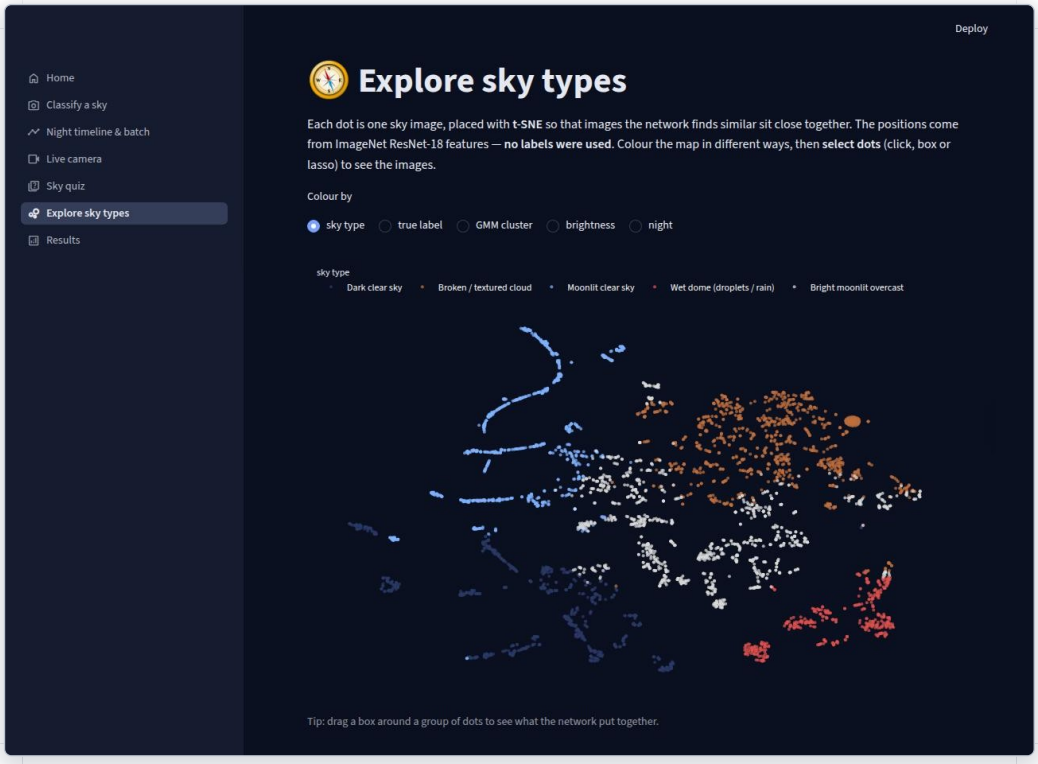


Figure 6.25 The Explore sky types page with the interactive t-SNE map of 4,000 frames, coloured by the five unsupervised sky types.

Below the map, one card per sky type shows its composition, its purity, a description and six example images. At the bottom of the page, a table lists the clustering scores of Chapter 4 (Table 4.1), so that the map, the examples and the numbers can be compared directly.



Figure 6.26 The five sky types with example frames: dark clear sky (98% pure), broken or textured cloud (92%), moonlit clear sky (97%), wet dome (100%) and bright moonlit overcast (81%). The counts give the number of cloudy and fine frames of the 4,000 clustering images in each type.

6.9 Results

Purpose: see the evaluation of every model interactively. **Input:** the decision threshold, and which training runs to show. **Output:** model comparison, per-night accuracy, Moon effect, smoothing, confusion matrices, training curves, baselines and limitations.

1. Click **Results**.
2. Move the **Decision threshold on P(clear)** slider. Every table and chart on the page is recomputed for the new threshold; AUC does not change, because it does not depend on the threshold.
3. In **Training**, choose which runs to display in the validation-loss chart.

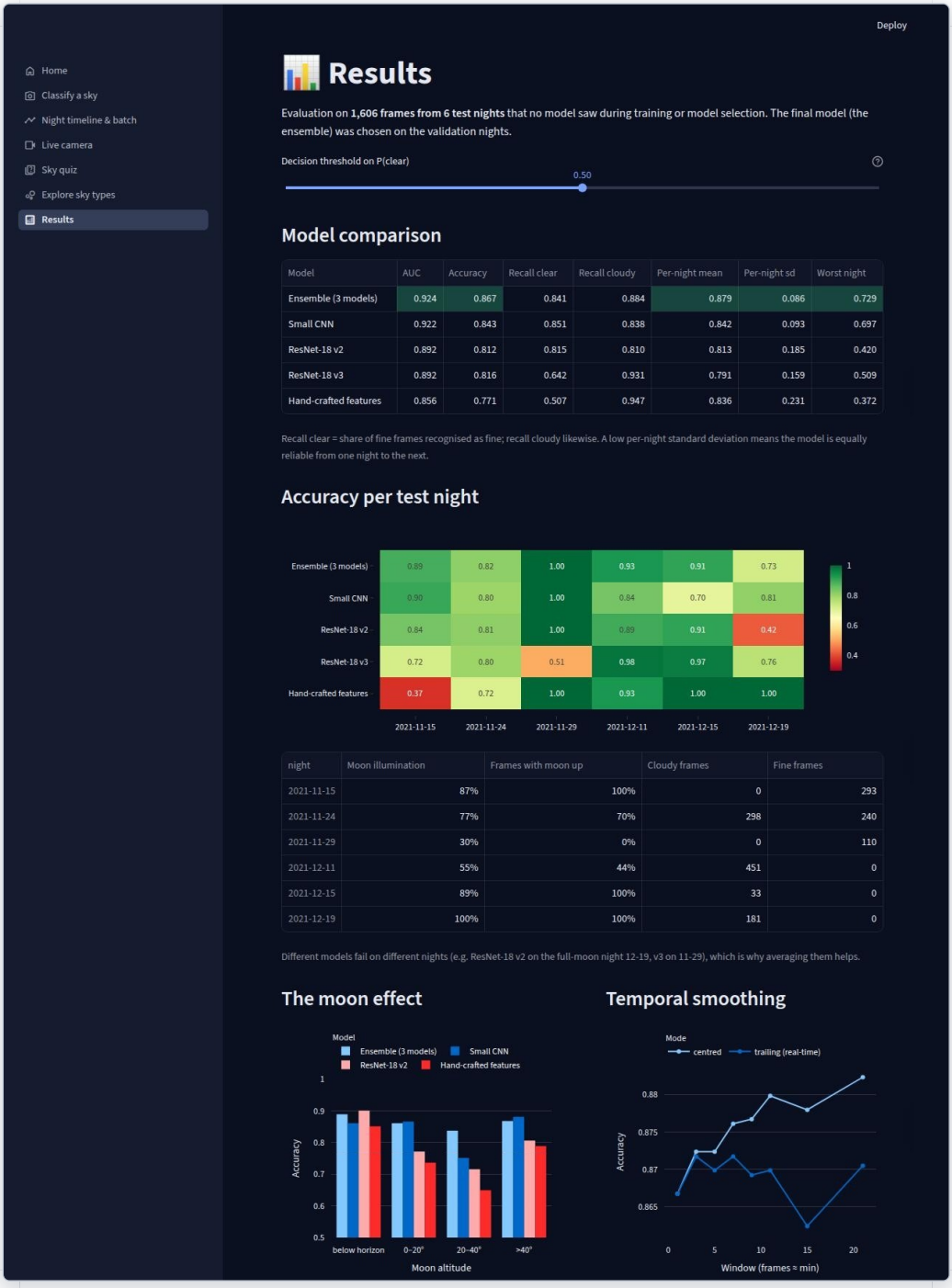


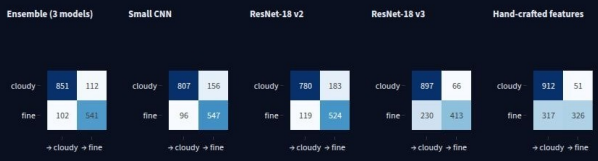
Figure 6.27 *(part 1 of 2, continued on next page)*

continued

All models do best with the moon below the horizon. The ensemble varies least across moon conditions, hand-crafted brightness features the most. Moon positions computed with astropy.

A rolling median over neighbouring frames gives only ~1 point: errors come in long runs (a systematic condition such as moonlight or a mislabelled block), not as isolated frames.

Confusion matrices



Rows: dataset label. Columns: prediction. For an observatory, 'cloudy → fine' (observing through clouds) and 'fine → cloudy' (lost observing time) have different costs.

Training: how much of the network should learn?



The ResNet runs train only the final layer first (frozen ImageNet backbone), then unfreeze more of the network. Validation loss jumps the moment deeper layers start learning: with ~24 training nights the network memorises night-specific conditions. Frozen BatchNorm, L2-SP and physical augmentation (v3) keep fine-tuning stable but do not beat the frozen backbone. The best checkpoint of each run (lowest validation loss) is the one kept.

Hand-crafted baseline

Feature (sky pixels only)	AUC
Mean sky brightness	0.654
Sky std (contrast)	0.741
Mean gradient (texture)	0.775
Star-like peaks	0.81
All 4 — logistic regression (CV by night)	0.858
All 4 — gradient boosting (CV by night)	0.87

Single physical features measured on the sky pixels only. Star count is best; brightness is weakest because the moon brightens clear and cloudy skies alike.

Validation (model selection)

Model	Val AUC	Val accuracy
Small CNN	0.953	0.854
ResNet-18 v2	0.919	0.906
ResNet-18 v3	0.927	0.892
Ensemble Small CNN + ResNet v2	0.959	0.901
Ensemble of all 3 (final)	0.954	0.908

Both ensembles beat every single model on validation, so the decision to ensemble was made before looking at the test nights.

What limits performance

- **Coarse labels.** Frames were labelled in time blocks of 1–2 h, while cloud cover changes within minutes. On the mixed night 2021-11-24, almost an hour of overcast frames is labelled *fine*; the model is confidently (and correctly) saying *cloudy*.
- **Two classes for a continuous sky.** Partly cloudy frames have no correct binary answer. The model's intermediate probabilities (0.3–0.7) are appropriate there.
- **Moonlight.** Bright moonlit skies are the hardest condition, and in this dataset moon phase is partly correlated with the class.
- **Few independent samples.** Frames within a night are highly correlated, so the effective sample size is closer to 32 nights than 8,806 images; results depend on which nights are in the test set.
- **Future work:** frame-level relabelling, three classes (clear / cloudy / rain) or cloud-fraction regression, and more nights — especially moonlit ones.

Figure 6.27 The complete Results page at the default threshold of 0.5. From top to bottom: model comparison (best values highlighted), accuracy per test night with Moon conditions, the Moon effect and temporal smoothing, confusion matrices for all five models, validation-loss curves of the four training runs, the hand-crafted baseline and validation tables, and what limits performance.

> **TIP**
> Raise the threshold to 0.7 and watch the confusion matrices: fewer cloudy frames are called clear, at the cost of more clear frames called cloudy. This is how an observatory would make the system more cautious.

6.10 Troubleshooting

Table 6.4 Common problems and their solutions.

Problem	Solution
"Python was not found" or `'python' is not recognized`	Install Python 3.11 or 3.12 and tick *Add python.exe to PATH*; on Windows also try `py` instead of `python`
`streamlit: command not found`	Activate the environment first (`conda activate skymonitor`), or use `python -m streamlit run streamlit_app.py`
Red box "Model files not found"	You started the application from the wrong folder: `assets/config.json` must exist inside the application folder
`error: externally-managed-environment` on Ubuntu	Do not install with the system Python; use a conda or virtual environment as in Section 6.1
"Port 8501 is already in use"	Close the other Streamlit window, or add `--server.port 8502` to the start command
The browser shows "CONNECTING" or cannot reach the page	The application stopped; start it again in the terminal
Google Drive: "Could not read the folder"	Share the folder as *Anyone with the link*
Google Drive: files skipped	The files are private or Google is limiting downloads; try fewer frames or wait a minute
Live camera: "Could not open camera"	Try index 1 instead of 0; test the stream address in VLC first; on Linux add yourself to the `video` group
Warning "looks unlike anything in the training data"	The image comes from a different camera or unusual conditions; treat the verdict with caution

Problem	Solution
USB camera works for a second, then "No such device"	The camera reconnected under a new number: use its `/dev/v4l/by-id/…-video-index0` path, plug it in directly and turn off USB power saving (Section 6.5.4)
Camera preview looks purple in ffplay	Add `-input_format mjpeg` to the ffplay command; the application is not affected
Weather & satellite: "Could not reach the weather service"	The page needs internet; check the connection or a firewall
Weather & satellite: only a daily satellite picture	No geostationary images exist for this longitude or date in NASA's service; this is expected outside Asia–Pacific and the Americas

6.11 Quick reference

Table 6.5 Sky Monitor AI at a glance.

I want to…	Go to	Do this
Check one image	Classify a sky	Upload, or paste a link
See where the clouds are	Classify a sky	Read the evidence map: red clear, blue cloudy
Analyse a night or a folder	Night timeline & batch	Choose the source, then Analyse
Analyse a Google Drive folder	Night timeline & batch	Google Drive link → List images → Download & analyse
Monitor the camera	Live camera	Choose the source → Grab one frame now → Monitoring on
Make the system more cautious	Classify a sky / Results	Raise the Clear threshold
Save results	Night timeline / Live camera	Download results (CSV) / Download log
Check the forecast and satellite picture	Weather & satellite	Choose a place → choose a time
Understand the model	Explore sky types, Results	Explore the map; read the evaluation

Table 6.6 The five sky conditions and what to do.

Condition	Meaning	Action
☀️ Clear	Clear sky	Open the dome
🌤️ Mostly clear	Small amount of cloud	Observe, monitor clouds
⛅ Mostly cloudy	Substantial cloud	Stand by
☁️ Overcast	Covered sky	Keep the dome closed
🌧️ Wet dome	Water on the camera dome	Close the dome

Glossary

Accuracy. The fraction of images classified correctly at a given decision threshold.

All-sky camera. A camera with a fisheye lens that photographs the whole visible sky in a single circular image.

AUC (area under the ROC curve). The probability that a randomly chosen clear image receives a higher score than a randomly chosen cloudy one. It does not depend on the decision threshold; 0.5 is chance, 1.0 is perfect.

Augmentation. Random changes applied to training images (rotation, flips, brightness, blur, noise) so that the network learns features that survive such changes.

Batch normalisation. A layer that normalises the activations of a network using statistics of the current batch, which stabilises training.

Class weights. Factors that make errors on the rarer class count more in the loss, compensating for class imbalance.

Cloud cover (estimated). In Sky Monitor AI, the fraction of the sky whose local probability of being clear is below 0.5.

Confusion matrix. A table of true classes against predicted classes, showing every kind of error separately.

Convolutional neural network (CNN). A neural network that applies small learned filters across an image, building up from edges and textures to larger patterns.

Ensemble. A combination of several models, here by averaging their probabilities.

Evidence map. A map showing which regions of an image push the decision towards clear or cloudy. Sky Monitor AI computes it exactly from local logits.

Feature map. The output of a convolutional layer: one value per position for each learned filter.

Fine-tuning. Continuing to train some or all of the layers of a pre-trained network on new data.

Frozen backbone. A pre-trained network whose layers are kept fixed while only a new final layer is trained.

ERA5 reanalysis. A reconstruction of past weather, hour by hour on a global grid, produced by combining a weather model with all available observations.

Geostationary satellite. A satellite that stays above one point of the equator and can therefore image the same region every few minutes, such as Himawari-9 over East Asia.

Grad-CAM. A method that weights the final feature maps by the gradient of a class score to show where a network looks.

GroupKFold. Cross-validation in which all images of one group, here one night, are kept in the same fold.

K-means. A clustering method that groups data around a chosen number of centres.

Label smoothing. Replacing hard targets of 0 and 1 with slightly softer values, which discourages over-confident predictions.

L2-SP. A regularisation that penalises the distance of fine-tuned weights from their pre-trained values.

Local logit. The decision a network would make if the whole image looked like one position of the final feature map.

Logit. The raw output of a network before conversion to a probability.

Mask (sky mask). A circle that keeps the sky and blanks out everything else, including the printed date.

ONNX. Open Neural Network Exchange, a framework-independent file format for trained networks.

Out-of-distribution. Describes an input unlike anything in the training data, for which predictions are unreliable.

Overfitting. Learning details of the training data that do not generalise to new data.

PCA (principal component analysis). A linear method that compresses data into the directions of largest variance.

Purity. In clustering, the fraction of images that belong to the majority label of their cluster.

ResNet-18. An 18-layer residual network, pre-trained on ImageNet, used here for transfer learning and for image embeddings.

Shortcut learning. Solving a task with a feature that is correlated with the label in the data but unrelated to the real task, such as a printed date.

Smoothing (temporal). Replacing each frame's probability with the median of its neighbours in time.

t-SNE. A method that places high-dimensional data on a plane so that similar items sit close together.

Transfer learning. Re-using a network trained on one task as the starting point for another.

Validation set. Data held out during training to choose models and settings, separate from the final test set.

References

1. Astropy Collaboration (2022). The Astropy Project: sustaining and growing a community-oriented open-source project and the latest major release (v5.0) of the core package. *The Astrophysical Journal*, 935, 167.
2. Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K. and Fei-Fei, L. (2009). ImageNet: a large-scale hierarchical image database. *Proceedings of CVPR 2009*.
3. Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M. and Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2, 665–673.
4. Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press.
5. He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of CVPR 2016*.
6. Hersbach, H. et al. (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146, 1999–2049.
7. Hubert, L. and Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2, 193–218.
8. Ioffe, S. and Szegedy, C. (2015). Batch normalization: accelerating deep network training by reducing internal covariate shift. *Proceedings of ICML 2015*.
9. Li, X., Grandvalet, Y. and Davoine, F. (2018). Explicit inductive bias for transfer learning with convolutional networks. *Proceedings of ICML 2018*.
10. Lin, M., Chen, Q. and Yan, S. (2014). Network in network. *Proceedings of ICLR 2014*.
11. Pedregosa, F. et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
12. Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*.
13. Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
14. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017). Grad-CAM: visual explanations from deep networks via gradient-based localization. *Proceedings of ICCV 2017*.

15. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J. and Wojna, Z. (2016). Rethinking the Inception architecture for computer vision. *Proceedings of CVPR 2016*.

16. van der Maaten, L. and Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.

17. Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014). How transferable are features in deep neural networks? *Advances in Neural Information Processing Systems 27*.

18. Zeiler, M. D. and Fergus, R. (2014). Visualizing and understanding convolutional networks. *Proceedings of ECCV 2014*.

19. Zhou, B., Khosla, A., Lapedriza, A., Oliva, A. and Torralba, A. (2016). Learning deep features for discriminative localization. *Proceedings of CVPR 2016*.

Weather and satellite data: Open-Meteo (open-meteo.com), weather forecasts and ERA5 reanalysis, CC BY 4.0; NASA Global Imagery Browse Services (GIBS), part of NASA's Earth Science Data and Information System (ESDIS), with imagery from Himawari-9 (Japan Meteorological Agency), GOES (NOAA) and VIIRS.

Software: Python, PyTorch, torchvision, scikit-learn, NumPy, pandas, Matplotlib, Astropy, ONNX and ONNX Runtime, Streamlit, Plotly, OpenCV, gdown.

Data: all-sky camera images of the Higashi-Hiroshima Observatory, Hiroshima University, November–December 2021, provided by Prof. Makoto Uemura.

Acknowledgements

I thank Prof. Makoto Uemura of Hiroshima University for providing the all-sky camera images of the Higashi-Hiroshima Observatory and for setting the project that started this work. I am grateful to the developers of the open-source software on which every step depended, from PyTorch and scikit-learn to Astropy and Streamlit, and to Google Colab for the computing resources used in training.

About the author

Muhammad Waqas developed the sky-condition classifier and the Sky Monitor AI application described in this book, from data exploration and model training to interpretation and deployment as an observatory tool.